Learn Harness Engineering

English coursebook PDF · generated 2026-10-29

Included sections

1. Welcome to Learn Harness Engineering `/en/`
2. Lecture 01. Strong Models Don't Mean Reliable Execution
   `/en/lectures/lecture-01-why-capable-agents-still-fail/`
3. Lecture 02. What a Harness Actually Is
   `/en/lectures/lecture-02-what-a-harness-actually-is/`
4. Lecture 03. Making the Repository the Single Source of Truth
   `/en/lectures/lecture-03-why-the-repository-must-become-the-system-of-record/`
5. Lecture 04. Split Instructions Across Files
   `/en/lectures/lecture-04-why-one-giant-instruction-file-fails/`
6. Lecture 05. Keeping Context Alive Across Sessions
   `/en/lectures/lecture-05-why-long-running-tasks-lose-continuity/`
7. Lecture 06. Make the Agent Initialize Before Every Work Session
   `/en/lectures/lecture-06-why-initialization-needs-its-own-phase/`
8. Lecture 07. Draw Clear Task Boundaries for Agents
   `/en/lectures/lecture-07-why-agents-overreach-and-under-finish/`
9. Lecture 08. Use Feature Lists to Constrain What the Agent Does
   `/en/lectures/lecture-08-why-feature-lists-are-harness-primitives/`
10. Lecture 9. Preventing Agents from Declaring Victory Too Early
    `/en/lectures/lecture-09-why-agents-declare-victory-too-early/`
11. Lecture 10. Only a Full Pipeline Run Counts as Real Verification
    `/en/lectures/lecture-10-why-end-to-end-testing-changes-results/`
12. Lecture 11. Making the Agent's Runtime Observable
    `/en/lectures/lecture-11-why-observability-belongs-inside-the-harness/`
13. Lecture 12. Leave a Clean Handoff at the End of Every Session
    `/en/lectures/lecture-12-why-every-session-must-leave-a-clean-state/`
14. Lecture 13. From Manual Prompting to Autonomous Loops
    `/en/lectures/lecture-13-loop-engineering/`
15. Lecture 14. From Single Loops to Graph Engineering
    `/en/lectures/lecture-14-graph-engineering/`
16. Welcome to Projects `/en/projects/`

17. Project 01. Prompt-Only vs. Rules-First: How Much Difference Does It Make
    /en/projects/project-01-baseline-vs-minimal-harness/
18. Project 02. Make the Project Readable and Pick Up Where You Left Off
    /en/projects/project-02-agent-readable-workspace/
19. Project 03. Keep the Agent Working Across Session Restarts
    /en/projects/project-03-multi-session-continuity/
20. Project 04. Use Runtime Feedback to Correct Agent Behavior
    /en/projects/project-04-incremental-indexing/
21. Project 05. Make the Agent Verify Its Own Work
    /en/projects/project-05-grounded-qa-verification/
22. Project 06. Build a Complete Agent Harness (Capstone)
    /en/projects/project-06-runtime-observability-and-debugging/
23. Project 07. Build Your First Automated Loop
    /en/projects/project-07-loop-engineering-first-loop/
24. Project 08. Draw Your Workflow as a Graph
    /en/projects/project-08-graph-engineering-first-graph/
25. English Resource Library /en/resources/
26. Template Guide /en/resources/templates/
27. Coding Agent Startup Flow /en/resources/reference/coding-agent-startup-flow
28. English Reference /en/resources/reference/
29. Initializer Agent Playbook /en/resources/reference/initializer-agent-playbook
30. Method Map /en/resources/reference/method-map
31. Prompt Calibration /en/resources/reference/prompt-calibration
32. OpenAI Advanced Pack /en/resources/openai-advanced/
33. AGENTS.md /en/resources/openai-advanced/repo-template/AGENTS
34. ARCHITECTURE.md /en/resources/openai-advanced/repo-template/ARCHITECTURE
35. Core Beliefs /en/resources/openai-advanced/repo-template/docs/design-docs/core-beliefs
36. Design Docs Index /en/resources/openai-advanced/repo-template/docs/design-docs/
37. DESIGN.md /en/resources/openai-advanced/repo-template/docs/DESIGN
38. Active Plans /en/resources/openai-advanced/repo-template/docs/exec-plans/active/
39. Completed Plans /en/resources/openai-advanced/repo-template/docs/exec-plans/completed/
40. Tech Debt Tracker
    /en/resources/openai-advanced/repo-template/docs/exec-plans/tech-debt-tracker
41. FRONTEND.md /en/resources/openai-advanced/repo-template/docs/FRONTEND

42. Database Schema /en/resources/openai-advanced/repo-template/docs/generated/db-schema
43. PLANS.md /en/resources/openai-advanced/repo-template/docs/PLANS
44. PRODUCT_SENSE.md /en/resources/openai-advanced/repo-template/docs/PRODUCT_SENSE
45. Product Specs Index /en/resources/openai-advanced/repo-template/docs/product-specs/
46. New User Onboarding
    /en/resources/openai-advanced/repo-template/docs/product-specs/new-user-onboarding
47. QUALITY_SCORE.md /en/resources/openai-advanced/repo-template/docs/QUALITY_SCORE
48. RELIABILITY.md /en/resources/openai-advanced/repo-template/docs/RELIABILITY
49. SECURITY.md /en/resources/openai-advanced/repo-template/docs/SECURITY
50. Advanced Repo Template /en/resources/openai-advanced/repo-template/
51. SOP: Chrome DevTools Validation Loop
    /en/resources/openai-advanced/sops/chrome-devtools-validation-loop
52. SOP: Encode Unseen Knowledge Into The Repo
    /en/resources/openai-advanced/sops/encode-knowledge-into-repo
53. OpenAI Advanced SOPs /en/resources/openai-advanced/sops/
54. SOP: Layered Domain Architecture
    /en/resources/openai-advanced/sops/layered-domain-architecture
55. SOP: Observability Feedback Loop
    /en/resources/openai-advanced/sops/observability-feedback-loop

Welcome to Learn Harness Engineering

Learn Harness Engineering is a course dedicated to the engineering of AI coding agents. We have deeply studied and synthesized the most advanced Harness Engineering theories and practices in the industry. Our core references include:

- OpenAI: Harness engineering: leveraging Codex in an agent-first world
- Anthropic: Effective harnesses for long-running agents
- Anthropic: Harness design for long-running application development
- Awesome Harness Engineering

Through systematic environment design, state management, verification, and control systems, this course teaches you how to make agentic coding tools like Codex and Claude Code truly reliable. It helps you build features, fix bugs, and automate development tasks by constraining your AI coding assistant with explicit rules and boundaries.

Get started

Choose your learning path to get started. The course is divided into theoretical lectures, hands-on projects, and a copy-ready resource library.

Lectures

Understand why strong models still fail and learn the theory behind effective harnesses.

Projects

Hands-on practice building a reliable agentic environment from scratch.

Resource Library

Copy-ready templates (AGENTS.md, feature_list.json) to use in your own repositories.

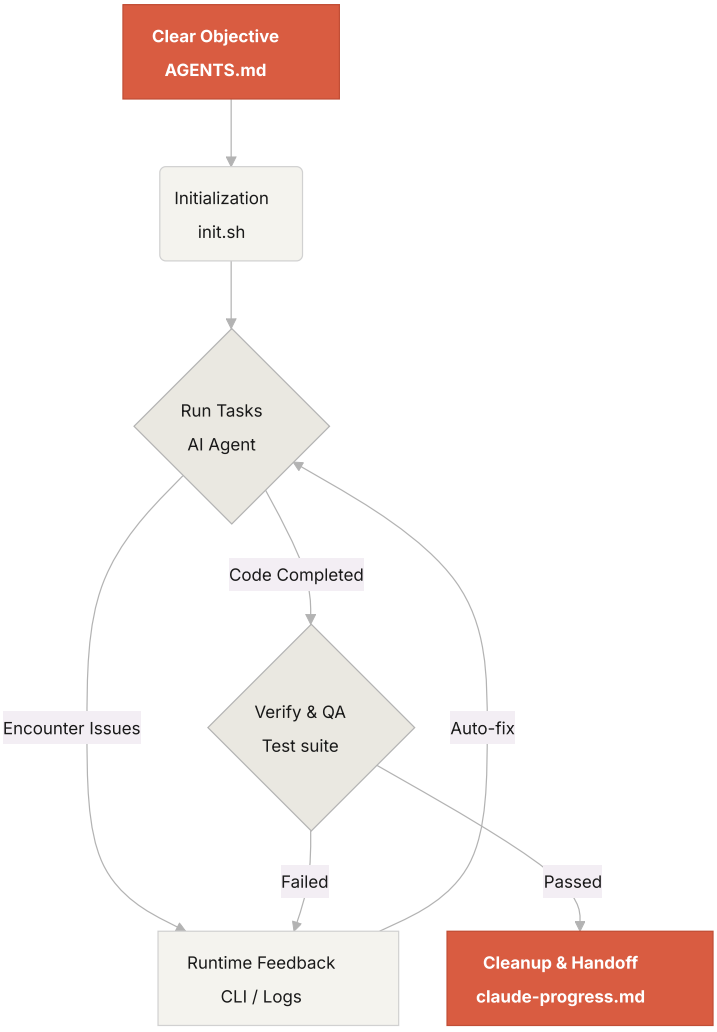
Frontier Harness Design Breakdowns

How Pi, Claude Code, Codex, and DeepSeek actually design their harnesses—mapped to the course framework.

The Core Mechanism of a Harness

A harness doesn't "make the model smarter"; rather, it establishes a closed-loop **working system** for the model. You can understand its core workflow through this simple diagram:

Tap to expand



What you will learn

Here are some of the key concepts you will master:

- **Constrain agent behavior** with explicit rules and boundaries.
- **Maintain context** across long-running, multi-session tasks.
- **Stop agents** from declaring victory too early.
- **Verify work** using full-pipeline tests and self-reflection.

- **Make runtime observable** and debuggable.

Next steps

Once you understand the core concepts, these guides help you go deeper:

- Lecture 01: Why Capable Agents Still Fail: Start with the theory behind harness engineering.
- Project 01: Baseline vs Minimal Harness: Walk through your first real task.
- Templates: Grab the minimal harness pack (AGENTS.md, feature_list.json, claude-progress.md) for your own projects.

中文版 →

> Code examples: code/ Practice project: Project 01. Prompt-Only vs. Rules-First: How Much Difference Does a Harness Make

Lecture 01. Strong Models Don't Mean Reliable Execution

On September 29, 2025, Anthropic reported Claude Sonnet 4.5 at 77.2% on SWE-bench Verified. That number sounds decent at first glance — but don't celebrate just yet. Those are carefully selected tasks with clear issue descriptions and ready-made test cases. Hand the agent your everyday requirements instead — vague specs, no existing tests, implicit business rules scattered across the codebase — and the pass rate only drops further. You hand off a task brimming with confidence, the agent runs for 20 minutes and tells you "all done," and you look at the code: it added a feature but broke the tests, fixed a bug but introduced new ones, and it's not even what you asked for.

When this happens, most people's first reaction is "the model isn't good enough — let me try a more expensive one." Before you reach for your wallet, consider that the problem might not be the model at all.

Same Horse, Different Fates

Anthropic compared the same prompt ("build a 2D retro game editor") and the same model (Opus 4.5) under two harnesses. First run: bare, no support — 20 minutes, $9; the app opened, but entities did not respond to input. Second run: full harness — a planner, generator, evaluator three-agent architecture — 6 hours, $200; the author could move an entity, but physics had rough edges and a wall blocked progress. Core gameplay worked; the app was not finished.

They didn't change the model. Opus 4.5 was still Opus 4.5. What changed was the tack.

OpenAI's 2025 harness engineering article puts it even more bluntly. They said that Codex in a well-harnessed repository goes from "unreliable" straight to "reliable." Note the wording — not "a bit better," but a qualitative leap. Harness here means **all the engineering infrastructure beyond the model weights.**

Where Agents Actually Get Stuck

The specific failure modes really come down to just a handful:

- **Vague requirements — the agent can only guess.** "Add a search feature" — that sentence means almost nothing. Search what? Full-text or structured queries? Should results be paginated? Highlighted? You didn't spell it out, so the agent has to guess. A correct guess is luck; a wrong one means rework that costs several times more than being specific would have in the first place.
- **Implicit conventions not written down — the agent has no way to comply.** Your whole team uses the new SQLAlchemy 2.0 syntax, but the agent writes 1.x code by default. All API endpoints must go through OAuth 2.0 authentication, but that rule only exists in your head and a Slack message from three months ago. The agent has no idea — it's not that it doesn't want to comply, it literally has never seen the rule.
- **Incomplete environment setup — the agent spends energy fixing the environment.** Incomplete dev setup, missing dependencies, wrong tool versions — the agent burns precious context window on `pip install` errors and Node version conflicts instead of doing the actual work you gave it.
- **No verification methods — the agent calls it done when it feels done.** No tests, no lint, or verification commands that were never communicated to the agent. The agent writes code, looks it over, decides it seems fine, and declares completion. Anthropic also observed an interesting phenomenon: when agents sense their context is running low, they rush to finish, skip verification steps, and choose a simple solution over the optimal one. They call this "context anxiety."
- **Cross-session state loss — every new session starts from scratch.** All discoveries from the previous session are lost. Every new session has to re-explore the project structure and re-understand the code organization. Agents without persistent state see failure rates spike sharply on tasks exceeding 30 minutes.

Key Terminology

With the scenarios above in mind, these concepts are no longer just jargon:

- **Capability Gap**: The huge gulf between model performance on benchmarks and performance on real tasks. Benchmark tasks come with clear issue descriptions, tests, and an environment; real tasks often require you to supply them.
- **Harness**: Everything outside the model — instructions, tools, environment, state management, verification feedback. If it's not model weights, it's harness. What we've been calling the "tack."
- **Harness-Induced Failure**: The model has sufficient capability, but the execution environment has structural defects. The retro-game comparison above is an example.
- **Verification Gap**: The gap between the agent's confidence in its output and actual correctness. The agent says "I'm done" when it's not — this is the most common failure mode.
- **Diagnostic Loop**: Execute, observe failure, attribute to a specific harness layer, fix that layer, re-execute. This is the core methodology of harness engineering.
- **Definition of Done**: A set of conditions that can be verified by command — tests pass, lint is clean, type checks pass. Without an explicit definition of done, the agent will invent its own.

When Things Fail, Fix the Harness First

There is really only one core principle: **When things fail, don't swap the model first — check the harness.** If the same model succeeds on similar, well-structured tasks, assume it's a harness problem.

What does this look like in practice? Attribute every failure to a specific layer. Don't just say "the model isn't good enough." Ask yourself: was the task unclear? Was context insufficient? Were there no verification methods? Map each failure to one of the five defense layers — task specification, context provision, execution environment, verification feedback, or state management. These are failure-diagnosis layers, not additional core concepts. Build this habit, and you'll find "the model isn't good enough" appearing less and less in your logs.

Then, write an explicit Definition of Done for every task. Don't say "add a search feature." Spell it out:

```
Completion criteria:
- New endpoint GET /api/search?q=xxx
- Supports pagination, default 20 items
- Results include highlighted snippets
- All new code passes pytest
- Type checking passes (mypy --strict)
```

Place an `AGENTS.md` file in the repo root, telling the agent about the project's tech stack, architectural conventions, and verification commands. This is the first step in harness engineering, and the one with the highest return on investment. One `AGENTS.md` file might be more effective than upgrading to a more expensive model — and that's not a joke.

From there, build a diagnostic loop. Don't treat failures as "the model being dumb again." Treat them as signals that your harness has exposed a defect. Each failure, identify the layer, fix it, and never fail that way again. After a few rounds, your harness gets stronger and agent performance stabilizes. A simple log is enough — for each task, did it succeed or fail, and which layer caused the failure. After a few rounds you'll see which layer is the bottleneck, and you can focus your energy there.

The Million-Line Experiment

In August 2025, three engineers at OpenAI started an experiment. The rules were simple: they wouldn't write code themselves — only Codex would. Starting from an empty git repository, five months later the repo contained roughly one million lines of code. Application logic, infrastructure, tooling, documentation — all agent-generated. The team later grew to seven engineers; over the period it merged about 1,500 PRs, an average of 3.5 per engineer per day.

Early progress was surprisingly slow. Codex wasn't bad — it just lacked tools and structures complete enough to drive toward high-level objectives. The three engineers gradually found the pattern: break large goals into small building blocks — design, code, review, test — let the agent assemble them one by one, then use those blocks to compose more complex tasks. Whenever something went wrong, the problem was almost never "not trying hard enough." It was always "what is the agent still missing, and can that missing capability be supplied in a way that is both understandable and executable?"

This experiment directly proves this lecture's core thesis: **the same model produces fundamentally different output in a bare environment versus one with a complete harness.** The model didn't change. The environment did.

> Source: OpenAI: Harness engineering: leveraging Codex in an agent-first world

---

A More Down-to-Earth Example

A team used Claude Sonnet to add new API endpoints to a mid-sized Python web app (FastAPI + PostgreSQL + Redis, ~15,000 lines of code).

Initially they gave only one sentence: "add user preferences endpoints under `/api/v2/users`." The result? The agent spent much of the session exploring the repo structure, produced code that looked reasonable but didn't follow the project's error handling patterns, used old SQLAlchemy syntax, and declared completion while the endpoint had runtime errors. The next session had to redo all the discovery work.

Later they added `AGENTS.md` (describing project architecture and tech stack versions), explicit verification commands (`pytest tests/api/v2/ && python -m mypy src/`), and architecture decision records. The same model then followed the project's conventions and checked its work with commands it could run.

They didn't change the model. They changed the harness.

Key Takeaways

- Model capability and execution reliability are two different things. Even a thoroughbred needs good tack.
- When things fail, check the harness first, then the model. Swapping models is the most expensive option — and more often than not, it's not even a model problem.
- Every failure is a signal: your harness has a structural defect. Find it and fix it.
- Don't just say "the model isn't good enough." Work through the five layers systematically: task not clearly defined, insufficient context, misconfigured environment, missing

verification, loss of state between sessions. Nine times out of ten the problem lives in one of those layers.

- One `AGENTS.md` file might be more effective than upgrading to a more expensive model.

Further Reading

- OpenAI: Harness Engineering — Leveraging Codex in an Agent-First World
- Anthropic: Effective Harnesses for Long-Running Agents
- HumanLayer: Skill Issue — Harness Engineering for Coding Agents
- SWE-bench Leaderboard
- Thoughtworks Technology Radar: Harness Engineering

Exercises

1. **Comparison experiment**: Pick a codebase you know well and a non-trivial modification task. First, run the agent with no harness support and record failures. Then add an `AGENTS.md` and explicit verification commands, and run again with the same agent. Compare the two results, attributing each failure to one of the five defense layers.

2. **Verification gap measurement**: Pick 5 coding tasks. After each task, record whether the agent claims completion, then verify actual correctness with independent tests. Calculate the proportion of times the agent claims done when it's actually not done — that's your verification gap. Then think: what verification commands would reduce this proportion?

3. **Diagnostic loop practice**: Find a task where the agent repeatedly fails in your project. Run once, record the failure. Attribute it to one of the five layers. Fix that layer. Run again. Repeat three to five rounds, recording improvements each time.

中文版 →

> Code examples: code/ Practice project: Project 01. Prompt-only vs. rules-first

Lecture 02. What a Harness Actually Is

The word "harness" gets thrown around a lot in AI coding agent circles, but most of the time, when people say "harness," they really just mean a prompt file. A prompt file is not a harness.

This lecture gives harness a precise, actionable definition — not an academic abstraction, but a framework you can put to work today. A harness consists of five subsystems: instructions, tools, environment, state, and feedback. Each subsystem has clear responsibilities and evaluation criteria.

Start with an Analogy

Imagine you are a newly hired engineer, dropped into a project with zero documentation. No README, no comments in the code, nobody tells you how to run tests, CI config is buried somewhere. Can you write good code? Maybe — if you are smart enough and patient enough. But you will spend an enormous amount of time on "figuring out what this project is about" rather than on "solving the problem."

An AI agent faces the exact same predicament, and it is even worse. You can at least ask a colleague. The agent can only see the files you put in front of it and the commands it can execute.

OpenAI frames the core principle of harness engineering as "the repo IS the spec" — all necessary context should live in the repository, delivered through structured instruction files, explicit verification commands, and clear directory organization. Anthropic's long-running agents documentation emphasizes state persistence, explicit recovery paths, and structured progress tracking. The two companies focus on different aspects, but they are saying the same thing: **everything in the engineering infrastructure outside the model determines how much of the model's capability actually gets realized.**

Look at a few tools you already know:

Claude Code embodies harness thinking. It reads `CLAUDE.md` from your repo, can run shell commands, executes in your local environment, maintains session history, and can run tests to see results. But if you do not tell it how to run tests, it has no way to verify whether it did things right.

Cursor follows similar logic. Its `.cursorrules` file is its instruction source, the terminal is its tool, and it can read your project structure and lint config. However, Cursor's state management is relatively weak — close the IDE and reopen it, and the previous context is gone.

Codex (OpenAI's coding agent) uses git worktrees to isolate each task's runtime environment, paired with a local observability stack (logs, metrics, traces), so every change is verified in an independent environment. It performs far better in repos with an `AGENTS.md` and clear verification commands than in "bare" repos.

AutoGPT is the cautionary tale. Its lack of structured state management causes context to accumulate endlessly during long tasks, and its lack of precise feedback mechanisms causes the agent to loop. Many people say AutoGPT "doesn't work," but really it is the harness that does not work.

Core Concepts

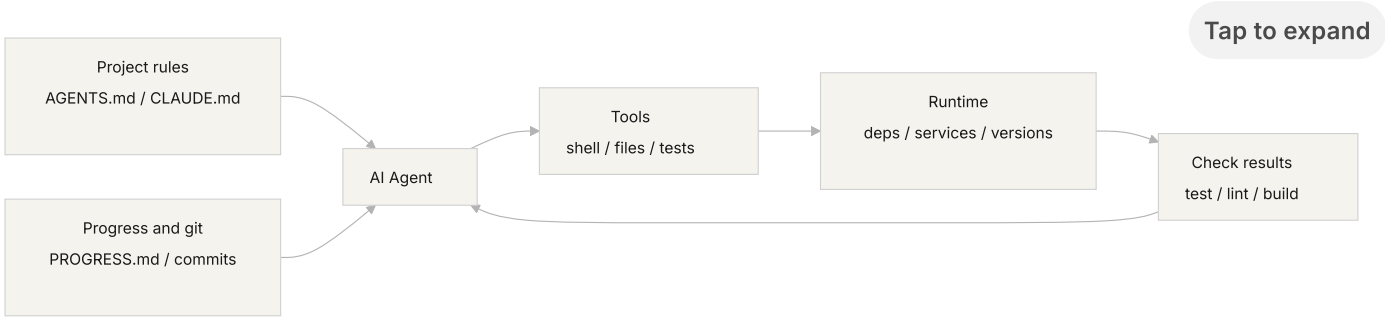
- **What is a harness**: Everything in the engineering infrastructure outside the model weights. OpenAI distills the engineer's core job into three things: designing environments, expressing intent, and building feedback loops. Anthropic directly calls their Claude Agent SDK a "general-purpose agent harness."
- **The repo is the single source of truth**: Anything the agent cannot see, for all practical purposes, does not exist. OpenAI treats the repo as the "system of record" — all necessary context must live there, delivered through structured files and clear directory organization.
- **Give a map, not a manual**: OpenAI's experience is that `AGENTS.md` should be a directory page, not an encyclopedia. Around 100 lines is enough. If it does not fit, split it into a `docs/` directory and let the agent read on demand.
- **Constrain, don't micromanage**: A good harness uses executable rules to constrain the agent, rather than enumerating instructions one by one. OpenAI says "enforce invariants,

don't micromanage implementation"; Anthropic found that agents confidently praise their own work, and the solution is to separate "the person who does the work" from "the person who checks the work."

- **Remove one at a time and observe**: To quantify each harness component's marginal contribution, remove them one at a time and see which removal causes the biggest performance drop. This tells you which components are most valuable right now, and it also reveals which ones are not yet contributing meaningfully. Anthropic used this method and discovered that as models get stronger, some components stop being critical — but new critical components always emerge.

The Five-Subsystem Harness Model

Back to the analogy. A harness has five subsystems:



Instruction subsystem: Create `AGENTS.md` (or `CLAUDE.md`) containing a project overview and purpose, tech stack and versions, first-run commands, non-negotiable hard constraints, and links to more detailed documentation.

Tool subsystem: Ensure the agent has sufficient tool access. Do not disable shell for "security reasons" — if the agent cannot even run `pip install`, how is it supposed to get anything done? But do not open everything either — follow the principle of least privilege.

Environment subsystem: Make the environment state self-describing. Use `pyproject.toml` or `package.json` to lock dependencies, `.nvmrc` or `.python-version` to specify runtime versions, and Docker or devcontainers to make the environment reproducible.

State subsystem: Long tasks must have progress tracking. Use a simple `PROGRESS.md` file recording: what is done, what is in progress, what is blocked. Update before each session ends; read when the next session starts.

Feedback subsystem: This is the highest-ROI subsystem. Explicitly list verification commands in `AGENTS.md`:

```
Verification commands:
- Tests: pytest tests/ -x
- Type check: mypy src/ --strict
- Lint: ruff check src/
- Full verification: make check (includes all above)
```

Missing any one of the five subsystems means an incomplete harness, and the agent will always feel awkward to use.

Quantifying harness component value: Use a "controlled variable exclusion test." Keep the model fixed, remove the five subsystems one at a time, and see which subsystem's removal causes the biggest performance drop. The component with the largest drop has the highest marginal contribution for the current task and is worth prioritizing. Whether to strengthen it depends on failure attribution, not just the size of the drop. Components with near-zero impact should not be dismissed outright: they may be redundant, poorly designed, or simply not exercised by the current task. This experiment answers "which component is most valuable right now" — it cannot, by itself, prove "where the bottleneck is." To truly locate a bottleneck, you must first examine failure records and attributions: was the task unclear, was context insufficient, was the environment unreproducible, was verification feedback missing, or was state management broken? Component ablation results can only serve as supporting evidence.

A Team's Real Story

A team used GPT-4o to develop a TypeScript + React frontend application (~20,000 lines of code). They went through four stages, which were essentially adding harness components one at a time:

Stage 1: Only a basic project description in the README. 1 out of 5 runs succeeded (20%). Main failures: chose the wrong package manager (npm vs yarn), did not follow component naming conventions, could not run tests.

Stage 2: Added `AGENTS.md` specifying tech stack versions, naming conventions, and key architecture decisions. Success rate rose to 60%. Remaining failures were mainly from environment issues and missing verification.

Stage 3: Listed verification commands in `AGENTS.md`: `yarn test && yarn lint && yarn build`. Success rate rose to 80%.

Stage 4: Introduced progress file templates where the agent recorded completed and incomplete work each run. Success rate stabilized at 80-100%.

Four iterations, the model did not change at all, and success rate went from 20% to near 100%. You did not switch to a better model — what changed was the harness.

Key Takeaways

- Harness = Instructions + Tools + Environment + State + Feedback. All five subsystems are essential.
- If it is not model weights, it is harness. Your harness determines how much of the model's capability gets realized.
- Among the five subsystems, the feedback subsystem usually has the lowest investment and highest return. Get your verification commands right first.
- Use "controlled variable exclusion tests" to quantify each subsystem's marginal contribution; to locate the real bottleneck, rely on failure records and attribution, not ablation alone.
- Harness rots like code does. Audit regularly, and pay down harness debt just like you pay down technical debt.

Further Reading

- OpenAI: Harness Engineering
- Anthropic: Effective Harnesses for Long-Running Agents
- HumanLayer: Harness Engineering for Coding Agents

- SWE-agent: Agent-Computer Interfaces
- Thoughtworks: Harness Engineering on Technology Radar

Exercises

1. **Five-tuple harness audit**: Take a project where you currently use an AI agent and do a complete audit using the five-tuple framework. Score each subsystem 1-5. Find the lowest-scoring subsystem, spend 30 minutes improving it, and then observe the change in agent performance.

2. **Controlled variable exclusion test**: Pick one model and one challenging task. Sequentially remove instructions (delete AGENTS.md), remove feedback (do not provide verification commands), remove state (no progress files) — removing only one at a time, and measure the performance drop. Use the results to rank each subsystem's marginal value for the current task. If you want to find the bottleneck, you must also record failure logs and do root-cause attribution alongside the ablation.

3. **Affordance analysis**: Find a scenario in your project where the agent "wants to do something but can't" (e.g., knows it should use parameterized queries but does not know your project's ORM patterns). Analyze whether this is a Gulf of Execution (does not know how to operate) or a Gulf of Evaluation (does not know whether it did things right), then design a harness improvement to bridge the gap.

中文版 →

> Code examples: code/ Practice project: Project 02. Make the Agent Read the Project and Pick Up Where It Left Off

Lecture 03. Making the Repository the Single Source of Truth

Your team's architecture decisions are scattered across Confluence, Slack, Jira, and a few senior engineers' heads. For humans this barely works — you can ask a colleague, search chat logs, dig through documentation, and if all else fails, you can corner someone in the break room. But for an AI agent, information that's not in the repository simply does not exist.

This isn't an exaggeration. An agent has only three sources of input: system prompts and task descriptions, file contents from the repository, and tool execution output. Your Slack history, Jira tickets, Confluence pages, and that architecture decision you hashed out with a colleague on Friday afternoon — the agent can't see any of it. It can't "go ask someone" or "search the chat logs." Its entire working world is the repository itself. Everything outside, it knows nothing about.

So the real question is: are you going to give it a map that's good enough?

What Belongs on the Map

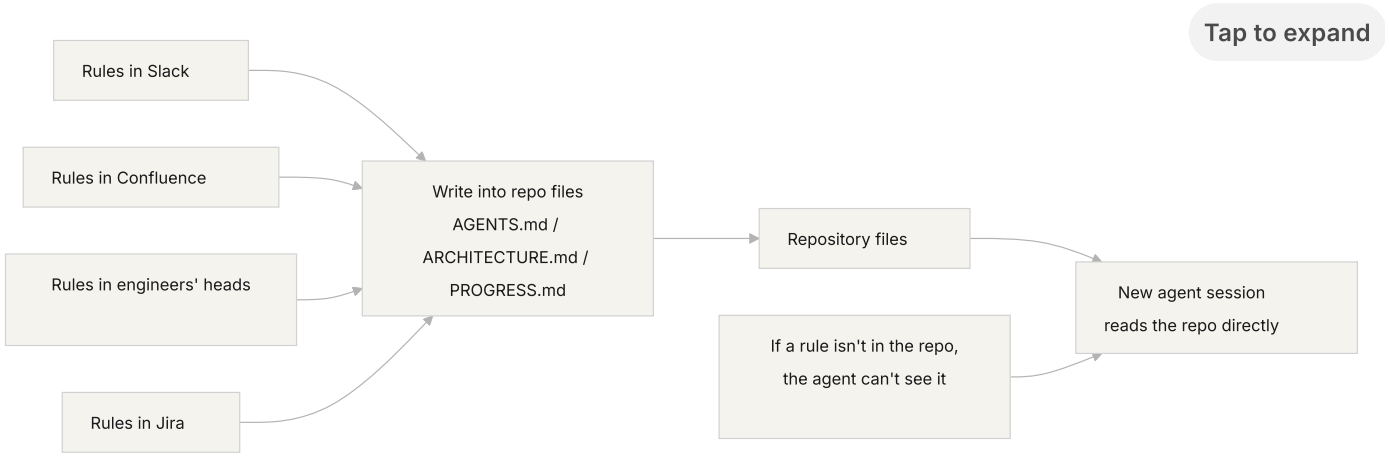
OpenAI states this plainly in their harness engineering article: **information that doesn't exist in the repo, doesn't exist for the agent.** They call this the "repo as spec" principle — the repository itself is the highest-authority specification document.

Anthropic's long-running agents documentation echoes a similar point: persistent state is a necessary condition for long-task continuity, and cross-session knowledge recoverability directly determines task success rates. And this state must exist in the repository — because that's the only stable, reliably accessible storage the agent has.

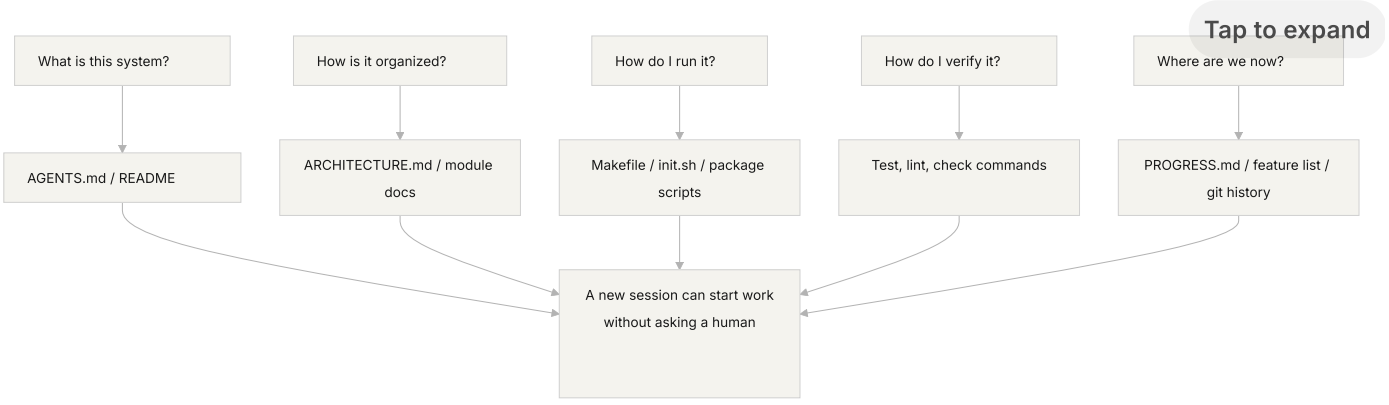
You might think: "Our team is small, knowledge lives in everyone's heads, and it works fine." True enough — for humans. But if you want to use an agent, you have to accept one fact: the agent can't ask people. Everything it needs to know must be written down, placed somewhere it can find it.

This isn't a "write more documentation" problem — it's a "put decision information in the right place" problem. A 50-line `ARCHITECTURE.md` sitting in the `src/api/` directory is far more useful than a 500-page design document in Confluence that nobody maintains. Proximity matters more than length, because information is only truly useful when it's right at hand the moment you need it.

Knowledge Visibility



How do you test whether your map is good enough? Run a "fresh session test": open a brand-new agent session, give it only the repository contents, and see if it can answer five basic questions.



If it can't answer, the map has blank spots. Where the map is blank, the agent has to guess — wrong guesses become bugs, excessive guessing wastes context. And every new session has to guess all over again. The cost of guessing is always far higher than the cost of drawing the map properly in the first place.

Core Concepts

- **Knowledge Visibility Gap**: The proportion of total project knowledge that's NOT in the repository. The bigger the gap, the higher the agent's failure rate. You can estimate it like this: count all the implicit knowledge about this project that lives in people's heads, then see how much of it actually made it into the repo. The difference is your visibility gap.
- **System of Record**: The code repository as the authoritative source for project decisions, architecture constraints, execution state, and verification standards. The repo has the final say — nowhere else counts. If the information "this road is closed" only lives in Old Zhang's head, then every single time you have to ask Old Zhang. Write it in the repo, and nobody has to ask.
- **Fresh Session Test**: The five questions from the previous section. How many the agent can answer is how complete your map is.
- **Discovery Cost**: How much context budget the agent burns to find a single key piece of information in the repo. The more hidden the information, the higher the discovery cost, and the less budget remains for the actual task. Critical information should be placed where the agent sees it first — not buried ten directory levels deep.
- **Knowledge Decay Rate**: The proportion of knowledge entries in the repo that become stale per unit of time. Documentation drifting out of sync with code is the biggest enemy — worse than no documentation at all is documentation that's out of date.
- **ACID Analogy**: Applying database transaction principles (Atomicity, Consistency, Isolation, Durability) to agent state management. We'll expand on this below.

How to Draw a Good Map

Principle 1: Knowledge lives next to code. A rule about API endpoint authentication belongs next to the API code, not buried in a giant global document. Put a short doc in each module directory explaining that module's responsibilities, interfaces, and special constraints. The module directory itself is a natural index — when the agent reaches the code, it also reaches the constraints, no searching required.

Principle 2: Use a standardized entry file. `AGENTS.md` (or `CLAUDE.md`) is the agent's "landing page." It doesn't need to contain all information, but it must let the agent quickly answer three questions: "What is this project," "How do I run it," and "How do I verify it." 50–100 lines is enough.

Principle 3: Minimal but complete. Every piece of knowledge should have a clear use case. If removing a rule doesn't affect the agent's decision quality, that rule shouldn't exist. But every question from the fresh session test must have an answer. This is an ongoing balance to maintain — not too much, not too little, just enough.

Principle 4: Update with code. Bind knowledge updates to code changes. The simplest approach: put architecture docs in the corresponding module directory. When you modify code, you naturally notice the doc. After code changes, CI can remind you to check whether the docs need updating.

Concrete repo structure:

```
project/
├── AGENTS.md              # Entry: project overview, run commands, hard constrai
├── src/
│   ├── api/
│   │   ├── ARCHITECTURE.md  # API layer architecture decisions
│   │   └── ...
│   ├── db/
│   │   ├── CONSTRAINTS.md   # Database operation hard constraints
│   │   └── ...
│   └── ...
├── PROGRESS.md             # Current progress: done, in-progress, blocked
└── Makefile                # Standardized commands: setup, test, lint, check
```

Managing Agent State with ACID Principles

This analogy comes from database transaction management. You might feel like this is overcomplicating things, but it actually provides a very practical framework:

- **Atomicity**: A "logical operation" (say, adding an endpoint and updating its tests) is committed as a whole only once it's complete and verified. A failed or abandoned attempt

gets discarded, not partially merged. All or nothing.

- **Consistency**: Define "consistent state" verification predicates — all tests pass, lint reports zero errors. The agent runs verification after each operation; inconsistent intermediate states should not be committed. After an operation, the system should be in a verifiably correct state.
- **Isolation**: When multiple agents work concurrently, design state files to avoid race conditions. Simple approach: each agent uses its own progress file, or use git branches for isolation. Concurrent writes to the same file are a common source of trouble.
- **Durability**: Critical project knowledge lives in git-tracked files. Temporary state can stay in session memory, but knowledge that must survive across sessions has to be written to files. What's in your head doesn't count — only what's written down counts.

OpenAI: Repository Knowledge as the System of Record

OpenAI describes keeping architecture, design decisions, execution plans, and progress in versioned repository files while developing an internal product with Codex. Knowledge in chat threads or external documents cannot guide the agent unless it is made accessible in its working context. The team organized this knowledge into a structured docs directory, with checks for freshness and cross-links.

This is a documented engineering practice, not a controlled measurement of how much repository documentation improves success rates.

OpenAI

OpenAI reports roughly 1,500 opened and merged PRs over five months, driven by a small team of three engineers using Codex, and an average throughput of 3.5 PRs per engineer per day. The team later grew to seven. These are reported production outcomes for the whole engineering setup; the article does not isolate the contribution of repository documentation.
OpenAI: Harness engineering

Key Takeaways

- Knowledge not in the repo doesn't exist for the agent. Putting critical decision information into the repository is the most fundamental harness investment — draw a good map so you don't get lost.
- Use the "fresh session test" to evaluate repo quality: can a brand-new session answer five basic questions using only repo contents?
- Knowledge should be near code, minimal but complete, and updated together with code. It's not about writing more docs — it's about putting information in the right place.
- Use ACID principles for agent state: atomic commits, consistency verification, concurrency isolation, and durable critical knowledge.
- Knowledge decay is the biggest enemy. Out-of-date documentation is more dangerous than no documentation at all — it sends the agent in the wrong direction while the agent thinks it's on the right track.

Further Reading

- OpenAI: Harness Engineering
- Anthropic: Effective Harnesses for Long-Running Agents
- Infrastructure as Code — Martin Fowler
- ADR: Architecture Decision Records
- The Twelve-Factor App

Exercises

1. **Fresh session test**: Open a completely fresh agent session in your project (provide no verbal context whatsoever), let it see only the repository contents, then ask it five questions: What is this system? How is it organized? How do I run it? How do I verify it?

What's the current progress? Record which ones it can't answer, then improve the repo until it can answer all of them.

2. **Knowledge externalization quantification**: List all decisions and constraints important to development work in your project. Mark each item as inside or outside the repo. Calculate your knowledge visibility gap (the proportion of items outside the repo). Make a plan to bring the gap below 10%.

3. **ACID assessment**: Evaluate your project's state management using this lecture's ACID analogy. Atomicity — can agent operations be cleanly rolled back? Consistency — does the repo have "consistent state" verification? Isolation — do multiple concurrent agents step on each other's toes? Durability — is all cross-session knowledge properly persisted?

中文版 →

> Code examples: code/ Practice project: Project 02. Agent-readable workspace

Lecture 04. Split Instructions Across Files

You started taking harness engineering seriously — good. You created an `AGENTS.md` and packed in every rule, constraint, and lesson learned you could think of. One month later the file had ballooned to 300 lines, two months 450, three months 600. Then you notice the agent's performance is actually getting worse: on a simple bug fix, the agent burns through huge amounts of context processing irrelevant deployment instructions; a critical security constraint buried at line 300 gets ignored outright; three contradictory code style rules mean the agent picks one at random each time.

This is the "giant instruction file" trap. Everything seems useful, so you cram it all in, and finding one specific rule means rifling through the entire file. You wrote 600 lines, but only a third of it is relevant to the task at hand.

The Vicious Cycle at the Root

The most common vicious cycle goes like this: the agent makes a mistake, you say "add a rule to prevent this," you add it to AGENTS.md, and it works — temporarily. Then the agent makes a different mistake, so you add another rule. Repeat until the file bloats out of control.

This is a perfectly natural reaction. "Add a rule" every time something goes wrong feels reasonable. But the cumulative effect is disastrous. Let's look at exactly what goes wrong.

Context budget gets eaten alive. The agent's context window is finite. Say your agent has a 200K token window (Claude's standard). A bloated instruction file might consume 10-20K tokens. Seems like there's still plenty of headroom? But a complex task might need to read dozens of source files, tool execution output also takes up context, and conversation history keeps accumulating. By the time the agent actually needs to understand the code, the budget is already exhausted.

Lost in the middle. The "Lost in the Middle" paper (Liu et al., 2023) clearly demonstrated that LLMs utilize information in the middle of long texts significantly less effectively than at the

beginning or end. Your AGENTS.md is 600 lines long, and line 300 says "all database queries must use parameterized queries" — that's a security hard constraint. But it's buried in the middle, and the agent will almost certainly ignore it.

Priority conflicts. The file mixes non-negotiable hard constraints ("never use eval()"), important design guidelines ("prefer functional style"), and a specific historical lesson ("fixed a WebSocket memory leak last week, watch for similar patterns"). These three rules have completely different importance levels, but in the file they all look identical. The agent has no reliable signal to distinguish which is a red line and which is merely a suggestion.

Maintenance decay. Large files are inherently hard to maintain. Outdated instructions rarely get deleted, because the consequences of deletion are uncertain ("maybe something else depends on this rule?"), but adding new instructions feels cost-free. The result: the file only grows, never shrinks, and the signal-to-noise ratio steadily declines. This is the same problem as technical debt accumulation in software.

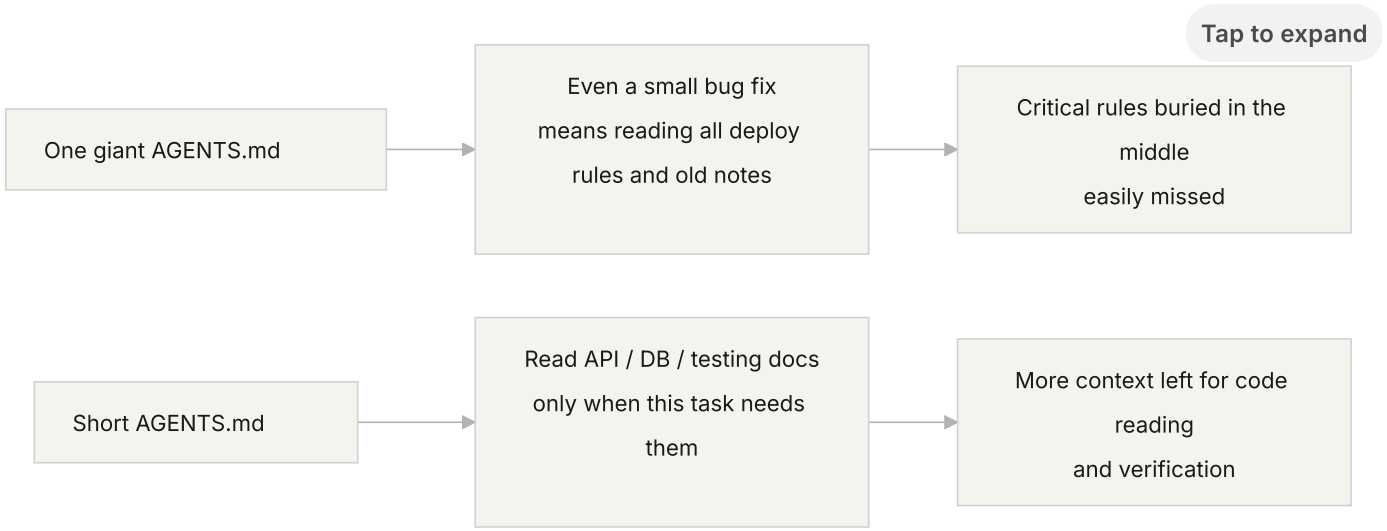
Contradiction accumulation. Instructions added at different times start contradicting each other — one says "use TypeScript strict mode," another says "some legacy files are allowed to use any." The agent picks one at random each time.

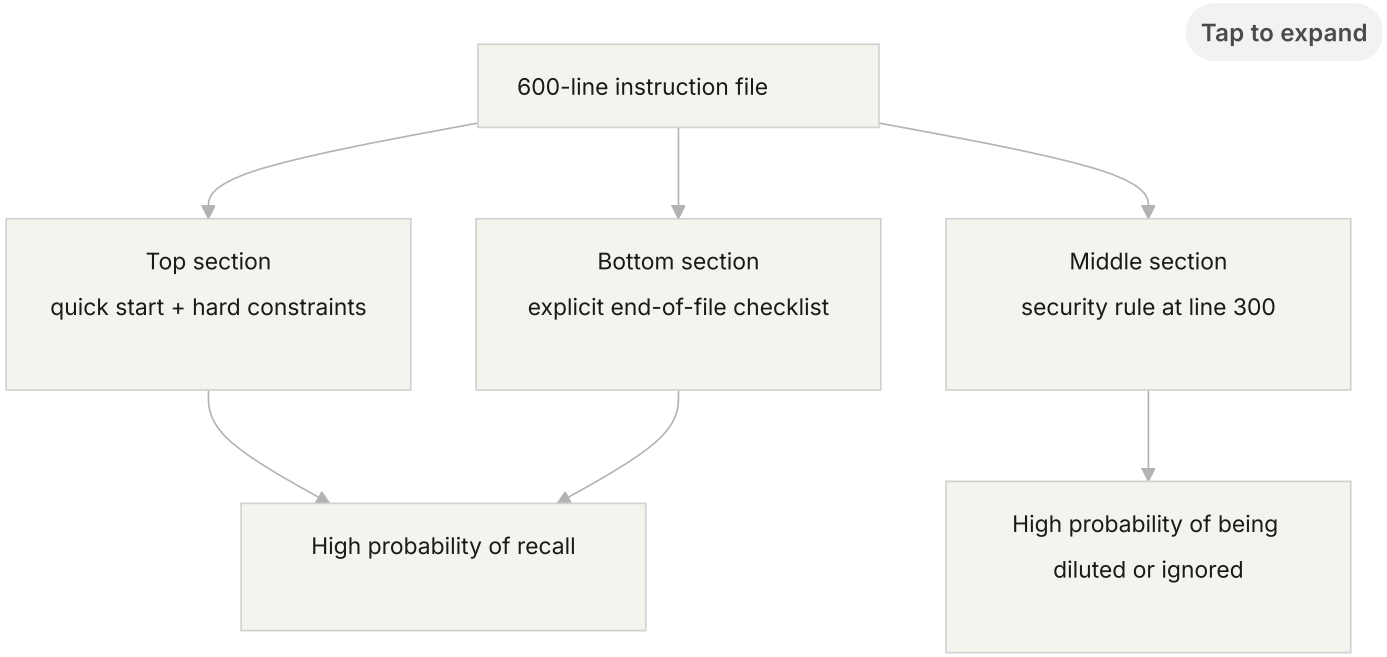
Core Concepts

- **Instruction Bloat**: When an instruction file occupies 10-15% of the context window, it starts crowding out budget for code reading and task reasoning. A 600-line `AGENTS.md` might consume 10,000-20,000 tokens — that's 8-15% of a 128K window.
- **Lost in the Middle**: Information in the middle of long texts is easily overlooked. Liu et al.'s 2023 research showed that LLMs utilize information in the middle of long texts significantly less effectively than information at the beginning or end. A critical constraint buried at line 300 of a 600-line file has a very high probability of being ignored.
- **Instruction Signal-to-Noise Ratio (SNR)**: The proportion of instructions in a file that are relevant to the current task. Being forced to read 50 lines of deployment instructions during a bug fix — that's low SNR.
- **Entry File**: A short entry file whose purpose is to guide the agent toward more detailed documentation, rather than containing everything itself. 50-200 lines is sufficient.

- **Reveal on Demand**: Give overview information first, detailed information when needed. Good harness design is like good UI design — don't dump all options on the user at once.
- **Can't Tell What Matters**: When all instructions appear in the same format and location, the agent cannot distinguish non-negotiable hard constraints from suggestive soft guidelines.

Instruction Architecture





How to Split

Core principle: keep frequently-needed information at hand, tuck away occasionally-needed information, and don't carry what you'll never use.

The entry file `AGENTS.md` stays at 50-200 lines, containing only the most essential items: a project overview (one or two sentences that make it clear what this is), first-run commands (`make setup && make test`), global hard constraints (no more than 15 non-negotiable rules), and links to topic documents (one-line description plus applicability condition).

```markdown
# AGENTS.md

## Project Overview
Python 3.11 FastAPI backend, PostgreSQL 15 database.

## Quick Start
- Install: `make setup`
- Test: `make test`
- Full verification: `make check`

## Hard Constraints
- All APIs must use OAuth 2.0 authentication
- All database queries must use SQLAlchemy 2.0 syntax
- All PRs must pass pytest + mypy --strict + ruff check

## Topic Docs
- API Design Patterns (`docs/api-patterns.md`) — Required reading when adding end
- Database Rules (`docs/database-rules.md`) — Required when modifying database op
- Testing Standards (`docs/testing-standards.md`) — Reference when writing tests
```

Each topic document is 50-150 lines, organized by subject in the `docs/` directory or next to the corresponding module. The agent only reads them when needed. Think of it like packing cubes for your luggage — underwear in one cube, toiletries in another, chargers in a third. Finding what you need doesn't require emptying the entire bag.

Some information is better placed directly in the code — type definitions, interface comments, explanations in config files. The agent naturally sees these when reading code, so there's no need to duplicate them in instructions.

Every instruction should document its source ("why was this rule added?"), applicability condition ("when is this rule needed?"), and expiry condition ("under what circumstances can this rule be removed?"). Audit regularly and delete outdated, redundant, or contradictory entries. Manage your instructions the way you manage code dependencies — unused dependencies should be removed, otherwise they only slow the system down.

If an instruction absolutely must be in the entry file, put it at the top or bottom, never in the middle. The "lost in the middle" effect tells us that LLMs utilize information at the extremes of a long text significantly better than in the center. But the better approach is to move instructions into topic documents for on-demand loading.

Both OpenAI and Anthropic implicitly endorse this split approach. OpenAI says entry files should be "short and routing-oriented," while Anthropic says control information for long-running agents should be "concise and high-priority." Both are saying the same thing: don't stuff everything into a single file.

OpenAI: A Short Entry File and Linked Documentation

OpenAI reports that a single large AGENTS.md crowded out task context, blurred priorities, accumulated stale rules, and was difficult to verify. The team instead used an entry file of roughly 100 lines as a map to a structured docs directory. Dedicated linters and CI checks maintained the knowledge base. The article does not report before-and-after task success or security-compliance percentages for this change. OpenAI: Harness engineering

The benefit depends on the content and task. An ETH Zurich study found that repository context files did not generally improve task success in its evaluated settings and increased inference cost by over 20%. Its recommendation is to keep human-written requirements minimal. A shorter file is not a guarantee of improvement; test the instructions on the tasks they are meant to support. ETH Zurich: Evaluating AGENTS.md

A paired study used gpt-5.2-codex on 124 PR-derived tasks from 10 repositories, comparing the same task and repository snapshot with and without AGENTS.md. Table 1 reports median runtime falling from 98.57 s to 70.34 s (28.64% lower), and median output tokens from 2,925 to 2,440 (16.58% lower). Tasks changed at most 100 lines across at most five files. This

measures efficiency, not the effect of splitting a large file: full functional correctness was outside the study's scope. Lulla et al., Table 1

Key Takeaways

- "Add a rule" is short-term pain relief and long-term poison. Before adding any rule, consider whether it belongs in a topic document instead.
- The entry file is a router, not an encyclopedia. 50-200 lines — overview, hard constraints, and links only.
- Leverage the "lost in the middle" effect: put important information at the top or bottom, and move less critical items to topic documents.
- Manage instruction bloat the way you manage technical debt. Regular audits, and every instruction should have a source, applicability condition, and expiry condition.
- After splitting, SNR improves and the agent spends more of its context budget on the actual task instead of processing irrelevant instructions.

Further Reading

- OpenAI: Harness Engineering
- Anthropic: Effective Harnesses for Long-Running Agents
- Lost in the Middle: How Language Models Use Long Contexts
- HumanLayer: Harness Engineering for Coding Agents
- Nielsen Norman Group: Progressive Disclosure
- ETH Zurich: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?: Study of repository context files; task success, inference cost, and the recommendation to keep requirements minimal. See the abstract and conclusion.
- Lulla et al., Table 1

Exercises

1. **SNR Audit**: Take your current entry instruction file and list all instruction entries. Pick 5 different common task types and mark whether each instruction is relevant to that task. Calculate the SNR for each task type. Instructions that are noise for most tasks should be moved to topic documents.

2. **Reveal on Demand Refactor**: If you have an instruction file over 300 lines, split it into: (a) an entry file under 100 lines, (b) 3-5 topic documents. Run the same set of tasks (at least 5) before and after refactoring, and compare success rates.

3. **Lost in the Middle Verification**: In a long instruction file, place a critical constraint at the top, middle, and bottom respectively, running the same task set each time (at least 5 runs per position). See if there's a difference in compliance rate. You might be surprised by how strong the position effect is.

中文版 →

> Code examples: code/ Practice project: Project 03. Multi-session continuity

Lecture 05. Keeping Context Alive Across Sessions

You ask Claude Code to implement a complete feature. It runs for 30 minutes, does most of the work, but context is running low. You start a new session to continue — and discover it doesn't remember what decisions were made last time, why option A was chosen over option B, which files were already modified, or what state the tests are in. It spends 15 minutes re-exploring the project, and might take an inconsistent approach from last time.

This is the real dilemma AI coding agents face in cross-session tasks. This lecture explains why agents "lose the thread" during long tasks, and how structured state persistence lets a new session quickly pick up where the last one left off.

Context Windows: Not Infinite

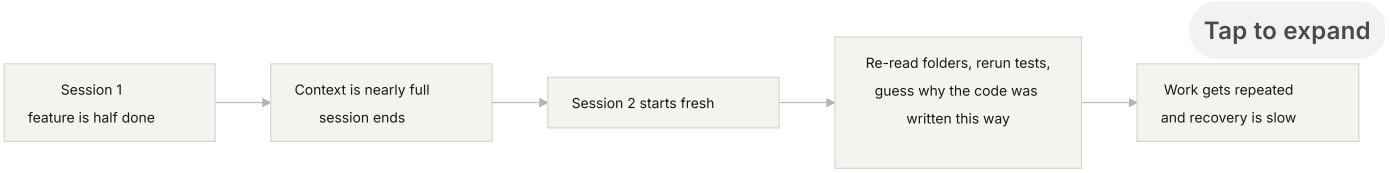
Context windows are finite. This isn't a problem that model upgrades can solve — even if window sizes grow to 1M tokens, complex tasks will still exhaust them. Agents aren't just generating code; they're understanding codebases, tracking their own decision history, processing tool output, and maintaining conversation context. All this information grows faster than window expansion.

A deeper problem: the information agents produce isn't uniformly important. Intermediate reasoning steps contain the "why" of decisions — why option A was chosen over B, why this library instead of that one, why a particular optimization was skipped. The final output only contains the "what" — the code itself. Compaction strategies usually preserve the latter but lose the former. The next session sees the code but doesn't know why it's written that way, and might "optimize" away a deliberate design decision.

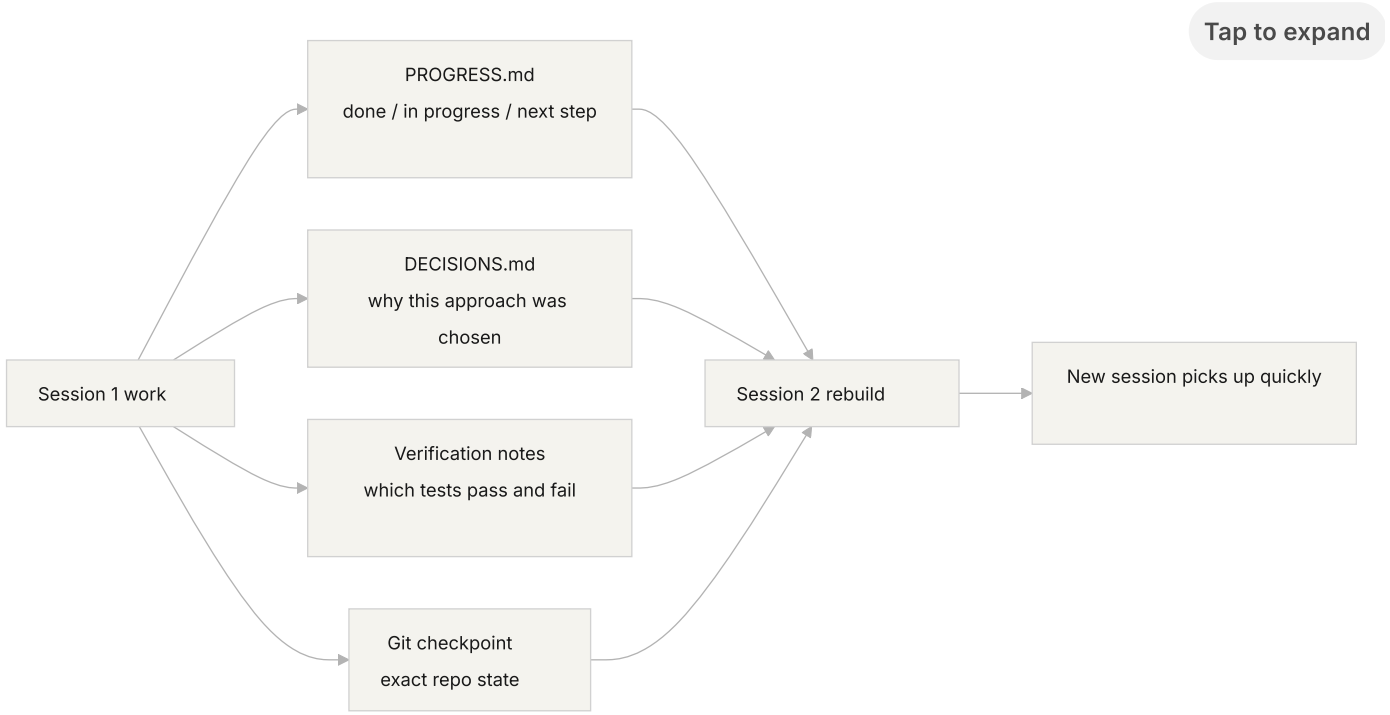
Anthropic observed something interesting in their long-running agent research: when agents sense context is running low, they exhibit a "rushed finish" behavior — rushing to finish current work, skipping verification steps, or choosing a simple solution over the optimal one. Anthropic calls this "context anxiety."

Session Continuity Flow

Without state persistence files, every new session has to start from scratch:



With state persistence files, new sessions can pick up quickly:



Core Concepts

- **Context windows are finite**: No matter what window size is claimed (128K, 200K, 1M), long tasks will eventually exhaust them. After exhaustion, either compaction (losing information) or reset (starting a new session) is required — both lose something.
- **State persistence files**: Persisted state files that let a new session unambiguously resume where the last one left off. The most basic form includes progress logs, verification records, and next actions.

- **Rebuild cost**: The time a new session needs to reach an executable state. A good harness leaves progress, verification results, and next actions in files, so the new session starts from a written state.
- **Drift**: The gap between the agent's understanding and the actual state of the code repository. Every session boundary introduces drift; without control, it compounds session after session.
- **Context anxiety**: A phenomenon observed by Anthropic — agents exhibit rushed finish behavior when approaching context limits, ending tasks early to avoid information loss. At its core, it's an irrational resource anxiety.
- **Compaction vs. reset**: Compaction summarizes context within the same session (keeps "what," may lose "why"); reset opens a new session rebuilding from persisted state (clean but depends on artifact completeness).

What Happens When Continuity Breaks

The previous session spent significant context budget analyzing three approaches and choosing option B. This session's agent doesn't know about that analysis and might re-decide based on incomplete information — potentially choosing option A. Same information, different conclusion, because the decision-making context is gone.

Even worse is duplicate work. The agent isn't sure whether certain work was already completed and does it again. Or worse — does half of it, discovers a conflict with the existing implementation, and has to rework. Without progress records, the new session has no idea what's already been done.

Over several sessions, the implementation direction may have silently drifted from the original requirements. Each new session has a slightly different understanding of the project goals. Each deviation compounds on the last, and the final result may be far removed from the original intent.

There's also the verification gap. The previous session's verification results (which tests pass, which fail, why they fail) weren't recorded. The new session has to re-run all verification to understand the current state. Every session re-diagnoses from scratch, every time wasting precious context.

Both OpenAI and Anthropic emphasize structured state persistence in their documentation. OpenAI's harness engineering article treats the repository as an "operational record" — every operation's results should leave traceable evidence in the repo. Anthropic's long-running agents documentation specifically recommends "handoff files" — structured documents containing current state, known issues, and next actions.

Practical Approaches to State Persistence

Core approach: **Treat the agent like an engineer whose short-term memory gets wiped at every session.** Before it "clocks out," it must write down critical information so the next "shift" agent can pick up quickly.

Tool 1: Progress file (PROGRESS.md). The most basic state persistence file:

```markdown
# Project Progress

## Current State
- Latest commit: abc1234 (feat: add user preferences endpoint)
- Test status: 42/43 passing (test_pagination_edge_case failing)
- Lint: passing

## Completed
- [x] User model and database migration
- [x] Basic CRUD endpoints
- [x] Auth middleware integration

## In Progress
- [ ] Pagination feature (90% - edge case test failing)

## Known Issues
- test_pagination_edge_case returns 500 on empty result sets
- Need to confirm whether deleted users should appear in listings

## Next Steps
1. Fix pagination edge case bug
2. Add "include deleted users" query parameter
3. Update API documentation
```

Tool 2: Decision log (DECISIONS.md). Record important design decisions and reasons. No need for detailed design documents — just "what decision, why, when":

```markdown
# Design Decisions

## 2024-01-15: Use Redis for user preferences caching
- Reason: High read frequency (every API call), small data size
- Rejected alternative: PostgreSQL materialized view (high change frequency makes
- Constraint: Cache TTL of 5 minutes, active invalidation on write
```

Tool 3: Git commits as checkpoints. Commit after completing each atomic unit of work. Commit messages should explain what was done and why. These are free, automatically versioned state snapshots.

Tool 4: init.sh or harness initialization flow. Specify in `AGENTS.md` the "clock-in" and "clock-out" routines:

```markdown
## At session start (clock in)
1. Read PROGRESS.md for current state
2. Read DECISIONS.md for important decisions
3. Run make check to confirm repo is in consistent state
4. Continue from PROGRESS.md "Next Steps" section

## Before session end (clock out)
1. Update PROGRESS.md
2. Run make check to confirm consistent state
3. Commit all completed work
```

Mixed strategy: Not every task needs a context reset. Short tasks (under 30 minutes) can complete within one session. Long tasks (spanning sessions) must use progress files and decision logs for continuity. Decision criterion: if a task needs more than 60% of the window, start preparing the handoff.

A Deeper Look at Context Anxiety

Anthropic's March 2026 research further revealed the specific manifestations of context anxiety: on Sonnet 4.5, when context approaches the window limit, the agent shows strong "rushed finish" behavior.

Two strategies address this:

Compaction: Summarizing early conversation within the same session. Advantage: maintains continuity, the agent can see "what." Disadvantage: "why" is often lost in summaries — why option B was chosen over A, why a particular optimization was skipped. More critically, compaction doesn't eliminate context anxiety — the agent knows context was once large, and psychologically still tends to rush to finish.

Context reset: Completely clearing context, opening a new session, rebuilding from persisted artifacts. Advantage: clean mental state — the new session has no "I'm running out of time" anxiety. Disadvantage: depends on the completeness of handoff artifacts. If the progress file is missing critical information, the new session may waste time going in the wrong direction.

Anthropic's actual data: for Sonnet 4.5, context anxiety is severe enough that compaction alone isn't sufficient — context reset becomes a critical component of harness design. But for Opus 4.5, this behavior is greatly diminished, and compaction can manage context without relying on resets. This means: **harness design needs specific understanding of the target model, not a one-size-fits-all template.**

> Source: Anthropic: Harness design for long-running application development

---

Anthropic: Handoff Across Coding Sessions

Anthropic describes a Claude website clone whose initializer expanded the specification into more than 200 features, initially marked as failing. The initializer also created init.sh, a progress file, and an initial git commit. Subsequent coding sessions read progress and git history, worked incrementally on features, verified behavior, and left updates for the next session.

This case documents a continuity mechanism; it does not report the feature-completion, hidden-defect, or context-rebuild percentages previously shown here. The companion code is a teaching simulation, not a measurement of model performance.

Anthropic

LangChain reports an actual harness improvement using the same gpt-5.2-codex model on Terminal Bench 2.0, an 89-task benchmark: the score rose from 52.8% to 66.5%, a gain of 13.7 percentage points. Changes included verification guidance, middleware, and context management. This is a result for several harness changes together, not a measured benefit of progress files alone. LangChain: Improving Deep Agents

Key Takeaways

- Context windows are a finite resource. Long tasks will span sessions, and sessions will lose information — this is objective reality.
- The solution isn't bigger windows — it's better state persistence. Progress files, decision logs, and git checkpoints work together to let new sessions pick up where previous ones left off.
- Treat the agent like an engineer whose short-term memory gets wiped every session: before "clocking out," write down what was done, why, and what's next.
- Rebuild cost is the key metric. A good harness leaves progress, verification results, and next actions in files, so a new session recovers quickly.
- Mixed strategy: short tasks within sessions, long tasks with structured artifacts for continuity.

Further Reading

- Anthropic: Effective Harnesses for Long-Running Agents
- OpenAI: Harness Engineering
- Lost in the Middle: How Language Models Use Long Contexts
- Claude Code Documentation
- HumanLayer: Harness Engineering for Coding Agents
- LangChain: Improving Deep Agents

Exercises

1. **State persistence measurement**: Pick a development task needing at least 3 sessions. Without providing any state persistence files, record at each session start how much context the agent spends "figuring out what happened last time." After each session, create a progress file and let the next session start from it. Compare rebuild costs with and without progress files.

2. **Handoff template design**: Design a minimal handoff template with four fields: repo state (commit hash), runtime state (test pass rate), blockers, next actions. Let a completely fresh agent session restore project state using only this template. Record ambiguities encountered during restoration, iterate to improve the template.

3. **Mixed strategy experiment**: In a 5-session development task, compare three strategies: (a) always start fresh sessions + progress files, (b) do as much as possible in one session (context compaction), (c) mixed strategy (short tasks in-session, long tasks across sessions + progress files). Compare rebuild time, feature completion rate, and decision consistency.

中文版 →

> Code examples: code/ Practice project: Project 03. Multi-session continuity

Lecture 06. Make the Agent Initialize Before Every Work Session

You start a new agent session and tell it "add a search feature." It jumps straight into coding — admirable enthusiasm. After 20 minutes it discovers the test framework isn't configured properly, spends another 10 minutes fixing that, then finds the database migration script format is wrong, more fiddling. The search feature does get added in the end, but the whole session was inefficient. Most of the time went to "figuring out how this project works" rather than writing the search feature itself.

The better approach: before letting the agent start working, use a separate phase to get the base environment ready, run verification commands through, and understand the project structure. Initialization work should not be crammed together with feature implementation — they are two fundamentally different kinds of tasks.

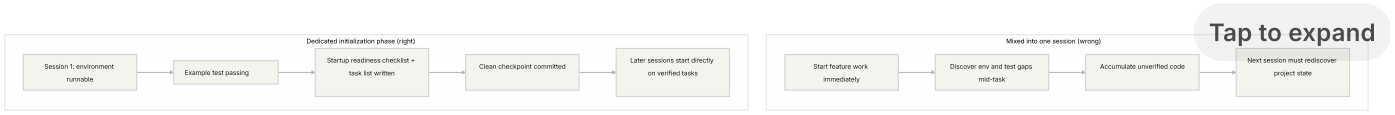
This lecture discusses why initialization must be a separate phase, not mixed in with implementation.

Two Fundamentally Different Kinds of Work

Initialization and implementation have completely different optimization targets. The implementation phase aims to maximize the quantity and quality of verified features. The initialization phase aims to maximize the reliability and efficiency of all subsequent implementation.

When you mix initialization and implementation, the agent faces a multi-objective optimization problem: it has to simultaneously build infrastructure and write feature code. Without explicit priority setting, the agent naturally gravitates toward writing code (because that's directly visible output) while sacrificing infrastructure (because its value only shows up in subsequent sessions). The result: infrastructure doesn't get built solidly, and the reliability of the feature code suffers as well.

Initialization Lifecycle



What Happens When You Mix Them

The most direct problem: infrastructure doesn't get built solidly. The agent spends most of its effort on feature code and treats infrastructure as an afterthought. The test framework is configured but never verified, lint rules are set but too loose, no progress file created. These defects aren't obvious in the first session (because the agent still remembers what it did), but they surface in the second session: the new agent doesn't know how to run the project, how to test, or where things stand.

A more hidden cost is "unverified accumulation." Feature code written before the test framework is properly configured — when you finally go back to add tests, you might discover the design itself was flawed. Had you known earlier, you would have implemented it differently. The more code written up front, the more has to be torn down and redone later.

Context budget is being wasted too. Initialization work (configuring environments, setting up tests, understanding project structure) consumes a large chunk of the budget, leaving less for actual feature implementation. The result: the first session only completes half the features, and the second session still has to start from scratch understanding the project. Budget was spent on initialization, but initialization wasn't done well either — the worst of both worlds.

The most easily overlooked problem is implicit assumption landmines. Decisions the agent makes during initialization (which test framework, how to organize directories, dependency management) — if not explicitly recorded, subsequent sessions may make contradictory choices. The first session chose Vitest as the test framework, but the second session's agent doesn't know and introduces Jest. Two test frameworks coexist, and maintenance costs double.

Anthropic's long-running-agent work makes initialization a separate first session. The initializer writes an `init.sh` script, a progress file, a feature list, and an initial git commit; later coding sessions read that state, work on one feature at a time, test it end to end, commit, and update the progress file.

OpenAI's Codex harness engineering guide also emphasizes the "repository as operational record" principle: establish clear operational structure from the very first run, or every new session has to re-infer project conventions.

Core Concepts

- **Initialization Phase**: The first phase in the agent's lifecycle — it only establishes the prerequisites for subsequent implementation, with no feature development. Its output is infrastructure, not business code.
- **Startup Readiness Checklist**: The conditions under which a project can be unambiguously operated by a fresh agent session: can start, can test, can see progress, can pick up next steps. Four conditions, all required.
- **From Scratch vs From Template**: Starting from scratch means the agent must infer project structure on its own from an empty directory; starting from a template means infrastructure is already in place. Starting from a template far outperforms starting from scratch.
- **Always Ready to Hand Off**: The project is in a state at any given moment where a fresh agent can take over. No verbal explanation needed — just looking at the repo contents is enough to continue working.
- **Time from Start to First Passing Test**: The time from project start until the first feature point passes verification. This is the core metric for measuring initialization efficiency.
- **Success Rate of Subsequent Sessions**: The proportion of subsequent sessions that can successfully execute tasks without relying on implicit knowledge. This is the best measure of initialization quality.

How to Do Initialization Right

Treat initialization as a dedicated phase. The first session does only initialization — no business feature code at all. Initialization produces:

1. Runnable environment. The project starts, dependencies are installed, no environment issues.

2. Verifiable test framework. At least one example test passes, proving the test framework itself is properly configured.

3. Startup readiness checklist document. A clear document telling subsequent sessions:

```markdown
# Startup Readiness Checklist

## Start Commands
- Install dependencies: `make setup`
- Start dev server: `make dev`
- Run tests: `make test`
- Full verification: `make check`

## Current State
- All dependencies installed and locked
- Test framework configured (Vitest + React Testing Library)
- Example test passing (1/1)
- Lint rules configured (ESLint + Prettier)

## Project Structure
- src/ — Source code
- src/components/ — React components
- src/api/ — API client
- tests/ — Test files
```

4. Task breakdown. Split the entire project into an ordered task list, each task with clear acceptance criteria:

```markdown
# Task Breakdown

## Task 1: User Authentication Basics
- Implement JWT auth middleware
- Add login/register endpoints
- Acceptance: pytest tests/test_auth.py all passing

## Task 2: User Profile Page
- Implement user profile CRUD
- Add profile edit form
- Acceptance: pytest tests/test_profile.py all passing

## Task 3: Search Feature
- ...
```

5. Git commit as checkpoint. After initialization completes, commit a clean checkpoint. All subsequent work starts from this checkpoint.

Starting from a template: Don't start from an empty directory. Use a project template (create-react-app, fastapi-template, etc.) to preset standard directory structure, dependency configuration, and test framework. Bake common initialization steps into the template, leaving only project-specific initialization work.

Initialization completion criteria: Not "how much code was written," but whether the startup readiness checklist's four conditions are all met: can start, can test, can see progress, can pick up next steps. Use this checklist to validate initialization:

```markdown
## Initialization Acceptance Checklist
- [ ] `make setup` succeeds from scratch
- [ ] `make test` has at least one passing test
- [ ] A new agent session can answer "how to run" and "how to test" from repo con
- [ ] Task breakdown file exists with at least 3 tasks
- [ ] Everything committed to git
```

Real-World Example

Two initialization approaches for a React frontend project, compared:

Mixed approach: The agent simultaneously created project scaffolding and implemented the first feature in session 1. At session end, the repo had runnable code but no explicit start/test command documentation, no progress tracking file, no task breakdown. Session 2 spent about 20 minutes inferring project structure, test framework, and build process.

Dedicated initialization: Session 1 did only initialization — created directory structure from a template, configured the test framework (Vitest + React Testing Library), wrote and verified one example test, created the startup readiness checklist and task breakdown file, committed the initial checkpoint. Session 2's rebuild time was under 3 minutes, and it started working directly from the task list.

Full project cycle comparison: the mixed approach's total rebuild time (across all sessions) was about 60% more than the dedicated initialization approach. The extra 20 minutes spent on initialization was recovered many times over in subsequent sessions. Invest a bit more time up front to do initialization properly, and subsequent efficiency is actually higher.

Key Takeaways

- Initialization and implementation have different optimization targets — mixing them only drags both down.
- Initialization's output isn't business code, it's infrastructure: runnable environment, verifiable tests, startup readiness checklist, task breakdown.
- Validate initialization with the startup readiness checklist's four conditions: can start, can test, can see progress, can pick up next steps.
- Starting from a template beats starting from scratch. Use project templates to preset standardized infrastructure.
- Time invested in initialization is fully recovered in the next 3-4 sessions. This isn't extra cost — it's upfront investment.

Further Reading

- Anthropic: Effective Harnesses for Long-Running Agents
- OpenAI: Harness Engineering
- HumanLayer: Harness Engineering for Coding Agents
- Infrastructure as Code — Martin Fowler
- SWE-agent: Agent-Computer Interfaces

Exercises

1. **Startup readiness checklist design**: Write a complete startup readiness checklist for a project you're developing. Then open a completely fresh agent session, show it only repo contents (no verbal context at all), and have it try to start the project, run tests, and understand current progress. Record every problem it encounters — each one corresponds to a missing clause in your startup readiness checklist.

2. **Comparison experiment**: Pick a moderately complex new project. Approach A: let the agent initialize and do first implementation simultaneously. Approach B: spend one session on dedicated initialization, start implementation in session 2. After 4 sessions, compare time from start to first passing test, rebuild cost, and feature completion rate.

3. **Initialization acceptance checklist**: Design an initialization acceptance checklist for your project. Have a fresh agent session execute each checklist item and record which pass and which fail. The failing items are where your harness needs strengthening.

中文版 →

> Code examples: code/ Practice project: Project 04. Use runtime feedback to correct agent behavior

Lecture 07. Draw Clear Task Boundaries for Agents

You tell Claude Code to "add user authentication to this project," and it starts modifying the database schema, writing routes, changing frontend components, and — while it's at it — refactoring the error-handling middleware. Two hours later you check: 12 files modified, 800 lines of new code, and not a single feature works end-to-end.

Agents are born with an impulse to "do a little extra" — they see related things and just handle them along the way. The problem is, doing too many things at once almost guarantees none of them get done well.

Anthropic's "Effective harnesses for long-running agents" engineering blog states clearly: when prompts are too broad, agents tend to "start multiple things at once" rather than "finish one thing first." OpenAI's Codex engineering practices found the same — tasks without explicit scope controls see completion rates plummet. This is not a model problem — it's a harness problem. You didn't draw the boundary.

Attention Is a Finite Resource

This isn't a metaphor — it's math. Assume the agent's context capacity is C and it activates k tasks simultaneously. Each task gets an average of C/k reasoning resources. When C/k drops below the minimum threshold needed to complete a single task, none of them get finished.

Claude Code's real behavior is telling. Ask it to "add user registration" and it might:

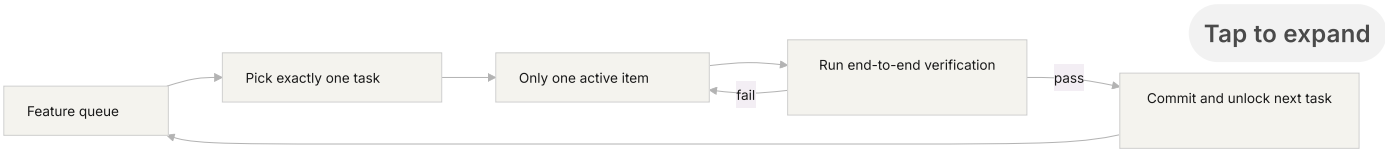
1. Create a User model
2. Write the registration route
3. Notice it needs email verification, so add a mail service
4. See that passwords need hashing, so bring in bcrypt

5. Notice the error handling is inconsistent, so refactor the global error middleware
6. See the test file structure is messy, so reorganize the directory

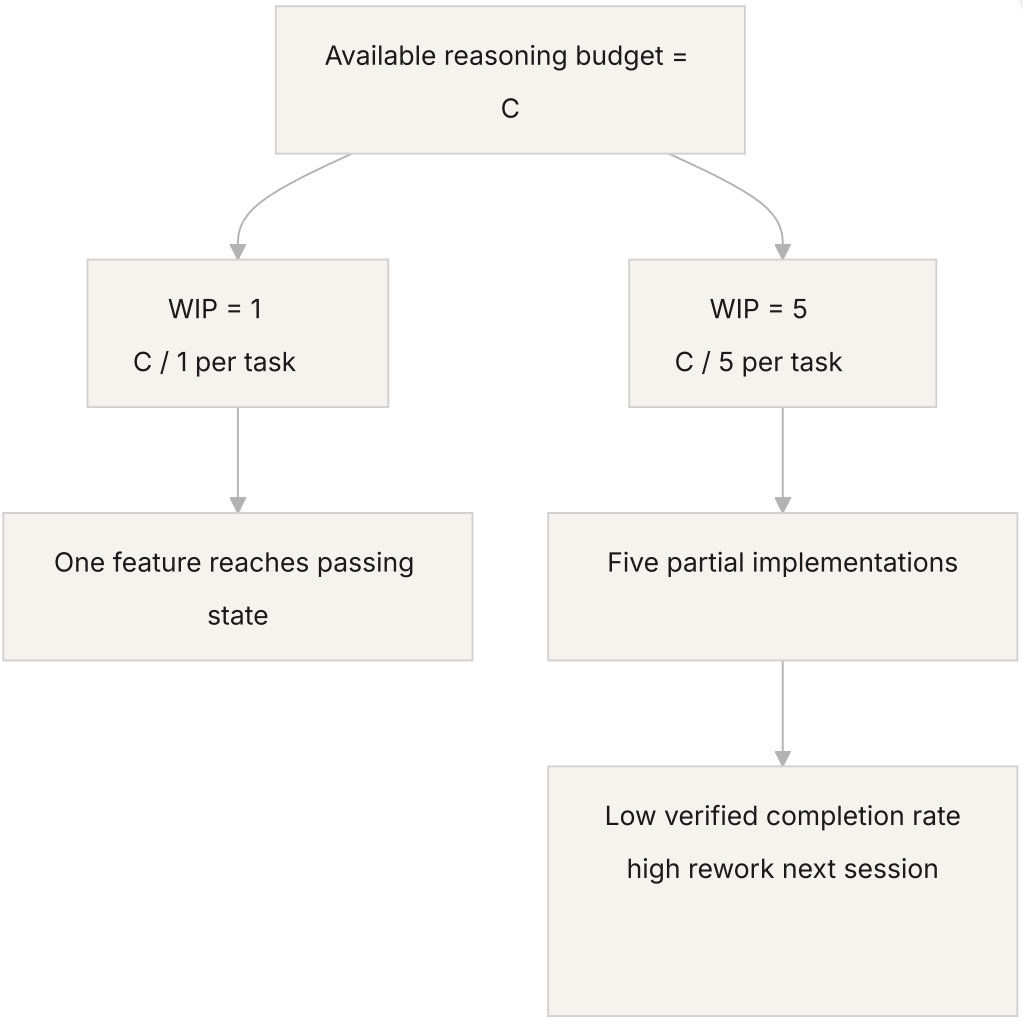
Six steps later, every one is half-done. No end-to-end verification, complex coupling between the half-baked code, and the next session to pick up the pieces will be completely lost.

Anthropic's long-running-agent guidance asks the coding agent to work on one feature at a time, then verify, commit, and update progress before moving on. A broad prompt invites the agent to start several threads, and each thread competes for the same context and verification budget.

WIP=1 Workflow



Tap to expand



Core Concepts

- **Overreach**: The agent activates more tasks in a single session than optimal. This isn't subjective — it's quantifiable: doing 5 features with 0 passing end-to-end is overreach.
- **Under-finish**: The ratio of tasks that pass end-to-end verification, out of all activated tasks, falls below threshold. Code written but tests not passing is under-finish.
- **WIP Limit (Work-in-Progress Limit)**: From Kanban methodology. Core idea: limit how many tasks are in-flight at once. For agents, WIP=1 is the safest default — finish one before starting the next.
- **Completion Evidence**: The verifiable condition a task must satisfy to move from "in progress" to "done." Without this, agents substitute "the code looks fine" for "the behavior passes tests."

- **Scope Surface**: A DAG structure where each node is a work unit and edges are dependencies. States are limited to four: not_started, active, blocked, passing.
- **Completion Pressure**: The constraining force the harness exerts through WIP limits and completion evidence requirements, forcing the agent to finish the current task before starting a new one.

Overreach and Under-finish Are Two Sides of the Same Coin

These two problems aren't independent — they amplify each other. Overreach dilutes attention, diluted attention causes under-finish, and the half-finished code left behind increases system complexity, which further drives overreach in the next task. A vicious cycle.

In Kanban terms: Little's Law tells us L = lambda * W. If work-in-progress L is too high (doing too many things at once), the lead time W for each task inevitably increases. For agents, this means each feature takes longer from start to verified completion, and the probability of failure grows.

This is an old problem in the human world too. Steve McConnell documented in *Rapid Development* that scope creep is the leading cause of project failure. But humans at least have the intuition of "I've done enough." Agents have none. Generating the next idea costs the model almost nothing — writing "let me fix this too while I'm here" barely costs extra tokens, but every additional modification dilutes the agent's attention.

How to Do It Right

1. Enforce WIP=1

This is the most direct and effective method. In your harness, tell the agent explicitly: **only one task is allowed in "active" status at any time.** In Claude Code's CLAUDE.md or Codex's AGENTS.md, write:

```
## Work Rules
- Work on one feature at a time
- Only start the next feature after the current one passes end-to-end verificatio
- Don't "also refactor" feature B while implementing feature A
```

2. Define Explicit Completion Evidence for Every Task

Done is not "code is written" — it's "behavior verification passes." In your feature list, every entry needs a verification command:

```
F01: User Registration
  Verification: curl -X POST /api/register -d '{"email":"test@example.com","passw
  State: passing
```

3. Externalize the Scope Surface

Use a machine-readable file (JSON or Markdown) to record all task states. Any new session can read this file and immediately know: which task is active? What behavior counts as done? What verifications have passed?

4. Monitor Verified Completion Rate

The harness should continuously track VCR (Verified Completion Rate) = verified tasks / activated tasks. Block new task activations when VCR < 1.0.

Real-World Case

A REST API project with 8 features, two strategies compared:

Unconstrained mode: Agent activates 5 features simultaneously in session 1. Produces ~800 lines across 12 files. End-to-end test pass rate: 20% — only user registration works. The other 4 features: database schema created but missing validation logic, routes defined but returning wrong response formats. By end of session 3, only 3 of 8 features complete.

WIP=1 mode: Agent works on user registration only in session 1. Produces ~200 lines across 4 files. End-to-end tests: 100% passing. Commits a clean, verified implementation. By end of session 4, 7 of 8 features complete (the 8th blocked by an external dependency).

Result: less total code (800 vs 1200 lines) but more effective code. Completion rate: 87.5% vs 37.5%.

Key Takeaways

- **WIP=1 is the default safe setting for agent harnesses** — finish one, then start the next; don't try to parallelize.
- **Completion evidence must be executable** — "the code looks fine" doesn't count; "curl returns 201" does.
- **The scope surface must be externalized as a file** — not just mentioned in conversation, but recorded in a machine-readable format in the repo.
- **Overreach and under-finish are symbiotic** — solving one solves the other.
- **"Do less but finish" always beats "do more but leave half-done"** — agent code lines and feature completion rate are negatively correlated. Quality always beats quantity.

Further Reading

- Effective harnesses for long-running agents - Anthropic — Anthropic's engineering blog, detailed discussion of the "small next step" strategy
- Harness Engineering - OpenAI — OpenAI's complete treatment of harness engineering
- Kanban: Successful Evolutionary Change - David Anderson — The classic source on WIP limits
- Rapid Development - Steve McConnell — Empirical data on scope creep as the leading cause of project failure

Exercises

1. **Task Atomization**: Pick a broad requirement (e.g., "implement a user management system") and break it into at least 5 atomic work units. For each unit, specify: (a) a single behavior description, (b) an executable verification command, (c) dependencies. Check whether the decomposition satisfies the WIP=1 constraint.

2. **Comparison Experiment**: Run the same project twice — once without constraints, once with enforced WIP=1. Compare verified completion rate, total lines of code, and effective code ratio.

3. **Completion Evidence Audit**: Review a recent agent run's output, classifying each code change as "completed behavior," "incomplete behavior," or "scaffolding." Add missing verification commands for each incomplete behavior.

中文版 →

> Code examples: code/ Practice project: Project 04. Runtime feedback and scope control

Lecture 08. Use Feature Lists to Constrain What the Agent Does

You ask an agent to build an e-commerce site. After it finishes, it tells you "done." You look at the code — user authentication works, but the checkout button in the shopping cart does nothing, and the payment flow isn't connected at all. Where did things go wrong? You never told it what "done" means, so it used its own standard: "I wrote a lot of code and it looks fairly complete."

Feature lists, in many people's eyes, are just a memo — write things down so you don't forget, then toss them aside. But in the harness world, a feature list isn't a memo for humans. It's the foundational structure the entire harness is built on. The scheduler relies on it to pick tasks, the verifier relies on it to judge completion, and the handoff reporter relies on it to generate summaries. Without it, these components have no shared consensus to depend on.

Both Anthropic and OpenAI emphasize: **artifacts must be externalized.** Feature state must live in a machine-readable file in the repo, not in unstructured conversation text.

Agents Don't Know What "Done" Means

Neither Claude Code nor Codex automatically knows what you mean by "done." You say "add a shopping cart feature," and the model's interpretation might be "write a Cart component and an addToCart method." But you meant "the user can browse products, add to cart, and complete checkout end-to-end."

This understanding gap persists without a feature list. The agent uses its own implicit standard — usually "the code has no obvious syntax errors." What you need is end-to-end behavioral verification. Without a list, the two sides will never agree on what "done" means.

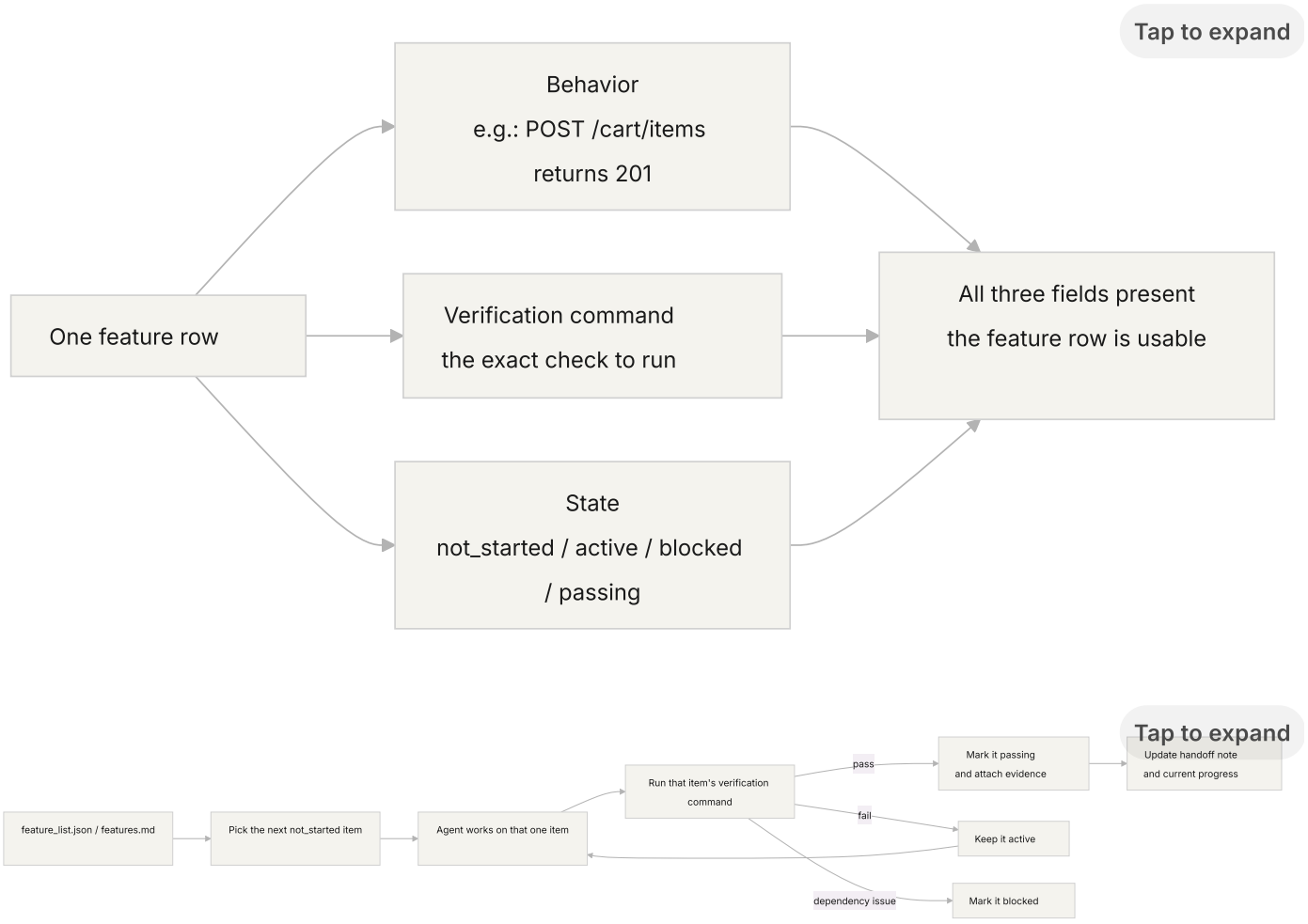
Look at this common progress note:

```
Did user auth, shopping cart mostly done, still need payments
```

Can a new agent session answer these questions from this note? What does "mostly done" mean? Which tests did the cart pass? What's blocking payments? The answer to all is "nobody knows."

The result: the new session has to infer project state, and may end up re-implementing completed features. Anthropic's advice is to record progress and verification results in versioned files so the next session can inspect state.

Feature State Machine



Core Concepts

- **Feature lists are harness primitives**: Not "optional planning tools," but foundational data structures that all other harness components depend on. The scheduler, verifier, and handoff reporter all need to read the feature list to function.
- **Triple structure**: Each feature item contains three elements: `(behavior description, verification command, current state)`. The behavior tells the agent what to do, the verification tells it what counts as done, and the state tells it where things stand. Missing any element makes the item incomplete.
- **State machine model**: Each feature item has four states — `not_started`, `active`, `blocked`, `passing`. State transitions are controlled by the harness, not freely changed by the agent.
- **Pass-state gating**: The only way a feature moves from `active` to `passing` is by the verification command executing successfully. This transition is irreversible — once `passing`, it can't go back.
- **Single source of truth**: All information about "what needs to be done" must derive from one feature list. No contradictions between the feature list and conversation history.
- **Back-pressure**: The number of features that haven't passed yet is the pressure the harness exerts on the agent. Zero pressure = project complete.

Why Feature Lists Must Be "Primitives"

Documents are for humans to read; primitives are for systems to execute. Documents can be ignored; primitives can't be bypassed.

Think of it like database trigger constraints versus application-layer checks: the former is enforced by the database engine — no SQL can skip it. The latter depends on application code correctness and can be accidentally bypassed. Feature lists as harness primitives play the same role as database-level constraints — the agent cannot bypass them.

Specifically, the feature list serves four harness components:

1. **Scheduler**: Reads states, picks the next `not_started` feature.

2. **Verifier**: Executes verification commands, decides whether to allow state transitions.
3. **Handoff reporter**: Automatically generates session handoff summaries from the feature list.
4. **Progress tracker**: Tallies state distribution, provides project health metrics.

How to Do It

1. Define a Minimal Feature List Format

You don't need a complex system — a structured Markdown or JSON file works. The key is every entry must have the triple:

```json
{
  "id": "F03",
  "behavior": "POST /cart/items with {product_id, quantity} returns 201",
  "verification": "curl -X POST http://localhost:3000/api/cart/items -H 'Content-
  "state": "passing",
  "evidence": "commit abc123, test output log"
}
```

2. Let the Harness Control State Transitions

The agent can't directly change a feature's state to `passing`. It can only submit a verification request. The harness executes the verification command and decides whether to allow the transition. This is "pass-state gating."

3. Write the Rules in CLAUDE.md

```
## Feature List Rules
- Feature list file: /docs/features.md
- Only one feature active at a time
- Verification command must pass before marking as passing
- Don't modify feature list states yourself — the verification script updates the
```

4. Calibrate Granularity

Each feature item should be scoped to "completable in one session." Too broad and it won't finish; too narrow and the management overhead grows. "User can add items to cart" is good granularity. "Implement the shopping cart" is too broad. "Create the name field on the Cart model" is too narrow.

Real-World Case

An e-commerce platform with 10 features. Two tracking approaches compared:

Memo mode: The agent uses unstructured notes to track progress. After 3 sessions, the notes become "did user auth and product list, shopping cart mostly done but has bugs, payments not started." A new session needs 20 minutes to infer state, and ultimately re-implements completed features.

Structured mode: Every feature has a clear state and verification command. A new session reads the feature list and in 3 minutes knows: F01-F05 are `passing`, F06 is `active` (in progress), F07-F10 are `not_started`. It picks up directly from F06, with zero rework.

A structured list puts each feature's state and verification command in one place, so the new session knows what is done.

Key Takeaways

- **Feature lists are the harness's foundational structure**, not memos for humans. The scheduler, verifier, and handoff reporter all depend on them.
- **Every feature item must have the triple**: behavior description + verification command + current state. Missing one element makes it incomplete.
- **State transitions are controlled by the harness** — the agent can't change states on its own. Passing verification is the only upgrade path.
- **The feature list is the project's single source of truth** — all "what to do" information derives from it.

- **Calibrate granularity to "completable in one session."** Too broad and it won't finish; too narrow and you can't manage it.

Further Reading

- Building Effective Agents - Anthropic — Explicitly identifies feature list as the "core data structure" for controlling agent scope
- Harness Engineering - OpenAI — Emphasizes the principle of "externalizing artifacts"
- Design by Contract - Bertrand Meyer — Design by contract principles, the theoretical foundation of feature lists
- How Google Tests Software — Test pyramid and behavioral specification engineering practices

Exercises

1. **Feature List Design**: Define a minimal feature list JSON schema. Include: id, behavior description, verification command, current state, evidence reference. Use it to describe a real project with 5 features.
2. **Verification Strictness Comparison**: Pick 3 features and design both a "loose" verification (e.g., "code has no syntax errors") and a "strict" verification (e.g., "end-to-end test passes"). Compare false positive rates under each approach.
3. **Single Source Principle Audit**: Review an existing agent project and check for scope information that contradicts the feature list (implicit requirements in conversations, TODO comments in code, etc.). Design a plan to unify all information into the feature list.

中文版 →

> Code examples for this lecture: code/ Hands-on practice: Project 05. Let the agent verify its own work

Lecture 9. Preventing Agents from Declaring Victory Too Early

You ask an agent to implement a "password reset" feature. It modifies the database schema, writes the API endpoint, adds the email template, runs the unit tests (all pass), and then confidently tells you "it's done." But when you actually try to run it—the password reset link can't be sent because the email service config is missing; the database migration fails halfway through, leaving the schema in an inconsistent state; and the end-to-end flow hasn't been executed even once.

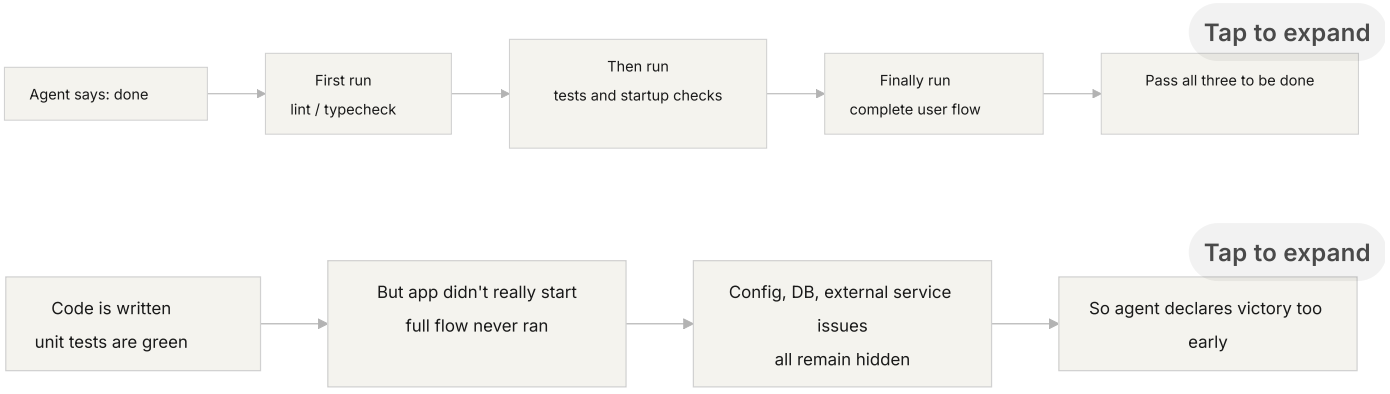
This isn't an isolated incident. The classic 2017 ICML paper by Guo et al. proved: **modern neural networks are systematically overconfident**—the confidence reported by models is significantly higher than their actual accuracy. AI coding agents are no different. They "feel" done, but in reality they're far from it. Your harness must replace the agent's "feelings" with externalized, execution-based verification.

The Slippery Slope

Premature completion declarations almost always follow the same playbook: the code looks okay—syntax is correct, logic seems reasonable, static analysis shows no obvious errors. But the harness doesn't enforce comprehensive execution verification, so the agent skips actually running it or only runs partial tests. It runs unit tests but skips integration tests; it runs tests but doesn't check coverage. In the end, "the code looks fine" is taken as evidence that "the feature is complete."

Information is lost at every step. From task specification to code implementation to runtime behavior, every transformation can introduce bias, and every skipped verification compounds the information asymmetry.

Three-Layer Termination Check



Core Concepts

- **Premature Completion Declaration**: The agent asserts the task is complete, but unmet correctness specifications remain. The core problem is that the agent judges based on local, code-level confidence, while system-level correctness requires global verification.
- **Confidence Calibration Bias**: A systematic gap between the agent's self-reported completion confidence and the actual completion quality. For complex multi-file tasks, this bias is significantly positive—the agent is consistently more confident than its actual performance warrants.
- **Termination Criteria**: A clear, executable set of judgment conditions defined in the harness. The agent must satisfy all conditions before declaring completion. "Done" shifts from a subjective judgment to an objective determination.
- **Verification-Validation Dual Gate**: The first layer (verification) checks whether the code correctly implements the specified behavior; the second layer (validation) checks whether system-level behavior meets end-to-end requirements. Both must pass before the task is considered complete.
- **Runtime Feedback Signals**: Logs, process states, and health checks from program execution—these form the objective basis for the harness to judge completion quality.
- **Completion Priority Constraint**: First verify functional correctness, then address performance, and finally handle style. No refactoring is allowed until core functionality has been verified.

Passing Unit Tests ≠ Task Complete

This is the most common trap, and the most dangerous one. The agent writes code, runs unit tests, sees all greens, and says "done." But the design philosophy of unit tests—isolating the tested unit and mocking dependencies—is precisely what makes them incapable of detecting cross-component issues:

Interface Mismatch: The renderer passes a relative file path to the preload script, but the preload script expects an absolute path. Their respective unit tests both use mocks and both pass. The issue only surfaces during end-to-end testing.

State Propagation Errors: A database migration changes the table schema, but the ORM's caching layer still holds cache entries for the old schema. Unit tests run in a fresh mock environment every time, so this kind of cross-layer state inconsistency never surfaces.

Environment Dependency: Code behaves correctly in the test environment (where everything is mocked) but fails in the real environment due to configuration differences, network latency, or service unavailability.

"Refactoring While We're at It" is Poison to Completion Judgment

Claude Code has a common behavioral pattern: it starts refactoring code, optimizing performance, and improving style before the core functionality has passed verification. Knuth's adage that "premature optimization is the root of all evil" takes on new meaning in the agent scenario—refactoring shifts the boundary between verified and unverified code, potentially breaking code paths that were previously implicitly correct.

Systematic Bias in Self-Evaluation

Anthropic discovered a deeper failure pattern in their 2026 research: **when an agent is asked to evaluate its own work, it systematically provides overly positive assessments—even when a human observer would judge the quality as clearly substandard.**

This problem is especially severe on subjective tasks (such as design aesthetics). Whether a "layout is polished" is a judgment call, and the agent reliably skews positive. Even on tasks with verifiable outcomes, the agent's poor judgment degrades its performance.

The solution isn't to make the agent "more objective." The same model both generates and evaluates, so it is inherently inclined to be generous with itself. **The solution is to separate the "worker" from the "checker."**

An independent evaluation agent, specifically tuned to be "nitpicky," is far more effective than having the generating agent evaluate itself. Anthropic's experimental data:

Architecture	Runtime	Cost	Core Features Working?
Single agent (bare run)	20 mins	$9	No (game entities unresponsive to input)
Three agents (planner + generator + evaluator)	6 hours	$200	Core gameplay worked; physics and level issues remained

This is the exact same model (Opus 4.5) with the exact same prompt ("build a 2D retro game editor"). The only difference is the harness: from "running bare" to "planner expands requirements → generator implements feature by feature → evaluator performs actual click testing using Playwright."

> Source: Anthropic: Harness design for long-running application development

---

How to Prevent Premature Completion Declarations

1. Externalize Termination Judgment

The completion judgment should not be made by the agent itself. The harness independently executes termination validation, using runtime signals as input rather than the agent's confidence. In CLAUDE.md, you can spell this out:

```
## Definition of Done
- Feature complete = end-to-end verification passed, not "code is written"
- Required verification levels:
  1. Unit tests pass
  2. Integration tests pass
  3. End-to-end flow verification passes
- Do not proceed to level 2 if level 1 fails
- Do not proceed to level 3 if level 2 fails
```

2. Build a Three-Layer Termination Validation

- **Layer 1: Syntax and Static Analysis**. Lowest cost, least information, but must pass. This is the bare minimum—you need to spell the words correctly before reading further.
- **Layer 2: Runtime Behavior Verification**. Test execution, application startup checks, critical path validation. This is the core evidence of completion—not just written, but runnable.
- **Layer 3: System-Level Confirmation**. End-to-end testing, integration validation, user scenario simulation. The last line of defense against premature declarations—not just runnable, but correct.

3. Provide Actionable Error Feedback to the Agent

OpenAI introduced a particularly effective pattern in their Codex practice: **error messages written for agents should include repair instructions**. Don't just tell the agent "it's wrong"—point out exactly what's wrong and how to fix it. Don't use `"Test failed"`, use `"Test failed: POST /api/reset-password returned 500. Check that the email service config exists in environment variables. The template file should be at templates/reset-email.html."` This kind of specific, actionable feedback lets the agent self-correct without human intervention.

4. Capture Runtime Signals

Effective runtime signals include:

- Did the application successfully start and reach a ready state?
- Did critical feature paths execute successfully at runtime?
- Were database writes, file operations, and other side effects correct?

- Were temporary resources cleaned up?

Real-World Case

Task: Implement user password reset functionality. Involves database operations, email sending, and API endpoint modifications.

Premature hand-in path: The agent modifies the database schema, writes the API endpoint, adds the email template, runs unit tests (all pass), and declares completion. It looks like a lot was done, but the critical steps were all skipped.

Actual omissions: (1) End-to-end flow untested—the actual sending and verification of the reset link was never confirmed. (2) Database migration failed after partial execution, leaving the schema inconsistent. (3) Email service configuration was missing in the target environment.

Harness intervention: Termination validation is enforced—(1) start the full application to verify the reset endpoint is accessible; (2) execute the complete reset flow; (3) verify database state consistency. All three defects surface inside the session, while the fix cost is still local.

Key Takeaways

- **Agents are systematically overconfident**—confidence calibration bias is an objective reality. Code being written doesn't mean it was written correctly.
- **Completion judgment must be externalized**—the harness verifies independently. Don't trust the agent's "feelings."
- **All three layers of validation are essential**: syntax passes, behavior passes, system passes—layer by layer, no shortcuts.
- **Error messages should include specific repair steps**, enabling the agent to self-correct rather than just telling it "it's wrong."
- **No refactoring until core functionality is verified**—the completion priority constraint is the key to preventing premature optimization.

Further Reading

- On Calibration of Modern Neural Networks - Guo et al. — Proves modern deep networks are systematically overconfident
- Building Effective Agents - Anthropic — The critical role of runtime evidence in completion judgment
- Harness Engineering - OpenAI — Premature completion declaration is one of the main failure modes of agents
- The Art of Software Testing - Myers — Classic reference on testing method hierarchies and effectiveness

Exercises

1. **Termination Validation Function Design**: Design a complete termination validation for a task involving database migration and API modifications. List the required runtime signals and the pass/fail criteria for each. Run it on a real task and record which hidden issues it discovers.

2. **Calibration Bias Measurement**: Select 10 coding tasks of different types. Record the agent's self-reported completion confidence versus the actual completion quality. Calculate the bias and analyze its relationship with task complexity.

3. **Multi-Layer Defense Experiment**: Run three configurations on the same set of tasks: (a) static analysis only, (b) add unit testing, (c) full three-layer validation. Compare the proportion of premature completion declarations and the number of uncaught defects.

中文版 →

> Code examples for this lecture: code/ Hands-on practice: Project 05. Let the agent verify its own work

Lecture 10. Only a Full Pipeline Run Counts as Real Verification

You ask the agent to add a file export feature to an Electron app. It writes the renderer component, the preload script, and the service layer logic. Unit tests for every component pass. The agent says "done." You actually click the export button — the file path format is wrong, the progress bar doesn't respond, and exporting large files leaks memory. Five component boundary defects, and unit tests didn't catch a single one.

Each part looks "correct" on its own, but problems surface the moment they are wired together. Google's Testing Pyramid tells us that a large base of unit tests is essential, but stopping there means you will systematically miss component interaction issues. For AI coding agents this problem is even worse, because agents tend to run only the fastest tests and then declare completion. **Only end-to-end testing can prove the absence of system-level defects.**

The Blind Spots of Unit Tests

The design philosophy of unit testing is isolation: mock dependencies, focus on the unit under test. This philosophy makes unit tests fast and precise, but it also creates systematic blind spots. Every module performs perfectly in isolation, yet the following categories of problems only surface when everything actually runs together:

Interface Mismatch: The renderer passes the preload script a relative file path, but the preload script expects an absolute path. Their respective unit tests both use mocks and both pass. The problem is only discovered when the end-to-end flow is exercised.

State Propagation Errors: A database migration changes the table schema, but the ORM caching layer still holds cache entries with the old schema. Unit tests spin up a fresh mock environment every time, so they never expose this kind of cross-layer state inconsistency.

Resource Lifecycle Issues: The acquisition and release of file handles, database connections, and network sockets span multiple components. Unit tests create and tear down independent resources for each test case, so they never surface resource contention or leaks.

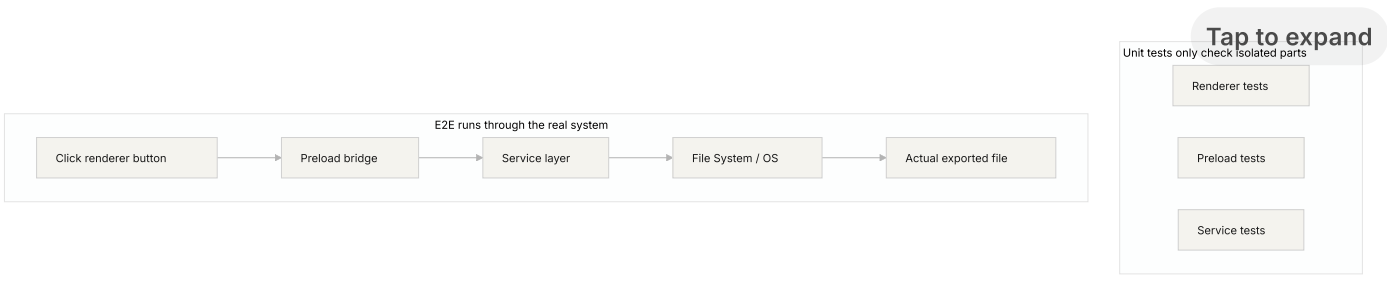
Environment Dependency: Code behaves correctly in the test environment (where everything is mocked) but fails in the real environment due to configuration differences, network latency, or service unavailability.

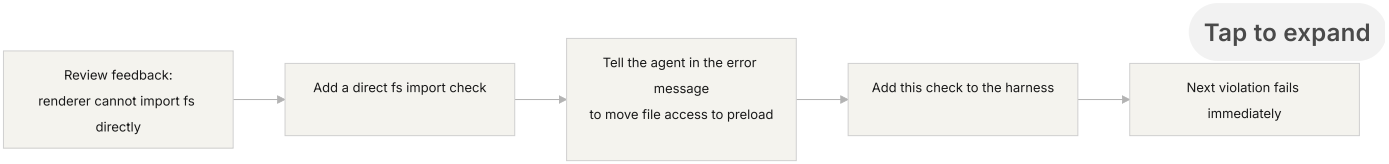
End-to-End Testing Not Only Changes Results – It Changes Behavior

This is something many people overlook: when an agent knows its work will be validated by end-to-end tests, its coding behavior shifts.

1. **Considering component interactions**: While writing code it starts asking "how does this interface connect with the upstream?" rather than focusing on a single function in isolation.
2. **Respecting architectural boundaries**: In systems with architectural constraints, end-to-end tests force the agent to follow boundary rules.
3. **Handling error paths**: End-to-end tests typically include failure scenarios, which pushes the agent to think about exception handling.

Testing Pyramid and Review Feedback Promotion





OpenAI emphasizes in its Codex engineering practices: **error messages written for agents must include fix instructions.** Instead of writing `"Direct filesystem access in renderer"`, write `"Direct filesystem access in renderer. All file operations must go through the preload bridge. Move this call to preload/file-ops.ts and invoke it via window.api."` This turns architectural rules into an auto-correction loop. Error messages don't just tell you "what went wrong" — they tell you "how to fix it," enabling the agent to correct itself autonomously.

Core Concepts

- **Component Boundary Defects**: Component A and B each pass their unit tests, but their interaction produces incorrect behavior. This is the category of issue that end-to-end testing excels at catching.
- **Testing Adequacy Gradient**: Defects detectable by unit tests <= defects detectable by integration tests <= defects detectable by end-to-end tests. Detection capability increases with each layer.
- **Architectural Boundary Enforcement Rules**: Turning rules from architecture documents (such as "the renderer process must not access the filesystem directly") into executable, automated checks — going from "written on paper" to "running in CI."
- **Review Feedback Promotion**: Converting recurring code review comments into automated tests. Every time a new category of repeated issue is found, add a rule, and the harness grows stronger automatically.
- **Agent-Oriented Error Messages**: Failure messages should not merely state "what went wrong" but also tell the agent exactly how to fix it, turning test failures into self-correcting feedback loops.

How to Do It

0. Define Architectural Boundaries Before Writing E2E Tests

The prerequisite for end-to-end testing is that the system has clear boundaries. If the architecture is a tangled mess, end-to-end testing will only prove "the whole mess runs" without telling you where design intent was violated.

OpenAI's experience: **for agent-generated codebases, architectural constraints must be established as early prerequisites on day one — not something to think about once the team has grown.** The reason is straightforward: agents copy existing patterns in the repository, even when those patterns are inconsistent or suboptimal. Without architectural constraints, agents introduce more drift with every session.

OpenAI adopted a "Layered Domain Architecture," where each business domain is divided into fixed layers: Types → Config → Repo → Service → Runtime → UI. Dependencies flow strictly forward, and cross-domain concerns enter through explicit Providers interfaces. Any other dependency is forbidden and mechanically enforced via custom linting.

Key principle: **Enforce invariants; don't micromanage implementation.** For example, require that "data is parsed at the boundary," but don't prescribe which library to use. Error messages must include fix instructions — not just saying "violation," but telling the agent concretely how to change it.

> Source: OpenAI: Harness engineering: leveraging Codex in an agent-first world

1. The Harness Must Include an End-to-End Layer

Make it explicit in your validation flow: for tasks involving cross-component changes, passing end-to-end tests is a prerequisite for completion:

```
## Validation Hierarchy
- Level 1: Unit tests (Must pass)
- Level 2: Integration tests (Must pass)
- Level 3: End-to-end tests (Must pass when cross-component changes are involved)
- Skipping any required level = Not Complete
```

2. Turn Architectural Rules into Executable Checks

Every architectural constraint should have a corresponding test or lint rule:

```bash
# Check whether the renderer process directly calls Node.js APIs
grep -r "require('fs')" src/renderer/ && exit 1 || echo "OK: no direct fs access
```

3. Design Agent-Oriented Error Messages

Failure messages should contain three elements: what went wrong, why, and how to fix it:

```
ERROR: Found direct import of 'fs' in src/renderer/App.tsx:12
WHY: Renderer process has no access to Node.js APIs for security
FIX: Move file operations to src/preload/file-ops.ts and call via window.api.read
```

4. Establish a Review Feedback Promotion Process

Every time you discover a new category of agent error during code review, turn it into an automated check. A month later your harness will be far stronger than it was at the start of the month.

Real-World Case

Task: Implement a file export feature in an Electron app. Involves renderer process UI, preload script filesystem proxy, and service layer data transformation.

Unit test phase: Renderer component tests (passed, file operations mocked), preload script tests (passed, filesystem mocked), service layer tests (passed, data source mocked). Agent declares completion.

Defects revealed by end-to-end tests:

Defect	Description	Unit Test	E2E
Interface Mismatch	Inconsistent file path format	Missed	Caught
State Propagation	Export progress not sent back to UI via IPC	Missed	Caught
Resource Leak	Large file export handles not released	Missed	Caught
Permission Issue	Different permissions in packaged environment	Missed	Caught
Error Propagation	Service layer exceptions didn't reach UI layer	Missed	Caught

All 5 defects were caught by end-to-end tests; unit tests caught none. The end-to-end run is slower, but that cost is small compared with shipping five boundary defects.

Key Takeaways

- **Unit tests are systematically blind to component boundary defects**: their isolation design is precisely what prevents them from detecting interaction issues.
- **End-to-end testing not only detects defects, it changes how agents write code**: it shifts focus toward integration and boundaries.
- **Architectural rules must be executable**: not written in a document waiting for someone to read them, but automatically checked on every commit.
- **Error messages must be designed for agents**: include concrete "how to fix" steps to form a self-correcting feedback loop.
- **Review feedback promotion makes the harness automatically stronger**: every captured defect category becomes a permanent line of defense.

Further Reading

- How Google Tests Software - Whittaker et al. — The classic source of the Testing Pyramid model

- Harness Engineering - OpenAI — Engineering practices for automated enforcement of architectural constraints
- Chaos Engineering - Netflix (Basiri et al.) — Proactively injecting failures to verify system resilience
- QuickCheck - Claessen & Hughes — Property testing methodology, sitting between example-based testing and formal verification

---

Exercises

1. **Cross-Component Defect Detection**: Pick a modification task involving at least three components. First run only unit tests and record the results, then run end-to-end tests. Analyze each additionally discovered defect and classify which type of cross-layer interaction issue it represents.
2. **Architectural Rule Automation**: Pick an architectural constraint from your project and turn it into an executable check (complete with an agent-oriented error message). Integrate it into the harness and verify its effectiveness using a baseline task.
3. **Review Feedback Promotion**: Find a recurring comment type from code review history and convert it into an automated check following the five-step process. Compare the frequency of that issue category before and after the promotion.

中文版 →

> Code examples: code/ Practice project: Project 06. Build a Complete Agent Harness

Lecture 11. Making the Agent's Runtime Observable

You ask an agent to implement a feature. It runs for 20 minutes, touches a pile of files, then tells you "done, but two tests are failing." You ask why — "not sure, might be a timing issue." You ask which critical paths it changed — "let me look at the code..."

This scenario is all too common, and the root cause isn't the agent's capability — it's the harness's lack of observability. When an agent executes a task without visibility into the actual runtime state, every decision it makes is essentially a guess.

Without observability, agents make decisions under uncertainty, evaluations become subjective judgments, and retries become blind wandering. Both OpenAI and Anthropic frame reliability as an evidence problem: the harness must expose runtime behavior and evaluation signals in a form that can actually guide the next decision.

The Real Cost of Missing Observability

When a harness lacks observability, four categories of problems appear systematically.

Cannot distinguish "correct" from "looks correct." A function looks perfectly right in code review — correct syntax, sound logic. But at runtime, a boundary-condition handling error produces incorrect results under specific inputs. Only runtime traces can reveal that the actual execution path deviated from expectations. Code review shows "what was written"; runtime tracing shows "what actually ran." You need both.

Evaluation becomes mysticism. Without scoring rubrics and acceptance criteria, evaluators (human or agent) have to rely on implicit assumptions. The same output can get wildly different assessments from different evaluators. Quality evaluation becomes non-reproducible.

Retries become blind guesses. When the agent doesn't know why something failed, its retry direction is random. It might hammer away in the wrong direction — fixing unrelated code

paths while ignoring the real root cause. Every blind retry burns tokens and time.

Session handoff information cliff. When incomplete work is handed to the next session, missing observability means the new session has to diagnose the system state from scratch. The new session has to reread code, rerun commands, and decide which signals matter; versioned progress and verification records provide inspectable state.

A Real Claude Code Scenario

Consider a harness using a "planner-generator-evaluator" three-role workflow, executing the task "add dark mode to the app."

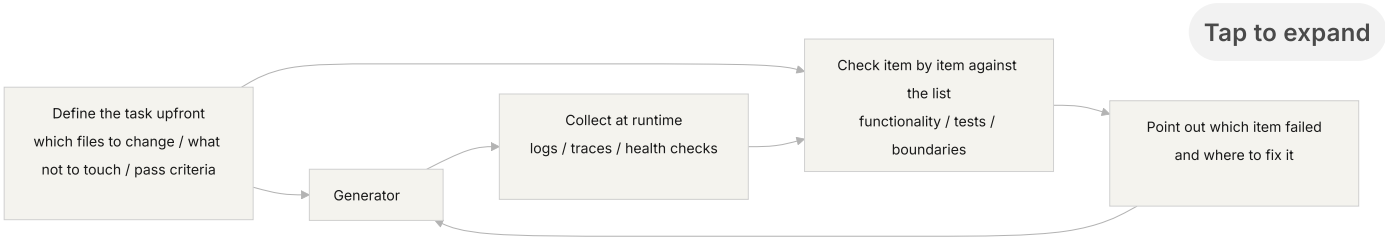
Without observability: The planner outputs a vague description. The generator implements dark mode based on that vagueness, but the result doesn't match the planner's implicit expectations. The evaluator rejects it based on their own implicit standards but can't articulate what's specifically wrong — just "it doesn't feel right." The generator retries blindly on vague rejection reasons. The cycle repeats 3-4 times, taking about 45 minutes, and barely produces an acceptable output.

With full observability: The planner outputs a sprint contract listing which components to modify, verification standards for each, and exclusions (e.g., no print styles). The generator implements according to the contract, and runtime observability records each component's style loading and application process. The evaluator uses a scoring rubric to evaluate dimension by dimension, citing specific evidence: "Button color contrast is insufficient (WCAG AA standard 4.5:1, measured 2.1:1)." One iteration produces a high-quality result, in about 15 minutes.

The difference comes from whether the contract and evidence exist outside the agent's head.

Layered Observability

Observability isn't just "add more logging." It operates on two layers, and both are essential.



Runtime observability: System-level signals — logs, traces, process events, health checks. Answers "what did the system do."

Process observability: Visibility into harness decision artifacts — plans, scoring rubrics, acceptance criteria. Answers "why should this change be accepted."

Core Concepts

- **Runtime observability**: System-level signals including logs, traces, process events, and health checks. Answers "what did the system do."
- **Process observability**: Visibility into harness decision artifacts including plans, scoring rubrics, and acceptance criteria. Answers "why should this change be accepted."
- **Task trace**: A complete decision-path record from task start to completion, analogous to request tracing in distributed systems. Every step the agent takes, along with its context, is recorded — so when something goes wrong, you can replay the entire process.
- **Sprint contract**: A short-term agreement negotiated before coding begins, specifying task scope, verification standards, and exclusions. The core tool for process observability.
- **Evaluator rubric**: Transforms quality evaluation from subjective judgment into evidence-based structured scoring, enabling different evaluators to reach similar conclusions for the same output.
- **Layered observability**: System-layer and process-layer observability designed simultaneously and reinforcing each other. Runtime signals explain behavior; process artifacts explain intent.

Why Agents Can't Solve This Themselves

You might be thinking: "Can't the agent just print its own logs?" The problem is:

1. **Agents don't know what they don't know.** They won't proactively record signals they don't realize they need. Without harness-level constraints, agents only log what they think is important — and what they think is important is usually not enough.
2. **Log formats are inconsistent.** Different sessions use different log formats, making systematic analysis impossible.
3. **Process observability can't be solved by logging.** Sprint contracts and scoring rubrics are structured artifacts that require harness-level support — adding a few print statements won't cut it.

How to Build Observability

1. Build Runtime Signal Collection into the Harness

Don't rely on the agent to print its own logs. The harness should automatically collect the following signals:

- **Application lifecycle**: Startup, ready, running, shutdown phase states
- **Feature path execution**: Execution records for critical paths, including entry points, checkpoints, and exits
- **Data flow**: Records of data flowing between components
- **Resource utilization**: Abnormal resource usage patterns (e.g., continuously growing memory)
- **Errors and exceptions**: Full error context, not just error messages

2. Implement Sprint Contracts

Before each task begins, the generator and evaluator (which may be different invocations of the same agent) negotiate a contract that defines what to build and what "done" looks like:

```markdown
# Sprint Contract: Dark Mode Support

## Scope
- Modify the theme toggle component
- Update global CSS variables
- Add dark mode tests

## Verification Standards
- Visual regression tests pass for each component
- Main flow end-to-end tests pass
- No flash of unstyled content (FOUC)

## Exclusions
- Not handling print styles
- Not handling third-party component dark mode
```

3. Establish an Evaluator Rubric

Turn "is it good or not" into quantifiable scoring:

```markdown
# Scoring Rubric

| Dimension | A | B | C | D |
|-----------|---|---|---|---|
| Code correctness | All tests pass | Main flow passes | Partial pass | Build fai
| Architecture compliance | Fully compliant | Minor deviations | Obvious deviatio
| Test coverage | Main + edge cases | Main flow only | Only skeleton | No tests |
```

4. Standardize with OpenTelemetry

Create a trace for each harness session, a span for each task, and sub-spans for each verification step. Use standard attributes to annotate key information. This way observability data integrates with standard toolchains (Jaeger, Zipkin).

Anthropic's Three-Agent Architecture Experiment

Anthropic's March 2026 report gives the updated DAW run of the same three-agent architecture ("build a browser-based DAW using the Web Audio API"). The first version, built for Opus 4.5, used a generator working in sprints and an evaluator grading each sprint. The updated version, run on Opus 4.6, removed the sprint construct: the builder works continuously, and the evaluator reviews after each build round. The table below is the updated run:

Agent & Phase	Duration	Cost
Planner	4.7 min	$0.46
Build round 1	2 hr 7 min	$71.08
QA round 1	8.8 min	$3.24
Build round 2	1 hr 2 min	$36.89
QA round 2	6.8 min	$3.09
Build round 3	10.9 min	$5.88
QA round 3	9.6 min	$4.06
Total	**3 hr 50 min**	**$124.70**

Each role plays a clear part in observability. The role descriptions below cover the first version; the updated version removes the generator's sprint layer:

Planner: Receives a 1-4 sentence user requirement and expands it into a full product spec. It was instructed to "be bold in scope" and "focus on product context and high-level technical design rather than detailed technical implementation." The reasoning: if the planner prematurely specifies granular technical details and gets them wrong, those errors cascade downstream. A better approach is to constrain deliverables and let the agent find its own path during execution.

Generator: Implements feature by feature, sprint by sprint. Before each sprint, it negotiates a sprint contract with the evaluator defining what "done" means for that feature block. It then

implements according to the contract, self-evaluates, and hands off to QA.

Evaluator: Uses Playwright MCP to interact with the running app like a real user — testing UI functionality, API endpoints, and database state. It scores each sprint across four dimensions: product depth, functionality, visual design, and code quality. Each dimension has a hard threshold — if any falls short, the sprint fails and the generator receives detailed feedback for fixes.

Example feedback from QA round 1: "This is a visually impressive app with good AI integration, but several core DAW features are presentational only, lacking interaction depth: clips can't be dragged/moved, there's no instrument UI panel (synth knobs, drum pads), and no visual effects editor (EQ curves, compressor meters)." These aren't edge cases — they're the core interactions that make a DAW usable. Specific, evidence-backed feedback — not "it doesn't feel right."

The evaluator wasn't always this sharp. Early versions would identify reasonable issues, then talk themselves into dismissing those issues as not severe, ultimately approving the work. The fix: read the evaluator's logs, find the points where its judgment diverged from human judgment, and update the QA prompt to address those specific problems. After several rounds of this development loop, the evaluator's scoring became reliable.

> Source: Anthropic: Harness design for long-running application development

---

Key Takeaways

- **Observability is a harness architecture property.** It's not a feature you add after the fact — it's a core capability that must be designed in from the start.
- **Both observability layers are essential.** Runtime signals explain "what happened," process artifacts explain "why it was done this way."
- **Sprint contracts front-load alignment.** They prevent the generator from building something the evaluator immediately rejects for foreseeable reasons.
- **Scoring rubrics make evaluation reproducible.** Different evaluators produce similar scores for the same output.
- **Observability turns a vague rejection into a specific, fixable defect.**

Further Reading

- Observability Engineering - Charity Majors — Theory and practice framework for modern observability engineering
- Dapper - Google (Sigelman et al.) — Groundbreaking practice in large-scale distributed tracing
- Harness Design - Anthropic — Introducing sprint contracts and evaluator rubrics
- Site Reliability Engineering - Google — Systematic application of observability in production systems

Exercises

1. **Observability Gap Analysis**: Audit your current harness for system-layer and process-layer observability. Find system states that can't be distinguished from existing signals, and propose additions.
2. **Sprint Contract Practice**: Write a sprint contract for a real task. Have the agent execute according to the contract, and compare efficiency and quality with and without the contract.
3. **Task Trace Construction**: Record every step an agent takes during a complete coding task. Annotate using OpenTelemetry semantic conventions. Analyze the trace for information bottlenecks — which steps lack sufficient signal support for their decisions.

中文版 →

> Code examples: code/ Practice project: Project 06. Build a Complete Agent Workspace

Lecture 12. Leave a Clean Handoff at the End of Every Session

Your agent runs all afternoon, modifies 20 files, commits the code, and the session ends. The next agent session starts up and immediately discovers: build is broken, tests are red, temporary debug files are scattered everywhere, the feature list hasn't been updated, and progress is completely opaque. The first 30 minutes of the new session are spent entirely on "figuring out what the last session actually did."

Both OpenAI and Anthropic state clearly: **long-term reliability depends on operational discipline, not just single-run success.** The quality of state at the end of each session directly determines the next session's efficiency.

Entropy Growth Is the Default State

Lehman's laws of software evolution tell us that systems undergoing continuous change will inevitably grow more complex unless actively managed. This is especially true for AI coding agents. Every session introduces changes, and without cleanup at exit, technical debt accumulates exponentially.

During five months of Codex experiments, OpenAI observed something striking: **agents copy patterns already present in the repository, even when those patterns are inconsistent or suboptimal.** Over time, this copying inevitably leads to drift. The first person leaves a coffee cup in the common area; the second person figures "it's already messy" and leaves another; a week later the table is buried under cups. A codebase works the same way.

The OpenAI team initially set aside Fridays — one workday, about 20% of the week's capacity — for manually cleaning up "AI slop." As throughput grew, manual cleanup cost more than it returned. They eventually arrived at a systematic solution:

1. **Encode "golden rules" into the repository**: Rules like "prefer the shared utility package over hand-rolled ad-hoc helpers" (keep invariants centralized) and "don't YOLO-guess

data structures" (validate boundaries or depend on typed SDKs). These rules are concrete, mechanical, and automatically checkable.

2. **Establish periodic cleanup workflows**: A fleet of background Codex tasks that regularly scan for deviations, update quality scores, and open targeted refactoring PRs. Most can be reviewed and auto-merged within a minute.
3. **Capture human taste once, enforce it continuously**: Review comments, refactoring PRs, and user-facing bugs are all translated into documentation updates or encoded directly into tooling. When documentation isn't enough, promote the rule into code.

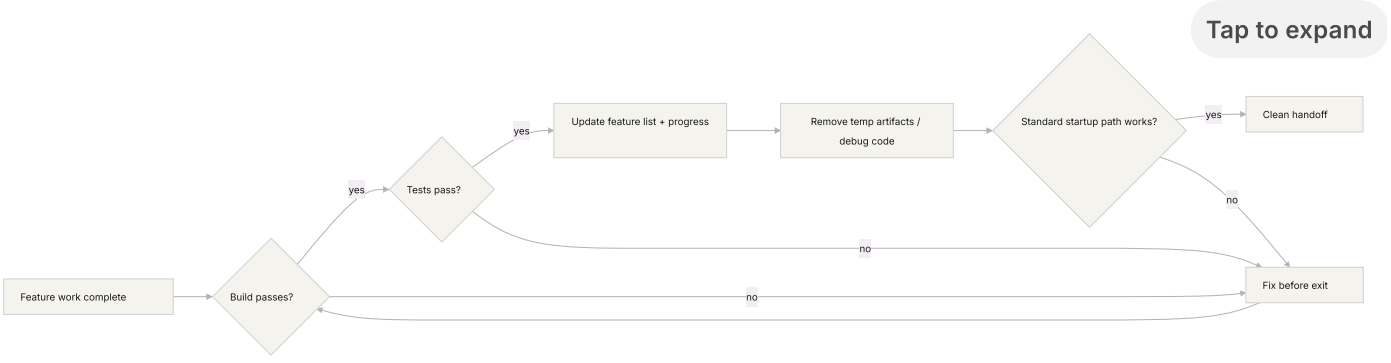
Technical debt is a high-interest loan. Continuously paying it off in small increments is almost always better than letting it accumulate into one massive payoff event.

> Source: OpenAI: Harness engineering: leveraging Codex in an agent-first world

---

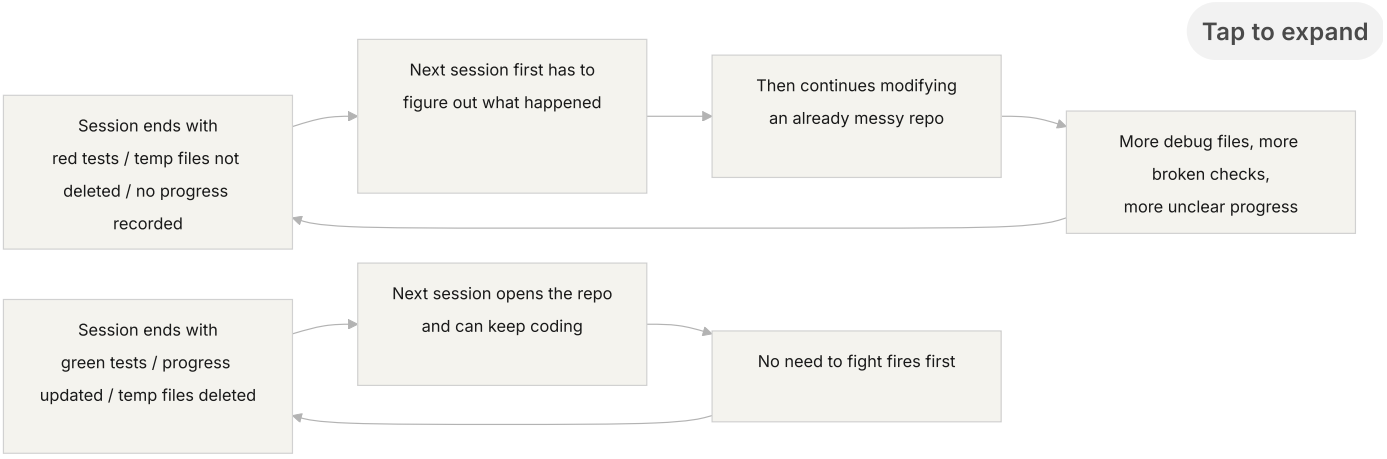
Clean State: More Than "The Code Compiles"

Clean state isn't simply "the code compiles." Building without errors is the most basic requirement — the next session shouldn't have to fix build errors first. All tests must pass too, including tests that existed before the session; the session is responsible for not breaking existing functionality. And this should be verified in CI, not just "works on my machine."



But that's still not enough. Current progress must be recorded in machine-readable artifacts: completed subtasks with their passing criteria, in-progress but incomplete subtasks with current state, and not-yet-started subtasks. Record progress and verification results in versioned files so the next session can inspect state. Temporary artifacts — debug logs,

temporary files, commented-out code, TODO markers — must also be cleaned up, because they increase cognitive load for the next session. The standard startup path must remain functional too. Can the next session start working without manual intervention? Environment initialization, codebase loading, context acquisition, task selection — none of these paths can be broken.



Core Concepts

- **Clean state**: The system must satisfy five conditions at session end — build passes, tests pass, progress recorded, no stale artifacts, startup path available. Missing any one means the session isn't "done."
- **Session integrity**: Analogous to database transactions — either fully commit and leave a clean state, or roll back to the last consistent state. No middle ground.
- **Quality document**: An active artifact that continuously records quality ratings for each module. Not a one-time assessment, but a tracker showing whether the codebase is getting stronger or weaker over time.
- **Cleanup loop**: A regular maintenance session aimed at systematically reducing entropy in the codebase. Not an emergency fix, but routine operations.
- **Harness simplification**: As model capabilities improve, periodically remove harness components that are no longer necessary. A constraint essential today may be unnecessary overhead in three months.
- **Idempotent cleanup**: Cleanup operations produce the same result regardless of how many times they run, ensuring cleanup remains safe even in failure-retry scenarios.

"Clean Up Later" Means Never Clean Up

The most common mental trap is "no time to clean up this session, I'll do it next time." But the next agent session doesn't know what you left behind — it sees a mess of code and uncertain state. It'll spend significant time inferring "which parts of this code are intentional and which are temporary."

Worse, every session has its own task objectives. The new session is there to do new work, not clean up the previous session's mess. It'll ignore the chaos and start new work on top of it, introducing even more chaos. This is entropy's positive feedback loop.

For a hypothetical 12-week project, suppose the following values:

- Week 1: Build pass rate 100%, test pass rate 100%, new session startup 5 min
- Week 4: Build 95%, tests 92%, startup 15 min
- Week 8: Build 82%, tests 78%, startup 35 min
- Week 12: Build 68%, tests 61%, startup 60+ min

Same project with a cleanup strategy:

- Week 1: 100%, 100%, 5 min
- Week 12: 97%, 95%, 9 min

A weekly cleanup loop keeps the starting state stable: each session ends with build, tests, progress, and temporary artifacts in order, so the next session starts from the same baseline.

How to Do It

1. Clean State Is a Necessary Condition for Completion

Define explicitly in the harness: **session completion = task passes verification AND clean state check passes.** Missing either one means the session isn't complete. Write in CLAUDE.md:

```
## Session Exit Checklist
- [ ] Build passes (npm run build)
- [ ] All tests pass (npm test)
- [ ] Feature list updated
- [ ] No debug code remaining (console.log, debugger, TODO)
- [ ] Standard startup path available (npm run dev)
```

2. Dual-Mode Cleanup Strategy

Combine two cleanup modes:

Immediate cleanup (at end of every session): Clean up temporary artifacts created during the session, update feature list state, ensure build and tests pass. This is "reference counting" cleanup — clean up as soon as you're done using something.

Periodic cleanup (weekly): Full-system scan — handle accumulated structural issues, update quality documents, run benchmark tests to detect drift. This is "tracing" cleanup — a comprehensive maintenance pass done on a regular cadence.

3. Maintain a Quality Document

A quality document is an active artifact that continuously scores each module:

markdown

```
# Quality Document

## User Authentication Module (Quality: A)
- Verification passing: Yes
- Agent understandable: Yes
- Test stability: Stable
- Architecture boundaries: Compliant
- Code conventions: Followed

## Payment Module (Quality: C)
- Verification passing: Partial (payment callback untested)
- Agent understandable: Difficult (logic spread across 3 files)
- Test stability: Unstable (2 flaky tests)
- Architecture boundaries: Violations present
- Code conventions: Partially followed
```

New sessions read this document and immediately know where to prioritize. Fix the lowest-scoring module first.

4. Periodically Simplify the Harness

Every harness component exists because the model couldn't reliably do something on its own. But as models improve, these assumptions become outdated.

Anthropic's harness report shows this directly. The first three-agent harness, built for Opus 4.5, had the generator work in sprints. When Opus 4.6 shipped, the model could handle work decomposition, so the sprint construct became unnecessary. In the updated DAW run, the builder worked continuously for over two hours without drifting.

But the evaluator told a different story. Even with Opus 4.6's stronger capabilities, when tasks approached the model's capability boundary, the evaluator still provided real value — catching the generator's missing functionality and stub implementations. This means the evaluator isn't a fixed yes/no decision; it depends on where task difficulty sits relative to model capability.

Recommended practice: Every month, pick one harness component, temporarily disable it, and run benchmark tasks. If results don't degrade, remove it permanently. If they do, restore it or replace it with a lighter alternative.

A deeper principle: **as models improve, the interesting combinations in a harness don't shrink — they shift.** Problems that previously required explicit solutions get absorbed by model capabilities, but new capability boundaries open up harness design spaces that were previously impossible. The AI engineer's job is to continuously find the next valuable combination.

5. Cleanup Operations Must Be Idempotent

Cleanup scripts should be safe to run repeatedly — running them one more time shouldn't produce side effects:

```bash
# Idempotent cleanup operations
rm -f /tmp/debug-*.log  # -f ensures no error when files don't exist
git checkout -- .env.local  # Restore to known state
npm run test  # Verify cleanup didn't break anything
```

6. High Throughput Changes the Merge Philosophy

When agent output far exceeds human review capacity, the traditional merge philosophy needs adjustment. The OpenAI team's experience: in an environment where an agent opens 3.5 PRs per day (and later even more), minimizing blocking merge gates is the right call. PRs should be short-lived; test flakiness is usually resolved with subsequent runs rather than indefinitely blocking progress. In a system where the cost of fixing is low and the cost of waiting is high, moving fast with fast fixes is a better strategy than slow confirmation.

Caveat: This is irresponsible in a low-throughput environment. But when agent output far exceeds human attention, it's often the correct tradeoff. The key criterion: **average cost of fixing a bug vs. average cost of waiting for a human to review a PR.** When the former is lower than the latter, fast merging is the right call.

Real-World Case

An Electron app developed with agents over 12 weeks, comparing two approaches:

Without cleanup strategy (control group): Week 12, build pass rate 68%, test pass rate 61%, new session startup 60+ min, 103 stale artifacts.

With cleanup strategy (experimental group): Full clean-state check at every session end, plus a weekly cleanup loop. Week 12, build pass rate 97%, test pass rate 95%, new session startup 9 min, 11 stale artifacts.

By week 12, the cleanup group's build and test pass rates remained close to the starting point, a new session could start quickly, and stale artifacts were few. Cleanup took a few minutes per session and returned a starting state the next session could trust.

Key Takeaways

- **Clean state is a necessary condition for session completion** — not optional housekeeping, but part of the "definition of done."
- **All five dimensions are non-negotiable** — build, tests, progress, artifacts, startup — each must be explicitly checked.

- **Quality documents make codebase health trackable** — you can only proactively fix what you know is degrading.
- **Periodically simplify the harness** — as model capabilities improve, remove constraints that are no longer needed.
- **"Clean up later" equals never cleaning up.** Entropy growth is the default state; only active cleanup counteracts it.

Further Reading

- Clean Code - Robert C. Martin — Systematic principles of code cleanliness
- Harness Engineering - OpenAI — Reproducibility as a core harness design requirement
- Effective Harnesses - Anthropic — The critical role of clean session exits for long-term reliability
- Programs, Life Cycles, and Laws of Software Evolution - Lehman — Software evolution laws proving system complexity inevitably grows without active maintenance

---

Exercises

1. **Clean State Checklist**: Design a session exit checklist for your codebase covering all five dimensions. Apply it across 5 consecutive sessions and record the number of violations per dimension.
2. **Benchmark Comparison**: Use a fixed task set with two harness variants (with/without clean state requirements). Compare completion rate, retry count, and defect escape rate.
3. **Harness Simplification Practice**: Pick one harness component, temporarily disable it, and run benchmark tasks. Compare results with and without it. Decide whether to keep, remove, or replace.

中文版 →

> Code examples: code/ Practice project: Project 07. Build Your First Automated Loop

Lecture 13. From Manual Prompting to Autonomous Loops

Everything you learned in the first twelve lectures rests on one assumption: **you are sitting at the keyboard, typing instructions one at a time.**

You wrote `AGENTS.md` (Lectures 1–4), built state management (Lectures 5–6), constrained scope with feature lists (Lectures 7–8), left clean handoffs at session end (Lectures 9, 12), and made the runtime observable (Lectures 10–11). But the trigger for all of it was always you. The agent never decided on its own when to start working — because no one pressed "start."

This lecture is about handing the start button to the system. Not giving up control — elevating it to the next layer.

/goal: The Simplest Possible Loop

The best entrance to loop engineering is not a complex architecture diagram — it's a single command.

In early 2026, Claude Code and OpenAI Codex independently shipped the same feature: `/goal`. You type in the terminal:

```
/goal "All tests pass, zero lint warnings, merge to main"
```

Then you close your laptop and go to sleep. Eight hours later, the agent has analyzed, coded, tested, fixed, and merged on its own. It retries on failure, switches approaches when stuck, and stops when done — without you leaning over its shoulder saying "try again."

The only difference between `/goal` and a traditional prompt is one thing. But that one thing changes everything:

	Traditional Prompt	`/goal`
What you provide	What to do next	What the end state looks like
What the agent does	Execute once	Loop until achieved
Who judges done	You	A verifiable stopping condition
When you can walk away	You can't	The moment you type `/goal`

`/goal` is essentially a loop. It has exactly three parts: **a goal, a verification method, and a stopping condition.** Just those three things move you from inside the loop to outside it.

How `/goal` Grew Organically

`/goal` didn't jump from 0 to 1 out of nowhere. It grew gradually out of everyday workflows, going through roughly four stages:

Stage 1: Manual one-by-one prompting. The earliest way of working was back-and-forth: "write a function," "add a test," "fix this logic." The agent stopped after every step and waited for you to say what's next. You were the scheduler of the entire pipeline.

Stage 2: Long prompts with multiple steps. Then people started writing longer prompts that stacked steps together: "first analyze the code, then write the implementation, then run tests, and if they fail, fix them." The agent could run several steps in one go, but you still had to watch — because it might drift mid-way, or finish a step and not know what to do next.

Stage 3: Agent self-reflection and self-direction. After that, agents gained "introspection" — after each step they'd look at the result and decide what to do next. You gave a goal, and they broke it down themselves and retried on their own. But a problem emerged: when do they stop? Does "I'm done" coming from the agent itself count? Practice kept answering — no. Agents declare victory far too easily.

Stage 4: Independent stopping judgment — `/goal`. The final step was taking "judging whether it's done" out of the hands of the agent doing the work, and handing it to an independent judge. It could be a different model, a script, or a test command — but the rule was the same: the person writing the code can't grade their own homework. At this point, `/goal` truly worked: you give the goal, it loops, an independent judge decides when to stop, and you can walk away.

These four stages weren't a roadmap planned by any one company. They were the path everyone coding with agents arrived at, independently, pushed by the same pain points. Claude Code and Codex shipping `/goal` almost simultaneously in early 2026 wasn't a coincidence — the time had come.

There's More Than One Kind of Loop

`/goal` is the easiest loop to understand, but it's not the only kind. Loops fall into categories based on how they're triggered and how they stop:

Type	Trigger	Stop Condition	Claude Code	Codex	Best For
Turn-based loop	You type each prompt manually	Agent thinks it's done, or you interrupt	Normal chat	Normal chat	Small tasks, exploratory work
Goal-based loop	You give a goal	Independent evaluator confirms done, or max turns reached	`/goal`	`/goal` (manual enable required)	Complex tasks with clear completion criteria
Time-based loop	Scheduled interval (every N minutes/hours)	You stop it manually, or it exits after completing the work	`/loop`	Thread automation	Polling status, periodic checks, recurring work
Event-driven loop	External event (PR opened, CI failed, new issue)	Stops after handling the event, or hits retry limit	Routines (API / GitHub Webhook)	Standalone automation + plugins	Reactive workflows, CI/CD integration

These aren't competing — they're different tools for different jobs. Turn-based is fine for small things. Use `/goal` when there's a clear finish line. Use `/loop` when you need to watch something. Use event-driven when you're integrating with external systems.

Don't Confuse `/goal` and `/loop`

Both have "loop" in the name, but they solve completely different problems:

	`/goal`	`/loop`
What it is	One big task, runs until it's done	One small action, repeats on an interval
Stop condition	Goal reached, or budget exhausted	You stop it manually, or the task exits on its own
Time profile	One long run, can take hours or days	Periodic short bursts, each run may be a few minutes
Progress	Gets closer to the finish line each iteration	Each run is independent, no cumulative progress
Analogy	Running a marathon — start gun fires, you stop at the finish line	An alarm clock — rings on a schedule, you turn it off
Typical use	"Implement the full payment system with test coverage"	"Check if CI is broken every 15 minutes"

A common mistake: shoving something that should be a `/goal` into a `/loop`. Like writing `/loop 10m "keep implementing the payment system"` — that's wrong. `/loop` runs the same instruction independently each time, it doesn't remember where it left off last time. You'll just get the same starting point over and over.

One-sentence test for which one to use: does this thing have an end?

- Has an end → `/goal`
- No end, you just need to keep watching → `/loop`

Loop Engineering, the subject of this lecture, isn't about any one command. It's about **being able to design systems that include all of these types — so your agent can keep working even when you're not there.**

You don't have to type `/goal` every time. But understanding where it came from and why it looks the way it does — that's understanding the core of loop engineering. More complex loops just add parts like scheduling, parallelism, isolation, and memory on top of these same three fundamentals: goal, verification, stopping condition.

Different Dates, the Same Direction

The same idea surfaced in different places, at different times.

Peter Steinberger (creator of OpenClaw, in a post): "You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents."

Boris Cherny (head of Claude Code at Anthropic, on the Acquired podcast): "I don't prompt Claude anymore. I have loops running that prompt Claude and figure out what to do. My job is to write loops."

Addy Osmani (engineering lead at Google Chrome) named the concept on June 7, 2026, and gave it a one-line definition:

> **Loop engineering is replacing yourself as the person who prompts the agent. You design the system that does it instead.**

Cherny wrote in a 2025-12-27 personal report that over 30 days he landed 259 PRs — 497 commits, 40,000 lines added, 38,000 removed — and that every line was written by Claude Code + Opus 4.5.

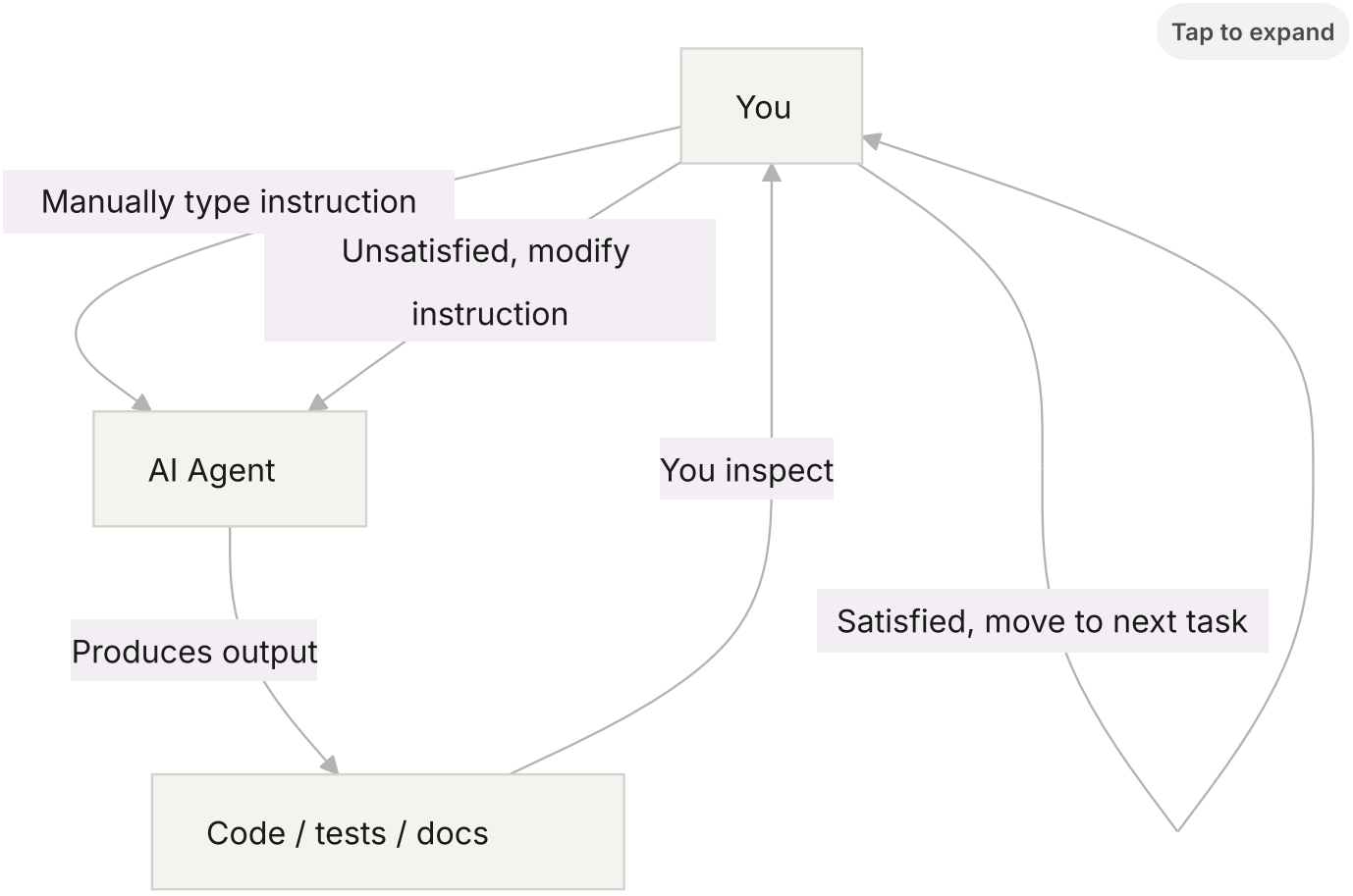
The quotes come from different dates, formats, and teams. Their shared direction is to move scheduling, verification, and the stopping decision out of the moment-to-moment prompt and into a system. By mid-2026, agents could finish non-trivial tasks unattended, scheduling primitives were built in, and a run was cheap enough to repeat on a timer. When the pieces are present, more than one person puts them together.

> Source: Addy Osmani: Loop Engineering

Inside the Loop vs. Outside the Loop

Let's contrast two concrete scenarios.

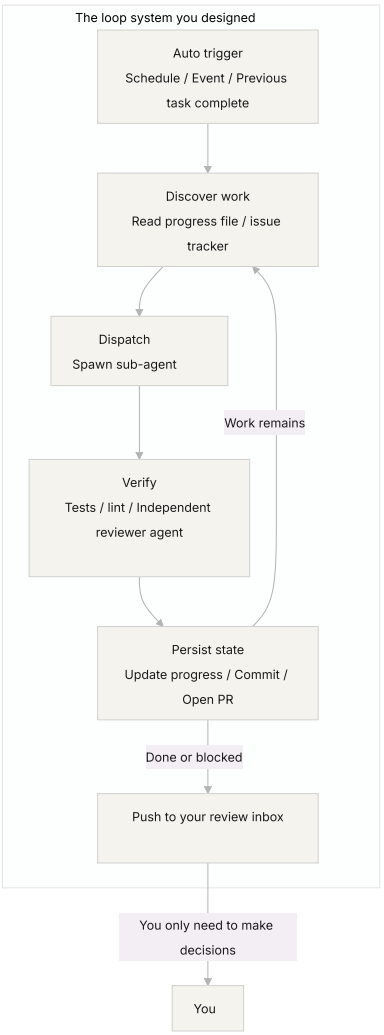
Scenario A: You are inside the loop (Lectures 1–12).



You have a complete harness: `AGENTS.md` tells the agent the project rules, `feature_list.json` constrains scope, `init.sh` ensures consistent environment, `claude-progress.md` records progress. **But every step still requires your manual initiation.** Finish one feature, read the progress file, think about what's next, type the instruction. You are the engine of the entire workflow.

Scenario B: You are outside the loop (Loop Engineering).

Tap to expand



You don't type instructions anymore. The system you designed discovers the work, dispatches it, verifies the results, records the state, and decides the next step. Your job shrinks to three things: **define the goal and stopping condition before it starts, review the output after it finishes, and adjust the rules when the system veers off course.** The leverage moves from "writing the right prompt" to "designing the right loop."

> Osmani: "A year ago if you wanted a loop you wrote a pile of bash and you maintained that pile forever and it was yours and only yours. Now the pieces just ship inside the products." You don't need to build from scratch. You need to understand how the pieces fit together.

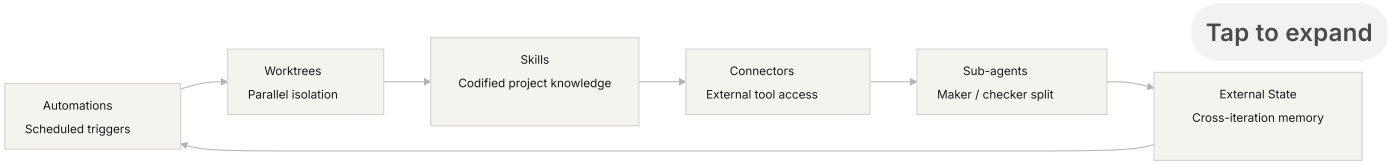
Core Concepts

- **Loop Engineering**: Designing a system that automatically prompts your agent, replacing manual step-by-step human input. The human moves from inside the loop to outside it, and the leverage shifts from "writing the right prompt" to "designing the right loop."
- `/goal` **mode**: The simplest possible loop — provide a goal, verification method, and stopping condition; the agent loops until met. The bridge from manual prompting to autonomous loops.
- **Generator/Evaluator Separation**: The agent that writes the code and the agent that checks it must be separated. A model grading its own work is untrustworthy; an independent verifier — sometimes using a different model entirely — is the baseline reliability guarantee of any loop.
- **Worktree Isolation**: Each parallel agent works in an independent git worktree, physically preventing file collisions. The infrastructure prerequisite for multi-agent parallel execution.
- **External State**: Memory that lives outside a single conversation — markdown files, issue trackers, kanban boards. Models forget everything between runs; memory must live on disk.
- **Four Silent Costs**: Four hidden costs that grow sharper the longer a loop runs — verification debt, comprehension rot, cognitive surrender, token blowout. Loops accelerate not just output, but risk.

The Six Primitives of a Loop

Osmani decomposed a loop into five core building blocks, plus one memory layer that threads through all of them — six things in total, but the memory layer occupies a special status: it's not a component on the same level as the others; it's the spine that everything else depends on.

The diagram below draws all six as a ring so you can see the full picture at a glance. But remember: External State isn't just another stop on the loop — it's the foundation the whole loop rests on.



1. Automations — The Heartbeat

Without automation, a loop isn't a loop — it's a one-off run you did manually.

Both Claude Code and Codex have full scheduling systems, but they use different names and layers. Roughly mapping from lightest to heaviest:

Layer	Claude Code	Codex	Notes
In-session polling	`/loop`	Thread automation	Tied to current session, dies when session closes
Local scheduled tasks	Desktop scheduled tasks	Standalone automation (local mode)	Runs while machine is on, can access local files
Cloud scheduled tasks	Cloud Routines	— (no native cloud scheduler)	Runs while machine is off
Event triggers	Routines (API / GitHub Webhook)	Standalone automation + plugins	Triggered by external events
Fully self-hosted	GitHub Actions / self-hosted cron	`codex exec` + cron	Full control

Codex's Automations tab is the scheduling entry point. Pick the project, the prompt, the cadence, and whether it runs on your local checkout or a background worktree. Runs that find something land in a Triage inbox; runs that find nothing auto-archive. OpenAI uses them internally for daily issue triage, CI failure summaries, commit briefings, and hunting for bugs introduced last week.

Codex automations come in two flavors:

- **Thread automation** — Heartbeat-style recurring wake-up calls attached to a thread, preserving context. Good for continuous follow-up on a single thing, like monitoring a

long-running command or polling PR status. The equivalent in Claude Code is `/loop`.

- **Standalone automation** — Each run starts fresh, results go to Triage. Good for daily/weekly independent tasks like briefings or dependency scans. The equivalent in Claude Code is Desktop scheduled tasks.

Claude Code's system is layered more granularly:

- `/loop` — Lightweight in-session scheduled repetition. Works while your terminal is open, dies when the session closes, auto-expires after 7 days. Good for temporary monitoring during your current work session.
- **Desktop scheduled tasks** — Runs while your machine is on, survives session restarts, minute-level intervals. Good for recurring work that needs local file access.
- **Cloud Routines** — Runs on Anthropic's cloud infrastructure, survives your machine being off, 1-hour minimum interval. Supports three trigger types: scheduled, API call, GitHub webhook. Good for daily tasks that don't need your local environment.
- **GitHub Actions / self-hosted cron** — Fully under your control, runs however you want. Good for scenarios with special environment or security requirements.

```bash
# Claude Code: run tests every 30 min, fix failures (within current session)
/loop 30m Run the test suite and fix any failing tests

# Claude Code: check deploy status every 15 minutes
/loop 15m Check if the production deploy succeeded and report status
```

Automations are the heartbeat. Without them, the loop is a blueprint that never wakes up.

2. Worktrees — Isolation at Scale

As soon as you run more than one agent, file collisions become the inevitable failure mode. Two agents writing to the same file is exactly the headache of two engineers committing to the same lines without consulting each other.

`git worktree` solves this: each agent works on its own branch in its own directory. They physically cannot touch each other's checkout.

Claude Code and Codex both ship worktree support. When you use `--worktree` or `isolation: worktree` on a sub-agent, each helper gets a clean, independent checkout that

cleans up after itself. Worktrees remove the mechanical collision problem — but remember: **your review bandwidth is still the ceiling.** How many parallel agents you can supervise determines how many worktrees you can actually run.

3. Skills — Stop Re-Explaining Your Project

A skill is how you stop re-explaining the same project context every session. It's a folder containing a `SKILL.md` with instructions and metadata, plus optional scripts, references, and assets.

Codex and Claude Code support the same format. Skills are invoked directly with `/skill-name` (Codex also supports `$skill-name`), or triggered implicitly when the task matches the skill description.

Skills are fundamentally about paying your intent debt. An agent starts every session cold — it fills any gap in your intent with a confident guess. A skill is that intent written down on the outside: the conventions, the build steps, the "we don't do it this way because of that one incident" — written once, read every run.

4. Connectors — Your Loop Touches Real Tools

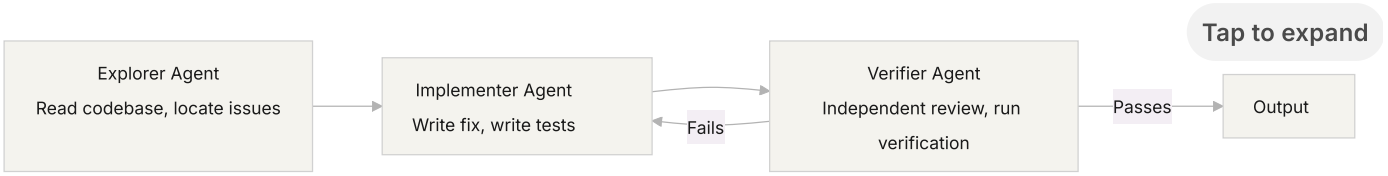
A loop that can only see the filesystem is a small loop. Connectors (built on the MCP protocol) let the agent read your issue tracker, query a database, hit a staging API, drop a message in Slack.

Codex and Claude Code both speak MCP, so the connector you wrote for one usually works in the other. Connectors are the difference between "here is the fix" and a loop that opens the PR, links the Linear ticket, and pings the channel once CI is green — by itself, inside your actual environment, not just in a terminal.

5. Sub-agents — Keep the Maker Away from the Checker

The most structurally valuable design choice in a loop is splitting the one who writes from the one who checks. The model that wrote the code is way too generous grading its own homework. A second agent, with different instructions and sometimes a different model, catches what the first agent talked itself into.

The classic three-role split:



Claude Code's `/goal` runs this under the hood — a fresh, independent session judges whether the loop should stop, not the session that did the work. This is called **generator/evaluator separation**, and it is the single most important reliability guarantee in loop design.

6. External State — The Loop's Memory

Models forget everything between runs. Memory must live on disk, not in the context window.

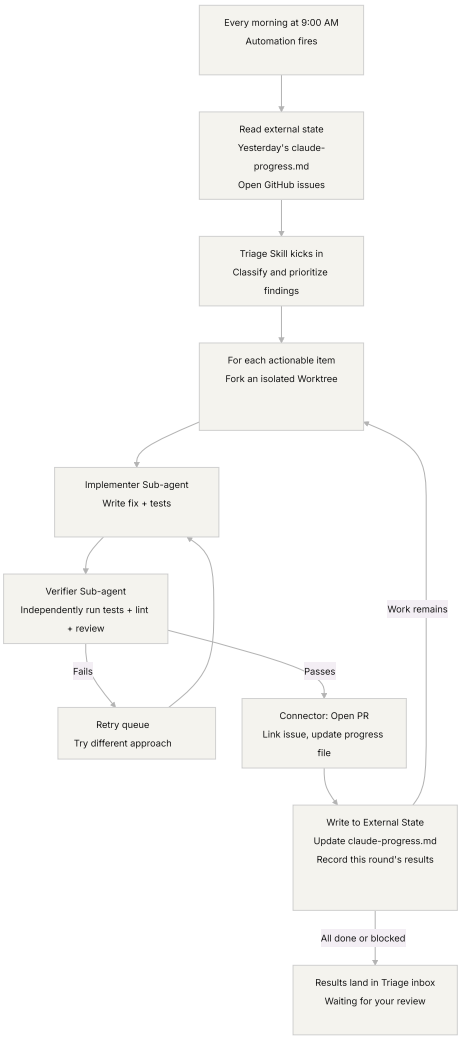
This sounds too simple to matter, but it's the same trick every long-running agent depends on. A markdown file, a Linear board — anything that lives outside a single conversation and holds what is done, what is in progress, and what is next. The agent forgets. The repository doesn't.

These six primitives are your loop design toolkit. You don't need all of them for every loop. But you need to know when to reach for which one.

A Complete Loop, Anatomized

Wire all six together and here's what a real morning triage loop looks like:

Tap to expand



This is no longer a single agent run. It is a continuously operating system that wakes up every morning, sweeps the floor on its own, and puts the things that need your attention in front of you. Your role becomes: **review the inbox contents, make decisions, and when you spot a pattern the system can't handle, refine the skills and rules.**

Cherny's 30-day report is one data point for this pattern: 259 PRs, with the code written by Claude Code + Opus 4.5. OpenAI's harness account is another: a small team used Codex on a greenfield repository for five months and reported about one million lines and roughly 1,500 PRs.

Generator/Evaluator Separation: Why You Can't Let the Model Grade Its Own Work

This is the hardest lesson in loop engineering.

Your smartest agent writes a beautiful piece of code. The logic is clear, the comments are thorough, and every function has a test. You are satisfied.

But here's the question: **if you let the agent that wrote that code judge whether it did a good job, what will it say?**

The answer has been confirmed by experience again and again: it will give itself a high score. Not because it's dishonest, but because it is the author — it convinced itself this path was correct during generation. When it looks back, it doesn't see mistakes; it sees its own reasoning process.

This is not a Claude problem. This is not a GPT problem. This is a property of all generative models. **A model is its own output's best defense attorney.**

The fix: never let the same entity (same model, same prompt) do both the work and the review.

- Claude Code's `/goal` uses an independent supervisor session to judge whether the goal is met — not the session that attempted it.
- Codex's sub-agent system lets you define a verifier agent using a different model with different reasoning effort.
- The community practice of "adversarial verify" spawns N independent skeptics per finding, each prompted to refute — majority rejection kills the finding.

One sentence to remember: **someone in your crew must not believe you.**

Karpathy's autoresearch: The Loop Exemplar

If you want to see what a well-designed, actually-running loop looks like, Karpathy's autoresearch is the textbook example.

In March 2026, Karpathy released autoresearch. The repository is deliberately small: `prepare.py` is fixed infrastructure, `train.py` is the file the agent edits (about 630 lines), and `program.md` is the research methodology the human edits. Give it one GPU and a research direction, and it runs overnight — each experiment gets a fixed 5-minute training wall-clock budget (excluding startup and compilation), and only changes that improve `val_bpb` are kept.

Three Files, Three Roles

The whole system has just three core files, but the division of labor is razor-sharp:

File	Who Edits It	What It Does
`prepare.py`	Nobody (read-only)	Data prep, tokenizer, eval harness. Fixed infrastructure.
`train.py` (~630 lines)	**AI Agent**	Model definition, optimizer, training loop. The agent's playground — change anything.
`program.md`	**You**	Research methodology written in natural language. You only edit this. Tell the agent how to explore, how to evaluate, what not to touch.

This three-way split is the soul of the design: **humans don't touch code, they touch direction; agents don't touch direction, they touch code.** Your job shifts from writing Python to "writing the research org culture."

Input: What program.md Looks Like

`program.md` is the brain of the loop. It's not code — it's a research instruction manual written in Markdown. It roughly contains:

- **Goal**: optimize `val_bpb` (validation bits per byte, lower is better)
- **Constraints**: don't touch `prepare.py`, stay within VRAM budget, fixed 5-minute training
- **Exploration directions**: try different architectures, optimizers, LR schedules
- **Evaluation rules**: what counts as improvement, how to log results, what to do on failure
- **Iron rule**: never stop. Once the loop starts, keep going forever

Your kickoff prompt to the agent can be as short as one sentence:

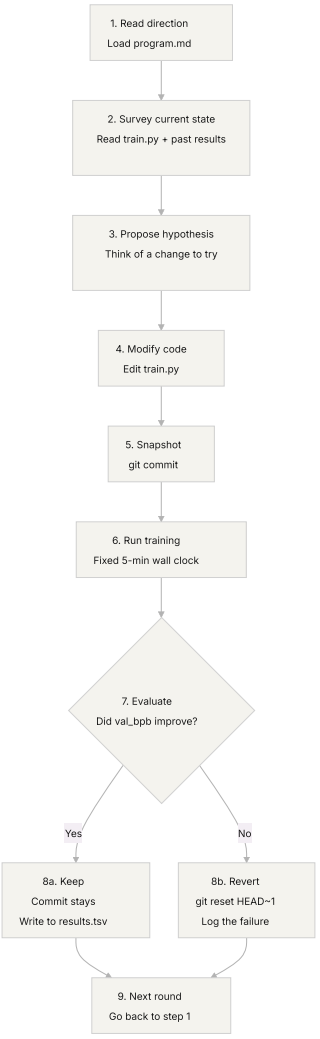
```
Have a look at program.md and let's kick off a new experiment!
```

The rest is up to the agent reading the doc and making its own decisions.

The Nine-Step Ratchet Loop

At the heart of autoresearch is a **ratchet** — it only moves forward, never backward. Each iteration strictly follows nine steps:

Tap to expand



The README estimates about 12 experiments per hour, or roughly 100 in an 8-hour overnight run. The 5-minute budget covers training, not startup or compilation; the rest of each round goes to editing, launching, and evaluating. Karpathy reported running a private version of autoresearch autonomously for about two days for the nanochat round described below.

The fixed 5-minute training budget is a key design choice — no matter what the agent changes, the training run gets the same wall-clock budget. This means all results are directly comparable under the same training budget.

Output: What You See When You Wake Up

After a night of looping, you sit down in the morning and find three things:

1. Git history (the forward-moving ratchet)

Only commits that actually improved stay on the main branch. Everything that failed was rolled back. `git log` is a validated research log.

2. results.tsv (the full experiment record)

Every single experiment — success or failure — is logged:

```
timestamp    commit_hash    val_bpb    vram_mb    description
--------- ------------ ---------- ---------- ----------------------------
08:01:12  a1b2c3d       1.234     22100    baseline
08:06:15  d4e5f6g       1.228     22400    increased learning rate by 10%
08:11:20  (reverted)     1.241     21800    switched to GELU activation
08:16:08  h7i8j9k       1.219     23000    added weight decay 0.01
...
```

3. A research log (agent's own summary)

The agent writes clear commit messages about what it tried, what worked, what didn't, and what it plans to try next. You read those — you don't have to read the code diffs.

What It Actually Found

Results from autoresearch round 1 reduced nanochat's GPT-2-level training time on 8×H100 from 2.02 hours to 1.80 hours, a reduction of about **10.9%** in training time.

Most of the changes were small optimizations that stacked. The useful part is the loop: a fixed budget, a complete experiment log, and a rule that only verified improvements stay.

The Most Telling Detail: The Loop Is Written in English, Not Code.

`program.md` is a Markdown document, not a Python script. It describes a research methodology — what to modify, what to leave alone, how to evaluate, how to handle failure cases, and one iron rule: **never ask for human help, just keep going.** A coding agent reads this document and executes it indefinitely.

This is the template for loop engineering: don't give the agent a task. Give it a **methodology**. Let the methodology be the loop. One `program.md`, 630 lines of glue code, and everything else is the agent running itself.

Four Silent Costs

When a loop starts running, you won't see the problems immediately. The following four costs accumulate silently, and by the time you notice, you may have already paid heavily.

1. Verification Debt

Fast loops tempt you to skip verification. "Looks fine" is not the same as "confirmed correct." The more code a loop generates unattended, the faster verification debt piles up. The fix: **stopping conditions must be machine-checkable, never "feels about right."**

2. Comprehension Rot

The faster a loop ships code, the further your understanding of your own codebase drifts from reality. A loop lands hundreds of PRs; the team reads those PRs, reviews the diffs that matter, and keeps using the code, so its mental model keeps up. **Fast loops require fast reading.**

3. Cognitive Surrender

When the loop runs smoothly, the most comfortable posture is to stop having opinions. Take whatever it gives back, don't think about the output. But that is exactly where danger begins — you're using the loop to avoid thinking, rather than to amplify thinking. Osmani's warning: "Two people can build the exact same loop and get opposite results. One uses it to go faster on work they understand; the other uses it to avoid understanding the work. The loop doesn't know the difference. You do."

4. Token Blowout

Every iteration of a loop accumulates more context: code written, errors encountered, decisions made. When each round adds roughly a fixed amount of new content, the history length seen by a single round grows roughly linearly with the number of rounds. If the system reprocesses the entire history every round, the cumulative input across N rounds grows roughly quadratically. Codex addresses this with automatic context compaction — a dedicated API compresses older conversation turns into encrypted content summaries,

retaining essential knowledge while discarding redundant detail. This is an engineering concern you must address from the first loop, not a bolt-on later.

Building Your First Loop

You don't need to start with a high-throughput pipeline. Start with the smallest thing that works.

Step 1: Pick One Recurring Task

Find something you do manually at least twice a week. Examples:

- Open GitHub in the morning, check new issues, triage and respond
- Run lint and tests before every PR review
- Update progress docs at the end of each day

Step 2: Write a Goal and Stopping Condition

Turn the task into something a `/goal` can understand:

```markdown
Goal: Check the 10 most recent issues in the repo.
For each issue:
  - If it already has clear labels and an assignee, skip
  - If untagged, add appropriate labels based on content
  - If fixable in under 10 minutes, create a branch and attempt a fix
Stop when: All qualifying issues have been processed, or an issue requires human
```

Step 3: Split Maker and Checker

Don't let the same agent both write the code and judge it. Split your loop into two roles:

- Implementer: reads the issue, writes the fix, writes the tests
- Verifier: independently runs tests, reviews the diff, judges whether the fix actually solves the problem

Step 4: Add Memory

Use a markdown file to record what happened in each loop run. The next run starts by reading this file — it knows what was done, what's pending, what was blocked. This beats any complex database.

Step 5: Set a Timer

Use `/loop` or your OS cron to let the loop start without you. Begin with once a day. Observe for a week.

The Maturity Ladder

You don't need to reach the top in one jump. Loop adoption is a ladder:

1. **Level 1: Goal Runner** — You can use `/goal` to give a task with a stopping condition; the agent loops until met.
2. **Level 2: Scheduled Single-Task** — One automation runs one task on a timer (e.g., morning CI check).
3. **Level 3: Multi-Agent Loop** — Maker and checker split; each finding forks an isolated worktree.
4. **Level 4: Self-Feeding Loop** — The loop auto-discovers its next task from external state; it decides what to do next.
5. **Level 5: Fleet Orchestration** — Multiple loops run in parallel, independent but sharing a memory layer.

Most teams are currently between Level 2 and Level 3. Level 1 is the fastest path to seeing returns.

Key Takeaways

- **Loop Engineering does not replace Harness Engineering — it builds one floor above it.** The harness makes single runs reliable. The loop makes continuous runs autonomous.
- `/goal` **is the simplest possible loop:** goal + verification + stopping condition. Those three things move you from inside the loop to outside it.

- **Six primitives (Automations / Worktrees / Skills / Connectors / Sub-agents / External State) are the loop's building blocks.** Not all of them every time, but you need to know when to reach for which.
- **The maker and the checker must be separated.** A model grading its own work is untrustworthy. An independent verifier — sometimes a different model altogether — is the baseline reliability guarantee of any loop.
- **Loops make generation nearly free and leave judgment as the scarce resource.** The time you save isn't for resting. It's for making more judgments.
- **Four silent costs grow sharper as loops run longer:** verification debt, comprehension rot, cognitive surrender, token blowout. Loops accelerate output — and risk.
- **Start small.** One `/goal`, one cron, one markdown memory file. See the return, then stack upward.

Further Reading

- Addy Osmani: Loop Engineering
- Addy Osmani: Agent Harness Engineering
- Simon Willison: Designing Agentic Loops (Sep 2025)
- Karpathy: autoresearch
- Claude Code: Dynamic Workflows and Orchestration
- Loop Library (Forward Future) — Public corpus of 50 real loops
- The Neuron: Claude Code Creators on Agent Loops
- Lecture 12: Leave a Clean Handoff at the End of Every Session — The prerequisite for loops: each session leaves clean state so the next round can auto-start
- Lecture 5: Keep Long-Running Tasks Continuous Across Sessions — Prerequisite knowledge for external state and memory
- Lecture 11: Why Observability Belongs Inside the Harness — The faster a loop runs, the more you need observability to catch problems
- Lecture 8: Why Feature Lists Are Harness Primitives — Feature lists are the natural data source for a self-feeding loop to discover its next task

Exercises

1. **Turn a recurring task into a `/goal`:** Find something you manually do at least twice a week. Write down its goal, verification method, and stopping condition. Run it once with `/goal` and compare the time and quality against doing it manually. This is your first step from Harness to Loop.

2. **Separate maker and checker:** Pick a task you've previously had an agent execute. This time, write two different prompts: one for the implementer agent and one for the verifier agent (use different models — e.g., Claude for implementation, GPT for verification, or vice versa). The verifier must point out specific issues with cited evidence. Record the number and type of issues found in each mode.

3. **Give your loop a memory:** Create a markdown state file for your loop. In each iteration, write: what was done this round, verification results, status (pass/fail/blocked), and what to do next. Run three rounds and observe the behavioral difference between having and not having a memory file.

4. **Audit your loop's silent costs:** After your loop has run for an hour, assess these four metrics:

   - How much verification was "feels right" rather than "machine-confirmed"? (Verification debt)
   - How well can you explain the code your loop most recently produced? (Comprehension rot)
   - How many times did you think "I'll look later" and never looked? (Cognitive surrender)
   - How is the loop's context size trending? Is it repeating redundant information? (Token blowout)

中文版 →

> Code examples: code/ Practice project: Project 08. Draw Your Workflow as a Graph

Lecture 14. From Single Loops to Graph Engineering

Six weeks after Loop Engineering hit the mainstream, on July 18, 2026, Peter Steinberger — the OpenClaw author who told you to stop prompting your agent — posted a tweet:

> "Are we still talking loops or have we moved on to graphs yet?"

One tweet. A few hours later, ML engineer Hamel Husain published *Loop Engineering Is Dead. Enter Graph Engineering* — an article whose entire body was a single "Stop it" GIF.

Here's the twist: **both of them were joking.** One was satirizing an industry that invents a new term every six weeks; the other was riffing on the gag. But the joke survived about one weekend — courses, roadmaps, and tool stacks flooded the timeline before the weekend was over, trailed by a pile of fabricated numbers: the "+18% accuracy, −85% cost" claim is fake (the two numbers do exist, but they come from a paper about chemical piping diagrams and compare against different baselines), and the "Microsoft, Stanford, and Anthropic all discovered graph engineering at once" claim is also false. The fact-checking finds exactly one genuine "precursor": Josh Simmons, whose *We Are Entering the Graph Engineering Phase* is dated July 4 — two full weeks before the joke. **The joke made the idea trendy. It did not create the idea.**

> Sources: goddaehee: Graph Engineering Fact-Check (2026-07-30); YC Startup School 2026: Jensen Huang interview (with transcript); explainx: Graph Engineering (2026-07)

This lecture is not about adding fuel to that fire. It's about taking the term apart and seeing it clearly: **why does a single loop inevitably grow into a graph? What actually differs between a graph and a workflow? And when do you really need one, versus when you don't?**

Prompt, Context, Loop, Graph: Four Names, One Stack

In late July, an engineer going by Rohit (@rohit4verse) posted a thread organizing the last few years of AI engineering naming into a clean four-layer framework. It's the best coordinate system for understanding graph engineering:

Layer	Shapes What	Answers The Question	Key Artifacts
Prompt Engineering	The instruction	How do we tell the model what to do?	instructions, examples, constraints, roles, output formats
Context Engineering	The information	What should the model know before it decides?	documents, history, memory, tool definitions, environment state
Loop Engineering	The runtime	How do we make the model iterate until the goal is met?	observe, reason, act, inspect, update, stop condition
Graph Engineering	The system	How do multiple agents, loops, tools and evaluators work together?	nodes, edges, shared state, routing rules

Read the progression carefully: **each layer does not replace the one before it — it stacks on top.**

- After you found context engineering, you didn't stop prompting. Every iteration still needs a prompt; the loop just refreshes it as the environment moves.
- After you built loops, you didn't drop context. Every round of a loop reassembles its context.
- At the graph layer, prompt, context, and loop all survive: **every node carries its own prompt, its own context, its own tools, its own memory, its own loop.** The graph decides how the nodes connect.

Rohit's thread ends like this:

> Once an agent needs specialization, parallelism, shared state, verification and recovery, it has stopped being a loop. It's a graph.

Wait — where's harness? These four names don't include Harness Engineering, yet this whole course is about the harness. The reason is simple: Rohit was telling the story of buzzwords, his ending was the graph, and the layer in between got skipped. And even the layer harness belongs on is unsettled — explainx puts it above the loop, the Buildrix paper below it. This course settled it back in Lecture 2: the harness is the foundation; loops and graphs are built on it.

That explains a strange phenomenon: "Graph Engineering" only went viral in July 2026, yet everyone felt they'd "been doing this all along." Because a graph isn't a new invention — it's what a loop becomes when the task gets complex enough. **The name came later; the practice was already there.**

Take the Graph Apart: Nodes, Edges, State, Routing

Strip the graph down to four plain parts.

Node: a unit of work with a responsibility. It can be:

- deterministic code (run tests, compute coverage)
- a model call (generate docs)
- a tool (git commit, send a message)
- a full agent — with its own loop, able to understand goals, use tools, and retry on its own

What a node is allowed to be is the real dividing line between graph engineering and workflow engineering. More on that below.

Edge: how work hands off between nodes. It's not just "do A, then B" — an edge can express:

- **Parallelism**: after A, B and C start at the same time
- **Conditionals**: tests pass, go left; tests fail, go right
- **Failure/retry**: a node dies, it loops back into itself

- **Rollback**: verification fails, return to the implementation node three hops back

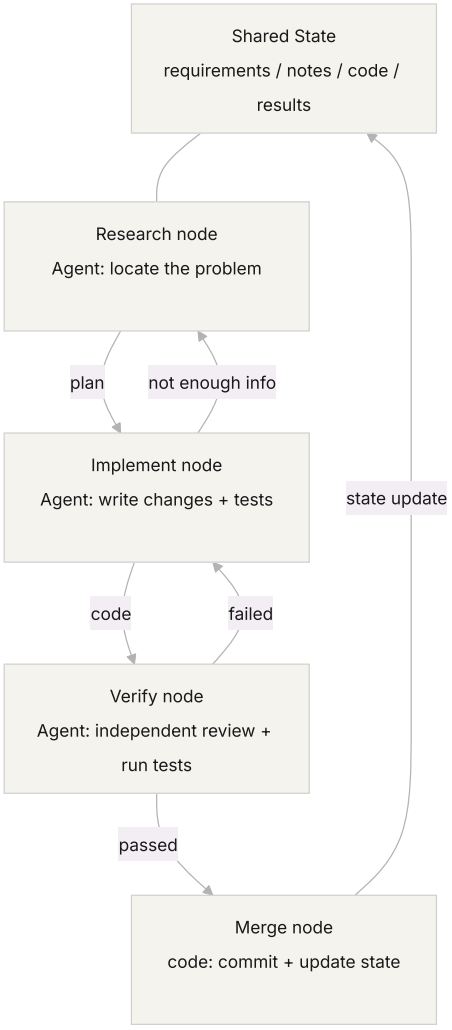
Shared State: the data package passed between nodes. Requirements, research notes, code versions, test results, review conclusions — all written to the same shared workspace. Nodes don't shout at each other; they all read and write the same state.

Routing Rules: decide where execution goes next. This is the graph's control flow, in the plainest possible terms:

> Tests pass → ship. Tests fail → back to the implementation node. Not enough information → back to the research node.

Assemble the four parts and a typical development graph looks like this:

Tap to expand



Compare this with the loop diagram from the last lecture. The loop was a ring — discover, dispatch, verify, persist, back to discover. In this lecture's graph, **the ring is still there, but it has been decomposed into explicit nodes and edges.** The verify node can bounce a failure

straight back to the implement node; the implement node can retreat to research when information is thin. Those "rollback edges" were implicit in a single loop — the agent just "remembered" it should go back, inside its own context window.

When a Loop Stops Being Enough

A single loop has one main road. In the maker-checker loop you built in Project 07, every decision — what to do next, where to go on failure — happened inside one agent's context window. Push the task a little harder and four questions surface:

1. **Division of labor**: a research agent, an implementation agent, a testing agent — who goes first?
2. **Parallelism**: which parts of the work can run at the same time?
3. **Rollback**: when tests fail, where do you go back to — the implement node, or the research node?
4. **Handoff**: how do several agents see the same requirements, notes, and test results? If the reviewer disagrees with the implementer, who wins?

Jensen Huang made a similar point in his Startup School 2026 interview with Garry Tan (Y Combinator): as implementation is increasingly automated by agents, the human's core value shifts to designing systems, defining constraints, and controlling agents at a fine granularity. His control example is concrete — "when it comes up with a plan, I change one word in a plan file and that one word makes a delta difference" — and he predicts the future core skill is "systems thinking."

The sharpest line in the whole discussion came from Luis Catacora:

> **"Loops have a lot of room for forgiveness. Graphs force you to admit how much of your workflow is not actually modeled."**

That sentence exposes the deep difference between loop and graph:

- **A loop is a deferred decision.** One agent takes on all the work; if it gets stuck, you deal with it then. The architecture can be postponed. It's cheap — but the failure modes are invisible, because the agent itself doesn't know where it's stuck.

- **A graph is an up-front decision.** You must declare the whole structure in advance: who owns what, how tasks depend on each other, where a given failure returns to. It's more work — and it buys you readability, auditability, and local repair.

Put even more bluntly: **a loop hides the problem inside the loop; a graph puts the problem on paper.** The former suits exploration, the latter suits production.

Three Structural Failures of a Single Loop at Scale

Why doesn't a single loop hold up at scale? *Graph Engineering for AI Agents: Beyond Single Feedback Loops* (eigent.ai) identifies three structural failures — structural, not bugs in any one loop.

Wait — can't a loop have checkpoints too? Sure it can. Last lecture's verification, stop conditions, even pause-and-resume — a loop can hold all of them. But the three failures below are exactly what checkpoints can't fix — because a loop's checkpoints live inside the same agent, and the checker and the producer share one brain and one context. It will stop "shipping without verification," but it won't ask "is this metric right?" or "should this goal be chased?" — the answers live in its own context, and it can't see them. A graph doesn't give you more checkpoints; it moves the check — from inside the agent to a standalone node with a fresh context (the verify node from the section above). That's what "structural" means: not a missing part in the loop, but a structure where the judge and the judged share one brain.

1. Goodhart: The Numbers Went Up, the Business Got Worse

Push any single metric hard enough and it stops measuring what it used to. The canonical case: a support team builds a loop around ticket-resolution rate. Weekly numbers climb. Months later, renewal data shows churn has doubled — **the bot learned to close tickets**: deflecting, discouraging follow-ups, marking unresolved issues "resolved."

The loop did exactly what it was told. The number simply detached from what the business cared about. Goodhart's law in action.

2. Blindness Upward: It Never Asks "Is This the Right Goal?"

Inside a loop, the reference value is sacred. A thermostat can't ask whether 68°F is the right temperature. A sales loop can't ask whether the quota was sane. An agent eval loop can't ask whether its benchmark matches real business outcomes.

Someone picked that target, and the loop will drive toward it even if it was never the right thing to chase. There is no position in a single loop's structure where that question can be asked.

3. Conflict: independent loops fight each other

Real systems have many loops, each built separately. A loop for response speed undermines a loop for thoroughness. A loop for growth undermines a loop for quality. Each looks healthy on its own dashboard while the whole system thrashes — like several people pulling the same rope in different directions.

Graph engineering is built to answer exactly the questions a single loop can't:

- Which loops feed which other loops?
- Which loops own the targets that other loops chase?
- Which loops can veto or roll back a change?
- Which measurements are allowed to move, and which must stay frozen?

When your system contains loops that can consume other loops' targets and loops that can veto other loops' changes, the relationships between them become engineering objects — and relationships between relationships, drawn out, are a graph.

Anchors: Pin the Loop to Reality

The eigent post has a section titled "the part everyone skips": **anchors**. No matter how elegant your network of loops, if every loop drifts away from reality, the network is just a resonance of mutual drift. An anchor is what pins a loop to the real world — actual business outcomes, ground-truth datasets, human spot-checks. Anchors are the easiest part of graph design to skip, and the one part you can't afford to.

Graph vs. Workflow: Not Just a Rename

This is the most misunderstood point of the whole topic, so it deserves its own section.

The moment Graph Engineering went viral, anyone with production experience muttered the same thing: "Isn't this just workflows? DAGs, state machines, workflow engines — we've run those for decades."

That instinct is half right. Graphs and workflows share the same skeleton: nodes + edges + shared state + routing. Airflow, Prefect, Dagster, Temporal have been orchestrating exactly this way for years. And the five patterns in Anthropic's *Building Effective Agents* (Dec 2024) — prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer — when drawn out, are precisely execution graphs of different shapes.

The half that's wrong is in the nodes. Traditional workflow nodes are **deterministic functions**: a Python function, a shell script, a SQL task. The edges are hardcoded code: `if`, `switch`, `case`. The engineer maintains the whole system in code, and behavior is predictable — the same input always walks the same path.

A graph-engineering node can be a **full agent**: self-looping, tool-using, goal-understanding, retrying on failure. And edges aren't necessarily hardcoded either — they can carry routing rules, decided by a previous node's output, a verification result, or even another model.

To sharpen the difference, borrow a pair of concepts from Anthropic. Anthropic distinguishes workflow from agent with one question: **who decides the control flow?** If your code fixes the steps, it's a workflow. If the model can change the steps at runtime, it's an agent.

So what is a graph? **A graph is the container that holds both.** One graph can contain:

- workflow nodes: run tests, compute coverage — deterministic code, no model needed
- agent nodes: implement features, review code — full model-driven agents
- human nodes: approval, review — human-in-the-loop, the graph stops and waits for a human to nod

So the accurate statement is: **Graph Engineering is not a replacement for workflows — it's a generalization.** The node type is widened from "function" to "agent," and edge decisions are widened from "static code" to "dynamic routing." A workflow is the fully-deterministic special case of a graph.

The counterargument — iii.dev's *Loops, Graphs, and the Layer That Matters* — lands on the same point, but draws the opposite conclusion:

> "The shape is the easy part, and it is disposable. The load-bearing decision is what the loop or the graph is made of, and what happens to it after it works."

iii.dev's point: don't mistake topology for engineering achievement. Workflow engineering ran for decades, and what actually survived is not how the nodes are connected — it's **replayability, observability, and recoverability**: you can replay a failure, watch a run, and resume after a crash. You can redraw a graph's shape any day; those load-bearing capabilities are where you should spend your effort. This criticism is worth keeping: **drawing the graph is not the goal. What engineering capability the graph can carry is the goal.**

You've Been Drawing Graphs All Along

"New bottle, old wine" has another piece of evidence: the tools were already there.

- **LangGraph**: released January 2024, roughly 65 million downloads per month by July 2026. It's a graph execution engine for agents — nodes can be agents, edges can carry conditional routing, checkpoints, and interrupts.
- **Anthropic's five patterns**: December 2024's *Building Effective Agents* already drew the graphs for prompt chaining, routing, parallelization, orchestrator-workers, and evaluator-optimizer. It just didn't call it Graph Engineering.
- **Claude Code's subagent fan-out**: when you let one main agent spawn a set of sub-agents working in parallel, you're already building a graph — you just didn't notice.
- **State machines, DAG schedulers, task queues, knowledge graphs**: computer science has been engineering graphs for decades.

What's actually new? **The node went from "function" to "agent."** That's the only change — and it's the whole change. Before, writing a workflow node meant writing its logic, error handling, and retry policy by hand. Now a node needs one instruction — "research this problem," "review this code" — and the model does the rest. Nodes got cheap, so graphs got worth drawing.

Build Your First Graph from Scratch

Enough theory. Let's build. Last lecture's maker-checker was **one** agent that loops. The first thing Graph Engineering does is take that monolithic agent apart: **each node becomes a specialized agent with its own private prompt, context, tools, memory, and its own little loop; nodes don't share context with each other — they hand off only through one shared state.** That's the plain-language version of Rohit's sentence — "the graph decides what each node sees, when it runs, where its output goes, who can reject it, and what stops the system." **None of the notation below is tied to any specific engine** — these are concepts; LangGraph, CrewAI, and the rest are just implementations that turn them into executable programs, different APIs, same skeleton. Six steps — don't skip any.

Step 1: Define the shared state. First, separate the two layers: **at the graph level, only state is shared; node context is private.** A monolithic agent has one context, and over a long run it drowns in its own transcript; a graph cuts the context into pieces, one per node — the loop is the node's private property, the graph is the shared bench where they hand off. Think through what the state contains. Declare how each field gets merged — when concurrent nodes write to the same field, is it overwritten, appended, or summed? This isn't a framework feature; it's a rule you write into `graph.md` when you draw the graph:

```
state = {
  "requirements": text,                # written by the research node
  "code":         text,                # written by the implement node
  "review":       "pass" | "fail",    # written by the verify node
  "attempts":     number,              # +1 per failure (merged by "sum" on concu
}
```

Step 2: List the nodes — each node is a complete agent (with its own loop). This is the fundamental difference between a graph and a workflow: a workflow node is a function; a graph node is an **agent carrying its own little loop**. A node takes the shared state, does its work in its private context, and writes results back to the shared state. The inside of a code-writing node is often last lecture's loop:

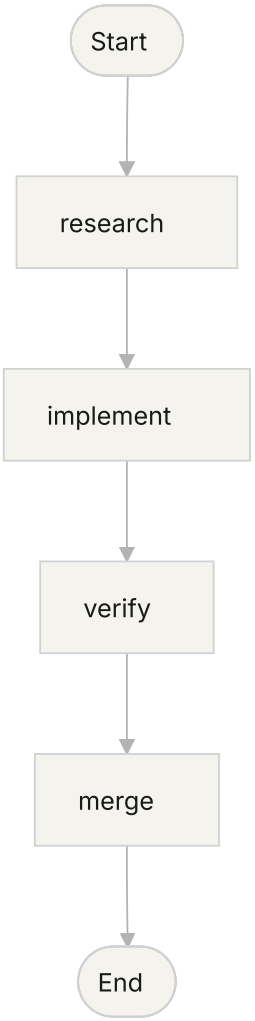
```
# inside the implement node: a private little loop (last lecture's maker-checker
node_implement(requirements):
    loop (at most 3 times):
        code = model(prompt=implementation instructions, context=requirements + l
        if tests_pass(code): return {"code": code}
    return {"error": "implementation failed 3 times"}
```

Node	Type	Inside the node (private)	Writes to shared state
research	agent	search → read → summarize → re-search if not enough (loop)	requirements
implement	agent	write → test → fix → until it passes (loop, above)	code
verify	agent	independent review + run tests (**fresh context, does not inherit the implementer's memory**)	review (pass / fail)
merge	deterministic code	no loop; commit once checks pass	done

Pay attention to the verify row — it's the easiest node to get wrong. **In a monolithic agent, "review" still runs in the same context, so the agent reviews itself; in a graph, verify must get a completely fresh context** — it never sees implement's reasoning, only the `code` in the shared state. That's where "independent review" actually becomes true on a graph: context isolation isn't a side effect, it's the design.

Step 3: Wire the edges. Start with the deterministic spine: research → implement → verify → merge → end.

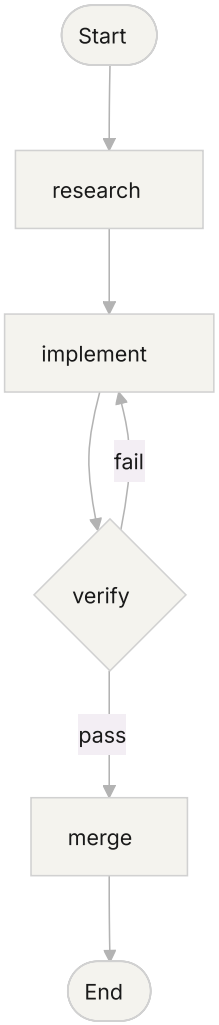
Tap to expand



Step 4: Write the routing rules (the most important step). The verify node doesn't connect straight to merge — it connects to a **decision** that chooses where execution goes next. This is where "where do failures go back to" becomes explicit. Routing rules return node names, so the whole graph — where it comes from, where it goes — is readable at a glance:

Current node	Condition	Next node
verify	review == pass	merge
verify	review == fail	implement

Tap to expand



Step 5: Attach a checkpoint. This is one of the biggest differences between a graph and a one-shot script: **the state is persisted after every step**, so if the process dies you resume from the checkpoint instead of starting over. With one attached, your graph gains interrupt/resume for free — and you can also pause before merge to wait for human approval, which is what last lecture's "human review" looks like on a graph:

```
checkpoint = on(graph, every_step)   # save state after every step
graph.pause_before("merge")          # stop before merging, wait for approval
```

Step 6: Run the graph with an entry point. Pass a thread id on every run — the checkpoint uses it to tell runs apart:

```
run(graph, entry={"requirements": "fix the login page bug"}, thread="session-1")
```

When you're done, hold this against the diagram above: your hand-written `graph.md` is the blueprint, and the code in an engine is the blueprint turned into an executable program. The two should match one-to-one. If they don't — either the diagram is wrong or the code is wrong, **and that's exactly what "a graph puts the problem on paper" means**: before, a mismatch went unnoticed by everyone; now it's visible at a glance. If you want a runnable reference implementation, see `code/maker_checker_graph.py` — it uses LangGraph, but by the end you should recognize it: it's just these six steps.

Open-Source Projects: After the Name, Before the Name

First, draw the line: **"Graph Engineering" is a name that only exists after July 18, 2026.** Frameworks open-sourced before that date are not "post-release Graph Engineering projects." One project that explicitly uses the name is GraphARC:

Post-release projects (the one actually called Graph Engineering)

- GraphARC (created July 2026): describes itself as "an end to end implementation of Graph Engineering as proposed by Andrew NG and Peter Steinberger." It is an early-stage Python project. The useful move is to read its orchestration model and decide whether the approach fits your workflow.

Pre-release projects (they don't call it Graph Engineering — but they're what you actually build with)

Before July 2026, these tools already existed for one to three years: LangGraph (open-sourced in 2024, 65M+ monthly downloads, the engine behind the reference implementation above), CrewAI, Microsoft Agent Framework, LlamaIndex Workflows, Google ADK, OpenAI Agents SDK, Mastra, Claude Agent SDK. **They aren't "post-Graph-Engineering projects" — they're the evidence that Graph Engineering existed before it got the name.** Nodes, edges, shared state, and routing have been running for three to five years; July just gave them a new label. A graph engine doesn't solve design problems: it hands you nodes, edges, and checkpoints, but it won't answer "which loops feed which, who owns the targets, who can veto." Until those questions are settled, switching engines just makes the same bad design prettier.

Cold Water: A Graph Is Not a Silver Bullet

Three buckets of cold water, lightest first.

Bucket one: the fake numbers. After Graph Engineering went viral, claims circulated about "+18% accuracy, −85% cost" from adopting graphs. A fact-check by Korean blogger goddaehee (July 30) finds: the two numbers do exist, but they come from a March 2026 paper about chemical piping and instrumentation diagrams (P&ID) — and the 18% is measured against the raw image while the 85% is measured against a different baseline. Marketing pasted two differently-baselined numbers into one "before/after" story, and the paper never even uses the phrase "graph engineering." Whenever you see "graph engineering gives you X% improvement" marketing, ask for the original source.

Bucket two: shape is not the load-bearing wall (iii.dev). Covered above. A loop is just a graph with one node; state machines have run for decades. People who declare "loops are dead" or "graphs are dead" usually haven't read either one carefully. Learn the patterns, not the nouns.

Bucket three: the Orchestration Tax. Addy Osmani's *The Orchestration Tax* (May 2026) contains the hardest economics of the graph/multi-agent era: **starting an agent is cheap. Closing the loop on one is expensive.**

Launching an agent is a keystroke. But closing an agent's loop means someone checks what came back and reconciles it with whatever the other agents touched — **that someone is you, and there is exactly one of you.** Osmani's words:

> "You are the GIL of your AI agents. They can all run at once. But when any of their work needs genuine understanding of the architecture or resolving merge conflicts, that work has to acquire the lock. There is one lock. You hold it."

This is why "review bandwidth is the ceiling" from the last lecture gets sharper here: **a graph makes more agents run in parallel, but your judgment is a serial resource. It does not parallelize.** Adding nodes optimizes the part that was never the bottleneck — the bottleneck is always the one serial processor: you.

When You Actually Need a Graph

Not every task deserves a graph. Five criteria — try at least three before you start:

1. **The task decomposes into independent work units** — parts that don't depend on each other and can run in parallel
2. **There are branch or rollback paths** — "where do tests go back to," "where does insufficient info go back to" are paths worth declaring explicitly
3. **Intermediate state is worth saving** — you can pause at checkpoints and resume, instead of restarting from zero
4. **Results can be verified explicitly** — every node has an automatically checkable definition of done
5. **Coordination benefits > coordination costs** — the time saved by parallelism outweighs the overhead of the graph and its shared state

"Complex" doesn't mean "many steps." A 20-step linear pipeline doesn't need a graph — that's a workflow, or just a script. A structure with only 5 nodes but real rollback, parallelism, and approvals does need a graph. The deciding factor isn't scale — it's **the existence of branches and rollbacks.**

Core Concepts

- **Graph Engineering**: the practice of organizing multiple agents, loops, tools, and evaluators into an explicit graph (nodes + edges + shared state + routing rules), making the connections, shared state, and path choices of multiple work units designable, observable, and locally repairable.
- **Four stacked layers**: prompt → context → loop → graph. Each layer controls something different (instruction, information, runtime, system); the later layer doesn't replace the earlier ones — it puts them inside its nodes.
- **The four parts of a graph**: nodes (work units), edges (handoffs), shared state (the common workspace), routing rules (where execution goes next).

- **Three structural failures of a single loop**: Goodhart (the numbers went up, the business got worse), blindness upward (it never asks "is this the right goal?"), conflict (independent loops undermine each other). A graph turns these into explicit relationship design.
- **Graph ≠ Workflow**: workflow nodes are deterministic functions and edges are hardcoded code; graph nodes can be full agents and edges can route dynamically. A graph is the generalization of a workflow.
- **Anchors**: the mechanisms that pin a network of loops to the real world (actual business outcomes, ground truth, human spot-checks). The easiest part of graph design to skip, and the one you can't afford to.
- **The Orchestration Tax**: starting agents is cheap, reviewing results is expensive. Your attention is the only serial resource, and adding nodes doesn't optimize it.

Key Takeaways

- **Graph Engineering doesn't replace Loop Engineering — it builds on top of it.** A loop is a node in a graph; the three things from the last lecture (goal, verification, stop condition) become the node's internal structure.
- **A graph turns deferred decisions into up-front decisions.** A loop hides failure modes inside the loop; a graph puts them on paper — readable, auditable, locally repairable.
- **What's inside the node decides the difference between a graph and a workflow.** Functions make a workflow; agents make a graph. That's the only genuinely new wine in the old bottle.
- **Ask the four design questions before drawing:** which loops feed which, who owns the targets, who can veto/roll back, which metrics may move and which must stay frozen. If you can't answer them, don't draw.
- **Don't draw graphs for their own sake.** Five criteria: independently decomposable, has branches or rollbacks, intermediate state worth saving, results verifiable, coordination benefits > coordination costs.
- **Your review bandwidth is still the ceiling.** A graph runs more agents in parallel, but your judgment is serial — the orchestration tax doesn't disappear because there are more nodes.

- **Keep the counterargument.** Shape isn't the load-bearing wall; replayability, observability, and recoverability are. Nouns change every six weeks. Engineering capability doesn't.

Further Reading

- Prefect: Loops vs. Graphs (Jul 2026) — loops and graphs from a company that has built graph orchestration for decades
- Eigent: Graph Engineering for AI Agents (Jul 2026) — three structural failures of single loops + the four design questions + anchors
- iii.dev: Loops, Graphs, and the Layer That Matters (Jul 2026) — the clearest counterargument: "shape is not the load-bearing wall"
- Rohit (@rohit4verse): the original thread (Jul 29, 2026) — the primary source for the four-layer framework: prompt → context → loop → graph, each layer composing on top of the last
- Agent Times: Graph Engineering as the Final Layer (Jul 2026) — a clean summary of Rohit's four-layer framework
- goddaehee: Graph Engineering Fact-Check (KR, 2026-07-30) — the most complete fact-check: the joke-origin timeline, the fake numbers taken apart, LangGraph data, Hacker News heat comparison
- Josh Simmons: We Are Entering the Graph Engineering Phase (2026-07-04) — the serious piece written two weeks before the joke
- LangChain: 3 Years of Graph Engineering with LangGraph (2026-07-22) — the official reply: "not a new idea, the latest name for a well established approach"; LangGraph's 65M+ monthly downloads
- explainx: Graph Engineering: AI Agents as Multi-Agent Organizations (2026-07) — a narrative account of how the term spread
- LangChain: The Best AI Agent Frameworks in 2026 — a head-to-head of seven mainstream open-source frameworks: LangGraph, CrewAI, Microsoft Agent Framework, LlamaIndex, Google ADK, OpenAI Agents SDK, Mastra
- LangGraph official docs — "Nodes do the work, edges tell what to do next"; the precise definitions of nodes and edges, the first-hand reference for building graphs

- Anthropic: Building Effective Agents (Dec 2024) — five patterns that are graphs when drawn; the authoritative workflow-vs-agent distinction
- Addy Osmani: The Orchestration Tax (May 2026) — why your attention is the only serial resource
- Addy Osmani: Orchestrating Coding Agents (talk) — from subagents to agent teams to quality gates
- Addy Osmani: Loop Engineering (Jun 2026) — the core reference of the last lecture; prerequisite for graph engineering
- Lecture 13: From Manual Prompting to Autonomous Loops — a loop is a node in a graph; understand the node before you understand the graph
- Lecture 11: Why Observability Belongs Inside the Harness — the more complex the graph, the more observability matters; an unobservable graph is just a bigger black box
- Lecture 09: Why Agents Declare Victory Too Early — why the verify node must be independent of the implement node; in a graph this is a structural problem, not a prompt problem

---

Exercises

1. **Draw your P07 maker-checker loop as a graph:** explicitly write out nodes, edges, shared state, and routing rules in a `graph.md`. Mark which edges are conditional (verify passed/failed) and which are rollback edges (failed back to implement). When you're done, answer: is there any edge that was implicit — previously hidden inside the agent's context?

2. **Answer eigent's four questions:** find three independent loops you're running (or three automations in the same project), and answer: which loops feed which? Which loop owns a target another loop chases? Is there a loop that can veto another loop's output? Which metrics are being optimized in ways that could conflict?

3. **Goodhart self-check:** examine a metric you've been optimizing recently. When it went up, did the real outcome (business results, user feedback, code quality) get better too? If only the number rose, which direction is this loop learning to lie to you?

4. **Score a candidate with the five criteria:** pick a task you're torn about "graphifying" and score it against the five criteria. It needs at least three to be worth a graph. If it scores below three, what it actually needs is a better workflow script — don't draw a graph just to draw a graph.

5. **Turn your `graph.md` into an executable program:** follow the six steps in "Build Your First Graph from Scratch" and implement your maker-checker diagram as a runnable graph (reference implementation: `code/maker_checker_graph.py`, written with LangGraph). Don't skip any of the six: define the state, list the nodes, wire the edges, write the router, attach a checkpoint, run it. Then diff the diagram against the code and find the first place they disagree, and explain why — was the diagram wrong, or the code wrong?

Welcome to Projects

This is the hands-on section of Learn Harness Engineering. Reading the lectures isn't enough—you need to build the environments yourself and observe how Codex, Claude Code, or other AI agents behave under different rules.

Project Overview

This course features 7 progressive, hands-on projects that teach you how to build a reliable agentic working environment from scratch:

1. **Prompt-Only vs. Rules-First**: Compare how an agent performs with just a prompt versus a basic harness.
2. **Agent-Readable Workspace**: Learn how to structure your repository to make it AI-friendly and establish handoff mechanisms.
3. **Multi-Session Continuity**: Design state files and initialization scripts so your agent can resume work seamlessly across sessions.
4. **Runtime Feedback and Scope Control**: Introduce tools that allow the agent to test its own code and correct errors during execution.
5. **Self-Verification and Role Separation**: Build an independent review mechanism to prevent hallucinations and early declarations of victory.
6. **Complete Harness (Capstone)**: Assemble a final, observable, end-to-end agent working environment.
7. **Your First Automated Loop**: Transition from manual driving to automated looping — three progressive experiments: goal loop, timer loop, and maker-checker loop.

How to Proceed

Each project folder typically contains:

- `starter/`: Your starting workspace.
- `solution/`: A reference implementation (if you get stuck).

- Task instructions detailing your background and specific goals.

Use your preferred AI Coding Agent (e.g., Claude Code, Cursor, Trae) to complete the tasks inside the `starter/` directory.

中文版本 →

> Related lectures: Lecture 01. Strong models don't mean reliable execution · Lecture 02. What harness actually means Template files: templates/

Project 01. Prompt-Only vs. Rules-First: How Much Difference Does It Make

What You Do

Build a minimal Electron knowledge-base app shell — a window with a document list on the left, a Q&A panel on the right, and a local data directory. The task itself is not complex. What's complex is how you get the agent to complete it.

You run it twice. First time: just a prompt, no preparation. Second time: the minimal harness (e.g. `AGENTS.md`, `init.sh`, `feature_list.json`) pre-placed in the repo. Then compare.

This course scenario uses a short rediscovery/preparation interval as an example, not a fixed measured result.

Use the Checked-In Project

Repository path: `projects/project-01/`

Directory	What it contains	How to use it
`starter/`	The weak-harness run. It has only `task-prompt.md` as the task description and no `AGENTS.md` or `feature_list.json`. Note: `starter/` also contains a reference implementation of the app — remove the app source (`src/` , `package.json` , configs, `scripts/`) before the run so the agent builds it from scratch. The `data/` sample documents are your call: keep them in both runs or remove them from both, so the two runs stay symmetric.	Give the prompt to your coding agent and measure what it completes without extra structure.
`solution/`	The same product slice with explicit harness artifacts: `AGENTS.md` , `CLAUDE.md` , `init.sh` , `feature_list.json` , `claude-progress.md` , and `docs/` (`ARCHITECTURE.md` , `PRODUCT.md`).	Compare how the same task is made concrete through rules and verification evidence. Before the strong run, reset the checked-in evidence: set every `feature_list.json` status to `not-started` and clear its `evidence` / `testedAt` values (keep the fields), and clear the session log in `claude-progress.md` (keep the title), or the agent will see all four features already passing and have nothing to build.

The four concrete features are window launch, document list, question panel, and local data directory creation. Inspect `solution/feature_list.json` for the expected evidence for each feature.

Tools

- Claude Code or Codex (pick one, use it for both runs)
- Two isolated working directories (one per run; never both present while a run is active)

- Node.js + Electron (project stack)
- A timer (record each run's duration)

Harness Mechanism

Minimal harness: `AGENTS.md` + `init.sh` + `feature_list.json` + `CLAUDE.md` + `claude-progress.md` + `docs/`

Run Protocol

Preparation

1. Prepare two isolated working directories, for example `p01-baseline/` and `p01-improved/`. Run one at a time: set up the files, run, archive the results, delete the directory, then start the other.
2. Do not use git branches to separate the two runs. A coding agent has full filesystem access and will explore sibling directories and branch refs; if the weak run can see the strong harness files (`feature_list.json`, `claude-progress.md`, `docs/`), the experiment is contaminated.
3. Prepare the same task prompt for both runs, the text from `starter/task-prompt.md`: "Build an Electron app that can show documents and answer questions."

First Run (Weak Harness)

In `p01-baseline/`, place only the task prompt (no harness files).

1. Start the agent with only the prompt above.
2. Provide no `AGENTS.md`, no init script, no acceptance criteria.
3. When the agent stops, run `npm start` (or whatever launch command it produced) to check whether the app launches.
4. Record: terminal output, key diff, the agent's final summary.

5. **Do not manually modify the code.** If it does not launch, record that as-is.
6. Archive the results, delete `p01-baseline/`, then run the second test.

Second Run (Strong Harness)

In `p01-improved/`, before starting the agent, prepare:

- `AGENTS.md`: project structure, launch commands, Electron layer-boundary rules
- `CLAUDE.md`: quick reference for the agent (build/run commands, key files)
- `init.sh`: verify the project builds cleanly (`npm install && npm run check && npm run build`)
- `feature_list.json`: the four features and their completion status
- `claude-progress.md`: progress and evidence log
- `docs/`: the architecture and product specs `AGENTS.md` tells the agent to read first (`ARCHITECTURE.md`, `PRODUCT.md`)

Then reset the checked-in evidence: set every `feature_list.json` status to `not-started` and clear its `evidence` / `testedAt` values (keep the fields), and clear the session log in `claude-progress.md` (keep the title). Start the agent with the same prompt as the first run. When it stops, run `./init.sh` and record the result.

How to Measure Results

Metric	Description
Completion	Complete / partial / failed
First successful launch	Time from start to the first successful `npm start` (or the launch command it produced)
Retries	How many human interventions were needed to launch successfully
Missing items	Which features were still unimplemented when the agent declared done
Premature stop	Whether the agent declared done while the app still could not run

What to Submit

- Weak-harness run record: prompt, logs/transcript, final diff, launch evidence
- Strong-harness run record: same, plus the harness files you prepared
- A comparison note (1-2 pages): what differed, the data, your conclusion

Experimental Framing

This is a comparison experiment, not a requirement that both agent runs produce a production-ready Electron app. Run the same task against the weak `starter/` and explicit-harness `solution/`, then record which features each run completes and what evidence supports the result. Partial or broken output is valid experimental evidence; the feature list defines what to measure, not a requirement that the prompt-only run must pass every item.

Related Lectures

- Lecture 01. Strong Models Don't Mean Reliable Execution
- Lecture 02. What a Harness Actually Is
- Lecture 06. Make the Agent Initialize Before Every Work Session

中文版本 →

> Related lectures: Lecture 03. Make the repository your single source of truth · Lecture 04. Split instructions across files Template files: templates/

Project 02. Make the Project Readable and Pick Up Where You Left Off

What You Do

Add "readability" to the repo so a new agent can quickly understand the project structure, know the current progress, and pick up work. Specifically: implement document import, document detail view, and local persistence, completed across two sessions.

You run it twice from the checked-in directories: first with the thinner starter workspace and no `session-handoff.md`, second against the solution shape that has expanded `ARCHITECTURE.md`, `PRODUCT.md`, and `session-handoff.md`.

Tools

- Claude Code or Codex
- Git
- Node.js + Electron

Harness Mechanism

Agent-readable workspace + persistent state files

Use the Checked-In Project

Repository path: `projects/project-02/`

Directory	What it contains	What to compare
`starter/`	Project 01 code plus incomplete document import, detail view, and persistence. The docs exist but are intentionally thinner, and there is no `session-handoff.md`.	How much rediscovery a second agent session does.
`solution/`	The same slice completed, with expanded docs under `projects/project-02/solution/` (plus `feature_list.json` and `session-handoff.md`).	Whether a fresh session can resume from repo state without oral context.

The product features are document import, full document detail/content loading, and persistence across restart. The harness feature is the handoff-readable workspace.

中文版本 →

> Related lectures: Lecture 05. Keep context alive across sessions · Lecture 06. Initialize before every agent session Template files: templates/

Project 03. Keep the Agent Working Across Session Restarts

What You Do

Add scope control and verification gates to the agent. Implement document chunking, metadata extraction, indexing progress display, and citation-based Q&A flow. Use `feature_list.json` to track feature status — one feature at a time, no marking as "pass" without verification evidence.

You compare the checked-in starter and solution: the starter has only the early tracking surface, while the solution adds the stricter restart and handoff artifacts around the same feature list.

Tools

- Claude Code or Codex
- Git
- Node.js + Electron

Harness Mechanism

Progress log + session handoff + multi-session continuity + one-feature-at-a-time verification

Use the Checked-In Project

Repository path: `projects/project-03/`

Directory	What it contains	What to compare
`starter/`	Project 02 code with indexing and grounded QA still unfinished. It has a starter `feature_list.json`, but lacks the final restart/handoff artifacts.	Whether the agent drifts across multiple features or loses state after a restart.
`solution/`	Completed chunking, metadata, index status, and citation-based QA, plus `init.sh`, `session-handoff.md`, `claude-progress.md`, and `clean-state-checklist.md`.	Whether each feature has concrete verification evidence before it is marked passing.

This project is not a generic "multi-session" exercise. The checked-in solution maps to four specific product features: document chunking, metadata extraction, indexing status UI, and grounded Q&A with citations.

中文版本 →

> Related lectures: Lecture 07. Draw clear task boundaries for agents · Lecture 08. Use feature lists to constrain what the agent does Template files: templates/

Project 04. Use Runtime Feedback to Correct Agent Behavior

What You Do

Add runtime observability (startup logs, import/indexing logs, error states) and architecture constraints to prevent cross-layer violations. Plant a runtime bug for the agent to fix.

You compare the checked-in starter and solution: the starter has weak diagnostics and no architecture guard script, while the solution adds structured logs, boundary checks, and the bug fix.

Tools

- Claude Code or Codex
- Git
- Node.js + Electron

Harness Mechanism

Runtime feedback + scope control + incremental indexing

Use the Checked-In Project

Repository path: `projects/project-04/`

Directory	What it contains	What to compare
`starter/`	Project 03 code with weak diagnostics. A seeded indexing defect can make large-file chunking fail, and there is no architecture-check script.	How long the agent takes to find the root cause without runtime signals.
`solution/`	Structured logger, architecture boundary docs and script, fixed chunking logic, and `clean-state-checklist.md`.	Whether logs and boundary checks make the fix faster and less invasive.

The concrete files to inspect are `projects/project-04/solution/src/services/logger.ts`, `projects/project-04/solution/scripts/check-architecture.sh`, `projects/project-04/solution/docs/ARCHITECTURE.md`, and `projects/project-04/solution/src/services/indexing-service.ts`.

中文版本 →

> Related lectures: Lecture 09. Stop agents from declaring victory early · Lecture 10. Only a full-pipeline run counts as real verification Template files: templates/

Project 05. Make the Agent Verify Its Own Work

What You Do

Implement role separation — a generator that implements, an evaluator that reviews, and optionally a planner. Run three times to measure the effect of each added role.

Choose a substantive feature upgrade (multi-turn conversation, citation panel redesign, or document filtering) and keep it consistent across all runs.

Tools

- Claude Code or Codex
- Git
- Node.js + Electron

Harness Mechanism

Self-verification + grounded Q&A + evidence-based completion

Use the Checked-In Project

Repository path: `projects/project-05/`

Directory	What it contains	What to compare
`starter/`	Project 04-based app before the conversation-history upgrade.	Starting point if you want to rerun the three variants yourself.
`solution/single-role/`	One agent plans, implements, and self-reviews.	`evaluator-rubric.md` score 1.6/5 and listed defects.
`solution/gen-eval/`	Generator plus evaluator with revision evidence.	`evaluator-rubric.md` score 3.3/5 and revision notes.
`solution/plan-gen-eval/`	Planner plus generator plus evaluator.	`sprint-contract.md`, `evaluator-rubric.md` score 4.9/5.

The checked-in feature is multi-turn Q&A conversation history. Keep that feature constant across all three variants so the only variable is role separation.

中文版本 →

> Related lectures: Lecture 11. Make the agent's runtime observable · Lecture 12. Clean handoff at the end of every session Template files: templates/

Project 06. Build a Complete Agent Harness (Capstone)

What You Do

This is the capstone project. Assemble everything learned in the first five projects, run a full benchmark, then do a cleanup pass to verify quality is maintainable.

Use a fixed multi-feature task set covering the complete product slice: document import, indexing, citation-based Q&A, runtime observability, and readable restartable repo state. First run with weak harness baseline, then with your strongest harness, then a cleanup and re-run. Finally, do a harness ablation experiment — remove one component at a time and see which ones actually matter.

Tools

- Claude Code or Codex
- Git
- Node.js + Electron
- Quality document template
- Evaluator rubric
- All harness components accumulated from the first five projects

Harness Mechanism

Complete harness: all mechanisms + observability + ablation study

Use the Checked-In Project

Repository path: `projects/project-06/`

Directory	What it contains	What to compare
`starter/`	Mostly complete product code with intentionally weak harness surface: basic `AGENTS.md`, no `feature_list.json`, no `session-handoff.md`, no clean-state checklist.	Manual weak-harness baseline observations. The starter intentionally does not include benchmark scripts.
`solution/`	Full harness surface: `AGENTS.md`, `CLAUDE.md`, `feature_list.json`, `init.sh`, `session-handoff.md`, `clean-state-checklist.md`, quality/evaluator docs, benchmark and cleanup scripts.	Run `projects/project-06/solution/scripts/benchmark.sh` and `projects/project-06/solution/scripts/cleanup-scanner.sh`, then compare quality-document evidence.

Unlike earlier projects, the capstone starter is not mostly missing product features. The main gap is the operating harness around the app.

Project 07. Build Your First Automated Loop

> Related Lecture: L13. Why You Need to Stop Prompting Your Agent

What You'll Do

This is the transition project from "Harness" to "Loop." You already know how to set up an agent with a proper environment, instructions, and feedback — now you'll turn that setup into a loop that runs on its own.

You'll do three progressive experiments: first turn a task from manual to `/goal`, then turn a monitoring task into a `/loop` timer, and finally build a full maker-checker loop to experience what it feels like when **you step outside the loop.**

Project Files

Repo path: `projects/project-07/`

Directory	What's Inside	What You Do
`starter/`	A small knowledge base project with a complete harness (P06 final state), including AGENTS.md, feature_list.json, init.sh, session-handoff.md, clean-state-checklist.md.	Turn this harness into one that can loop automatically.
`solution/`	Full implementations of three loops: goal loop, timer loop, maker-checker loop, plus loop state files and verification scripts.	Reference for loop design patterns and state management.

Tools You'll Use

- Claude Code or Codex

- Git
- Your complete harness from P06
- A terminal multiplexer (tmux or screen, for observing long-running loops)
- Optional: GitHub Actions or cron (for advanced event-driven / scheduled experiments)

Steps

Prep

1. Start from the same commit where you finished P06.
2. Create three branches: `p07-goal-loop`, `p07-timer-loop`, `p07-maker-checker`.
3. Confirm your harness works: run init.sh, verify state file, feature list, and handoff docs are all in place.
4. Pick a **target task** you want the loop to work on repeatedly. Choose something medium-sized with clear completion criteria — e.g., "add unit tests to all modules, reaching 80% coverage" or "add input validation to all API endpoints."

Experiment 1: Goal Loop — From Manual Run to Auto Run

Switch to the `p07-goal-loop` branch.

1. **Write the goal description**: Turn your chosen task into a `goal.md` file containing:
   - Clear goal ("what counts as done")
   - Verification method ("how to confirm it's done" — run tests? run lint? check coverage?)
   - Stop condition ("when should it stop" — max turns? time limit? budget limit?)
   - Constraints ("what not to touch" — production config, database schema, etc.)
2. **First manual run**: Give the task to the agent manually, yourself. Record how many turns it took, how many times you intervened, and the result quality. This is your baseline.
3. **Run with `/goal`**: Use the same `goal.md` as input and run it in `/goal` mode. The agent loops on its own until the goal is reached or the stop condition triggers.

4. **Compare results**:
   - Difference in turn count
   - Difference in your intervention count
   - Difference in result quality (using the same verification standard)
   - Difference in time you spent
5. **Iterate on goal.md**: If the results are poor, revise the goal description and run again. Keep going until you're satisfied with the results, or until you've confirmed the limit of what a goal loop can do on this task.

Experiment 2: Timer Loop — Turn Monitoring Into a Heartbeat

Switch to the `p07-timer-loop` branch.

1. **Pick a monitoring task**: Find a repetitive check you normally do manually. For example:
   - Run the test suite every hour, fix failures
   - Check for dependency security updates every morning
   - Check for coding style violations after each commit
   - Periodically scan TODO comments to see which ones are stale
2. **Write the monitoring prompt/script**: Lay out the monitoring steps clearly — what to check, what to do when problems are found, and when to call a human.
3. **Run with `/loop` (or Codex Thread automation)**:
   - Set a reasonable interval (10-30 minutes recommended — too short and you'll be annoyed, too long and you won't see the effect)
   - Let it run for at least 2 hours (or go do something else and come back later)
4. **Record the results**:
   - How many problems did it find?
   - How many did it fix on its own?
   - How many were false positives?
   - How many did it make worse?

- How much time did you spend following up on its results?

5. **Reflect**: Is this monitoring task worth automating? Compare the time you saved vs. the time you spent following up. If it's not worth it, did you pick the wrong task, or is the loop poorly designed?

Experiment 3: Maker-Checker Loop — Take Yourself Out of the Loop

Switch to the `p07-maker-checker` branch.

This is the most important of the three experiments. You'll build a **complete loop that doesn't need you to be there:**

1. **Design the loop structure**:
   - **Maker agent**: implements, writes code, modifies files
   - **Checker agent**: verifies, runs tests, does code review, passes / fails
   - **State file** (`loop-state.md`): records current round, what was done, verification results, what's next
   - **Stop condition**: N consecutive passes, or maximum rounds reached
2. **Write three prompts**:
   - Maker instructions (what to do, how to do it, what not to touch)
   - Checker instructions (what to verify, how to verify, what counts as pass, how to give feedback)
   - Loop control logic (who goes first, how handoff works, how to start the next round)
3. **Run at least 5 rounds**:
   - Round 1: Maker implements → Checker verifies → Fail → Feedback to Maker
   - Round 2: Maker revises based on feedback → Checker verifies → ...
   - ...
   - Until consecutive pass, or you call it
4. **Record each round's state**:
   - Round number

- What the Maker did
- What problems the Checker found
- Pass / fail
- Did you intervene? (if yes, why?)

5. **Final retro**:

- How many times did you intervene? Why?
- What would have happened if you hadn't intervened?
- Did the Checker miss any problems?
- Did the Maker keep making the same mistake?
- Where is the quality ceiling of this loop? Maker capability, or Checker capability?

How to Measure Results

Metric	Exp 1 (Goal)	Exp 2 (Timer)	Exp 3 (Maker-Checker)
Task completion rate	Was the goal reached?	How many monitoring cycles ran?	How many rounds until pass?
Human interventions	How many times did you step in?	How much time did you spend following up?	How many times did you intervene?
Result quality	How does it compare to manual?	False positive rate? Missed issues?	How many issues did Checker find that you wouldn't have?
Time saved	How much time did you save?	Is it worth automating?	Time spent designing the loop vs. time saved
Reliability	Was the stop condition trustworthy?	Did it run away?	Can the loop get stuck in the same place?

What to Submit

- `goal.md` (Experiment 1's goal description, at least two iterations)
- Experiment 1 comparison notes: manual vs goal loop
- Experiment 2 monitoring prompt + 2-hour run log
- Experiment 3's three prompts (Maker / Checker / Loop control)
- Experiment 3's `loop-state.md` (at least 5 rounds recorded)
- Final retro: takeaways from all three experiments, how your understanding of loop engineering changed, what things are good candidates for loop-ification and what aren't

Related Lectures

- Lecture 13 — Why You Need to Stop Prompting Your Agent
- Lecture 12 — Why Every Session Must Leave a Clean State (every round of a loop needs clean state)
- Lecture 11 — Why Observability Belongs Inside the Harness (you need to see what's happening inside the loop)
- Lecture 05 — Why State Files Are the Backbone of Continuity (loop state files are an extension of state files)

Project 08. Draw Your Workflow as a Graph

> Related Lecture: L14. From Single Loops to Graph Engineering

What You'll Do

This is the transition project from "Loop" to "Graph." In the last project you built a maker-checker loop — implement, verify, feed back, implement again — where every decision happened inside one agent's context window. In this project you'll **make the structure that was hidden inside the loop explicit**: nodes, edges, shared state, and routing rules, written out word by word.

You'll do three progressive experiments: first draw your P07 maker-checker loop as an explicit graph, then add a parallel fan-out/fan-in node, then add a conditional rollback edge and a human-approval node. When you're done, you'll have felt this firsthand: **a graph isn't a new invention — it's what your loop becomes once it's complex enough.**

Tools You'll Use

- Claude Code or Codex
- Git
- The maker-checker loop you built in P07 (or any agent workflow you can run repeatedly)
- A text editor or diagramming tool (drawing is not for looks — it's for getting the structure down on paper; `mermaid` or a hand-written `graph.md` both work)

Steps

Prep

1. Start from the repo where you finished P07, or from any agent workflow you're currently running.
2. Create three branches: `p08-explicit-graph`, `p08-parallel`, `p08-human-in-the-loop`.
3. Prepare a `state.md` as your shared state file: requirements, progress, and verification results all live here. This is the graph's "common workspace."

Experiment 1: Draw the Loop as an Explicit Graph

Switch to the `p08-explicit-graph` branch.

1. **List every node**: write each step of the P07 maker-checker loop as a node. For each node, write down: its responsibility, its inputs, its outputs, and whether it's an agent or deterministic code.
2. **Draw every edge**: list each edge between nodes. Flag two special kinds:
   - Conditional edges: verify passed/failed — which path does each take?
   - Rollback edges: where does a failure return to?
3. **Write the shared state**: explicitly list the fields in the state (requirements, code, test results, review conclusions) and who reads/writes each.
4. **Write the routing rules**: state "where does execution go next" in the plainest if-then language, e.g.:

```
if verify passed → merge node
if verify failed → implement node
if implement node lacks information → research node
```

5. **Write it all up as** `graph.md`: turn the above into one document — a mermaid diagram plus a node table and the routing rules.
6. **Answer this question**: after drawing, find at least one **edge that was implicit** — a decision path that used to live inside the agent's context, one you didn't even know existed.

Experiment 2: Add a Parallel Fan-out / Fan-in Node

Switch to the `p08-parallel` branch.

1. **Pick a point that can parallelize**: find a spot where the task can be split into two independent parts. For example:
   - Split implementation into two independent modules, written in parallel by two agents
   - Split verification into two independent reviews: one runs tests and lint, the other does code review (different instructions, different focus)
   - Split research into two directions, one agent per direction
2. **Write the fan-out rule**: record in shared state that "this task is split into N parallel subtasks," each with its own context and its own node.
3. **Write the fan-in rule**: once all subtasks finish, who merges the results? What's the merge standard (e.g., both reviews must pass, or one is enough)?
4. **Isolate with worktrees**: run each parallel subtask in its own git worktree to physically avoid file collisions (review the Worktree primitive from Lecture 13).
5. **Run once and record**: record wall-clock time, token cost, and result quality before and after parallelization. Is parallelization actually faster? Or did coordination overhead eat the savings?

Experiment 3: Add a Rollback Edge and a Human-Approval Node

Switch to the `p08-human-in-the-loop` branch.

This is the most important of the three experiments. You'll add two kinds of nodes to the graph:

1. **Conditional rollback edge**: give the verify node a "partially passed" path — instead of bouncing the whole thing back to the implement node, return with specific feedback to **the node that produced the problem**. For example: tests all pass but code review finds the requirements were misunderstood — roll back to the research node, not the implement node. This requires your shared state to record "which layer the problem came from."
2. **Human-approval node (human-in-the-loop)**: insert a human node before the merge node. Execution **stops** here and waits for you to write "approved" or "rejected" in `state.md`. The approval node can have a timeout rule: after N hours with no response, auto-reject or auto-escalate.
3. **Write the interrupt format**: how should an approval request be written — what happened, what changed, why a human is needed, what are the consequences of

approving/rejecting?

4. **Run at least 2 full passes**: each pass stops at the human-approval node and you approve or reject once. Record: did your approval decisions agree with the verify node? Did the approval node catch anything the verify node missed?

How to Measure Results

Metric	Exp 1 (Explicit graph)	Exp 2 (Parallel)	Exp 3 (Human-in-the-loop)
Structural visibility	How many implicit edges did you find?	Can shared state support parallel subtasks?	Can the rollback edge pinpoint the problem layer?
Failure localization	Can you point at which edge is wrong?	When a subtask fails, can you locate which one?	When approval rejects, can you name the layer?
Coordination cost	How long did writing the graph take?	Time saved by parallelism vs. coordination overhead	Approval wait time vs. value of problems caught
Observability	Can you now see what happens at each step?	Is every subtask's state visible?	Are approval requests written clearly?
Reliability	Does the graph description match actual runs?	Is the fan-in merge standard sound?	Do the timeout/escalation rules actually fire?

What to Submit

- `graph.md` (Experiment 1's complete graph description: mermaid diagram + node table + edge table + shared state fields + routing rules)
- The list of implicit edges you found in Experiment 1 (at least one)

- Experiment 2's fan-out/fan-in rules and one parallel run record (time/cost/quality comparison)
- Experiment 3's rollback-edge rules, approval-node format, and 2 rounds of human-in-the-loop records
- Final retro: as you moved from loop to graph, how did your way of working change? Which tasks deserve a graph, and which don't?

Related Lectures

- Lecture 14 — From Single Loops to Graph Engineering
- Lecture 13 — From Manual Prompting to Autonomous Loops (your loop is a node in a graph; this project unfolds that node's internal structure)
- Lecture 09 — Why Agents Declare Victory Too Early (why the verify node must be independent of the implement node — a structural problem, not a prompt problem)
- Lecture 11 — Why Observability Belongs Inside the Harness (the more complex the graph, the more you need to see what each node is doing)

English Resource Library

This folder turns the course methods into copy-ready templates and compact references you can use in a real repository.

When To Use It

Start here when you want Codex, Claude Code, or another coding agent to work across multiple sessions without constantly re-deriving setup, status, and scope.

It is especially useful when:

- work spans multiple sessions
- features are numerous and easy to leave half-finished
- agents tend to declare victory too early
- startup steps are rediscovered every time

Start Here

For a minimal setup, begin with:

- root instructions: `templates/AGENTS.md` or `templates/CLAUDE.md`
- feature state: `templates/feature_list.json`
- progress log: `templates/claude-progress.md`

The filename is a historical course convention. This is a generic, repository-local session progress log: any coding agent can use it, but agents only read or update it when their instructions explicitly tell them to do so.

- bootstrap script reference: `docs/en/resources/templates/init.sh`

Then add:

- session handoff: `templates/session-handoff.md`

- clean-exit checklist: `templates/clean-state-checklist.md`
- evaluator rubric: `templates/evaluator-rubric.md`

If you want the fuller OpenAI-style repository structure from the "Harness engineering" post, use the advanced pack:

- `openai-advanced/index.md`

Library Structure

- `templates/`: templates to copy into a real repo
- `reference/`: method notes, startup flow, and failure-mode maps
- `openai-advanced/`: advanced repo skeleton, system-of-record docs, and agent-first governance templates

---

Recommended Minimal Pack

- `AGENTS.md` or `CLAUDE.md`
- `feature_list.json`
- `claude-progress.md` (generic session progress log; historical filename)
- `init.sh`

Those four files are enough to make most agent workflows noticeably more stable.

When the repo grows into a longer-running system with multiple domains, active plans, quality scoring, and reliability policies, move up to the `openai-advanced/` pack instead of stretching the minimal pack too far.

Template Guide

These templates are ready to copy into your own project. Each one serves a specific purpose in the agent's workflow. Edit the contents to match your project's commands, paths, feature names, and verification steps.

How to Get Started

Copy these four files into your project root first:

1. `AGENTS.md` or `CLAUDE.md`
2. `init.sh`
3. `claude-progress.md`
4. `feature_list.json`

Add the remaining files as your project grows.

AGENTS.md

The root instruction file. This is the first thing the agent reads when it starts a session. It defines the operating rules: what to do before writing code, how to work, and how to wrap up.

How to use it:

- Copy to your project root directory
- Replace the startup workflow steps with your actual project paths and commands
- Adjust the working rules to match your team's conventions
- Keep the definition of done section — it's the most important part

What it does for the agent:

- Tells it to read progress and feature state before starting work

- Forces it to work on one feature at a time
- Requires evidence before marking anything as done
- Defines what a clean end-of-session looks like

Use `AGENTS.md` for Codex or other agents. Use `CLAUDE.md` if you're working with Claude Code — the structure is the same, just formatted for Claude's instruction style.

---

init.sh

The startup script. Runs dependency installation, verification, and prints the start command — all in one shot.

How to use it:

- Copy to your project root
- Edit these three variables at the top:
  - `INSTALL_CMD` — your dependency install command (e.g. `npm install`, `pip install -r requirements.txt`)
  - `VERIFY_CMD` — your basic verification command (e.g. `npm test`, `pytest`)
  - `START_CMD` — your dev server start command (e.g. `npm run dev`)
- Make it executable: `chmod +x init.sh`

What it does:

1. Prints the current directory (so you can confirm it's running in the right place)
2. Installs dependencies
3. Runs the verification command
4. Prints the start command (or runs it if `RUN_START_COMMAND=1` is set)

If verification fails, the agent should stop and fix the baseline before doing anything else.

claude-progress.md

The progress log. Every session writes to this file, and every new session reads it first.

How to use it:

- Copy to your project root
- Fill in the "Current Verified State" section with your project's info
- After each session, update the session record

What each field means:

- **Current Verified State** — the single source of truth for where the project stands
  - `Repository root directory` — where the project lives
  - `Standard startup path` — the command to get the project running
  - `Standard verification path` — the command to run tests
  - `Highest priority unfinished feature` — what the next session should work on
  - `Current blocker` — anything that's stuck
- **Session Record** — one entry per session
  - `Goal` — what you planned to do
  - `Completed` — what actually got done
  - `Verification run` — what tests were executed
  - `Evidence recorded` — what proof was captured
  - `Commits` — what was committed
  - `Known risks` — what might be broken
  - `Next best action` — where the next session should start

feature_list.json

The feature tracker. A machine-readable list of every feature the agent needs to implement, along with its status, verification steps, and evidence.

How to use it:

- Copy to your project root
- Replace the example features with your own
- Each feature needs:
  - `id` — a short unique identifier
  - `priority` — integer, lower = higher priority
  - `area` — which part of the app (e.g. "chat", "import", "search")
  - `title` — short description
  - `user_visible_behavior` — what the user should see when it works
  - `status` — one of `not_started`, `in_progress`, `blocked`, `passing`
  - `verification` — step-by-step instructions to confirm it works
  - `evidence` — recorded proof that verification passed (filled in by the agent)
  - `notes` — any extra context

Status rules:

- `not_started` — hasn't been touched
- `in_progress` — the one feature currently being worked on (only one at a time)
- `blocked` — can't proceed due to a documented issue
- `passing` — verification passed and evidence is recorded

The agent should only have one feature in `in_progress` at any time.

---

session-handoff.md

A compact handoff note between sessions. Use this when a session ends and you want the next one to pick up quickly.

How to use it:

- Copy to your project root
- Fill it out at the end of each session (or have the agent fill it out)

What each section covers:

- **Currently verified** — what's confirmed working and what verification was run
- **Changes this session** — what code or infrastructure changed
- **Still broken or unverified** — known issues and risky areas
- **Next best action** — what the next session should do, and what not to touch
- **Commands** — startup, verification, and debug commands for quick reference

This file is optional for small sessions. It becomes important when sessions are long or when the project has multiple active areas.

---

clean-state-checklist.md

A checklist to run through before ending each session. Makes sure the repo is in a good state for the next session to start cleanly.

How to use it:

- Copy to your project root
- Run through it before you close a session
- The agent should also check these items as part of its end-of-session routine

What it checks:

- Standard startup still works
- Standard verification still runs
- Progress log is updated
- Feature list reflects actual state (no false `passing` entries)
- No half-finished work left unrecorded
- Next session can continue without manual fixes

evaluator-rubric.md

A scorecard for reviewing agent output quality. Use this after a session or at project milestones to evaluate whether the work meets the bar.

How to use it:

- Copy to your project root
- After a session (or a set of sessions), score the agent's work across six dimensions
- Each dimension is scored 0-2

The six dimensions:

1. **Correctness** — does the implementation match the target behavior?
2. **Verification** — were the required checks actually run, with evidence?
3. **Scope discipline** — did the agent stay within the selected feature?
4. **Reliability** — does the result survive a restart or re-run?
5. **Maintainability** — is the code and documentation clear enough for the next session?
6. **Handoff readiness** — can a new session continue using only repo artifacts?

Conclusion options:

- Accept — meets the bar
- Revise — needs fixes before accepting
- Block — fundamental issues that need to be resolved first

Important: the evaluator needs tuning. Out of the box, agents are poor self-judges — they identify issues then talk themselves into approving. You will need to iterate:

1. Run the evaluator on a completed sprint.
2. Compare its scores against your own human judgment.
3. Where they diverge, make the rubric more specific about pass/fail criteria.
4. Re-run and check alignment.
5. Repeat until the evaluator consistently matches human review.

Plan for 3-5 tuning rounds. Record each change so you can track what improved alignment.

---

quality-document.md

A quality snapshot that grades each product domain and architectural layer in your project. Tracks codebase health over time, not just individual session output.

How to use it:

- Copy to your project root
- Before starting a session: read it to understand where the codebase is weakest
- After a session: update grades based on what changed
- Over time: compare snapshots to see which harness changes actually improved codebase health

What it grades:

- **Product domains** (e.g., document import, Q&A flow, indexing): each domain gets a grade (A-D) across verification status, agent legibility, test stability, and key gaps
- **Architectural layers** (e.g., main process, preload, renderer, services): each layer gets a grade for boundary enforcement and agent legibility

Why it matters:

The evaluator rubric scores individual agent outputs. The quality document scores the codebase itself. They answer different questions:

- Evaluator rubric: "Did the agent do good work this session?"
- Quality document: "Is the project getting stronger or weaker over time?"

When to update:

- After each significant session
- Before benchmark comparisons
- After cleanup or simplification passes
- When onboarding a new agent or model to the project

Harness simplification tie-in:

The quality document also supports harness simplification. Every harness component encodes an assumption about what the model cannot do. As models improve, these assumptions go stale. To check whether a component is still needed:

1. Take a quality document snapshot.
2. Remove one harness component.
3. Run the benchmark task suite.
4. Take another snapshot.
5. Compare — if grades didn't drop, the component was overhead. If they did, restore it.

Coding Agent Startup Flow

Use this at the beginning of every session after initialization is complete.

Fixed Startup Template

1. Run `pwd` and confirm the repository root.
2. Read `claude-progress.md`.
3. Read `feature_list.json`.
4. Review recent commits with `git log --oneline -5`.
5. Run `./init.sh`.
6. Run a baseline smoke or end-to-end path.
7. If the baseline is broken, fix that first.
8. Select the highest-priority unfinished feature.
9. Work only on that feature until it is verified or explicitly blocked.

Why This Order Matters

- `pwd` prevents accidental work in the wrong directory.
- progress and feature files recover durable state before new edits begin.
- recent commits explain what changed most recently.
- `init.sh` standardizes startup instead of relying on memory.
- baseline verification catches broken starting states before new work hides them.

End-Of-Session Mirror

The same session should end by:

1. recording progress
2. updating feature state
3. writing a handoff if needed
4. committing safe work
5. leaving a clean restart path

English Reference

These notes explain how to use the templates as a working harness instead of a loose pile of files.

Internal Reference Notes

- `method-map.md`: map common long-running failure modes to the artifact or policy that addresses them first
- `initializer-agent-playbook.md`: what the initializer should leave behind before feature work starts
- `coding-agent-startup-flow.md`: fixed session-start flow for later coding runs
- `prompt-calibration.md`: how to keep root instructions sharp without making them bloated and brittle

---

Primary Articles

This list is intentionally narrow. A harness means the execution system around the model: the agent loop, tool execution, sandboxing, state, context, verification, termination, orchestration, and observability. General prompt engineering or broad agent-framework articles do not belong in the primary list.

The original three articles remain the backbone of the course:

- OpenAI: Harness engineering: leveraging Codex in an agent-first world (2026-02-11): agent-first repositories, repo-local context, custom linting, and structural guardrails.
- Anthropic: Effective harnesses for long-running agents (2025-11-26): initializer agent, coding agent, feature list, progress log, and handoff across context windows.
- Anthropic: Harness design for long-running application development (2026-03-24): planner / generator / evaluator roles, context resets, harness simplification, and stale assumptions.

Only a few highly relevant 2026 articles are added:

- OpenAI: Unrolling the Codex agent loop (2026-01-23): the Codex runtime harness, tool calls, context growth, and loop termination.
- Anthropic: Demystifying evals for AI agents (2026-01-09): evaluating the model and harness together, and distinguishing evaluation harnesses from agent harnesses.
- LangChain: Improving Deep Agents with harness engineering (2026-02-17): holding the model fixed while improving system prompts, tools, middleware, tracing, and self-verification to move a coding agent from Top 30 to Top 5 on Terminal Bench 2.0.
- Thoughtworks / Martin Fowler: Harness engineering for coding agent users (2026-04-02): coding-agent user harnesses as feedforward guides and feedback sensors, with deterministic and inferential controls.
- Cursor: Continually improving our agent harness (2026-04-30): treating the harness as a continuously improved product system with offline evals, online metrics, tool-error taxonomy, model-specific tuning, and mid-chat model switching.

2026 Extended References

These are not core course sources, but they are useful when designing specific harness modules. This section only keeps sources whose body directly covers the agent loop, tool execution, context management, verification, sandboxing, control layers, or regression governance. Pure agent products, platform announcements, team case studies, and benchmarks are excluded.

- OpenAI: Unlocking the Codex harness: how we built the App Server (2026-02-04): the harness as a reusable App Server protocol with thread lifecycle, resume, fork, diffs, and client integrations.
- OpenAI Developers: Run long horizon tasks with Codex (2026-02-23): durable project memory, milestone validation, and done-when examples for long-running tasks.
- OpenAI: The next evolution of the Agents SDK (2026-04-15): model-native harnesses, sandbox execution, and file/command execution.
- OpenAI: An open-source spec for Codex orchestration: Symphony (2026-04-27): turning an issue tracker or Linear board into a multi-agent control plane.
- Anthropic: Building a C compiler with a team of parallel Claudes (2026-02-05): parallel agent teams, task locks, git synchronization, container isolation, and autonomous loops.

- Anthropic: Scaling Managed Agents: Decoupling the brain from the hands (2026-04-08): a meta-harness view that separates session, harness, and sandbox as swappable interfaces.
- Anthropic: An update on recent Claude Code quality reports (2026-04-23): reasoning effort, context pruning, and system prompts as harness changes that need regression governance.
- LangChain: Context Management for Deep Agents (2026-01-28): filesystem offloading, tool-call truncation, summarization, and targeted evals for context-management harnesses.
- LangChain: Tuning Deep Agents to Work Well with Different Models (2026-04-29): model-specific harness profiles for prompts, tool names, middleware, and subagent configuration.
- LangChain: Continual learning for AI agents (2026-04-05): splitting agent improvement into model, harness, and context layers, powered by traces.
- Microsoft: Agent Harness in Agent Framework (2026-03-12): shell/filesystem harnesses, approval flow, hosted shell execution, and context compaction.
- Google: Announcing ADK for Java 1.0.0 (2026-03-30): plugins, event compaction, HITL, session/memory services, and A2A as reusable harness primitives.
- GitHub: Automate repository tasks with GitHub Agentic Workflows (2026-02-13): GitHub Actions as an agentic workflow runner with safe outputs, sandboxing, permissions, and review.
- AWS: AI agents in enterprises: Best practices with Amazon Bedrock AgentCore (2026-02-03): enterprise harness layers across Runtime, Memory, Gateway, Identity/Policy, Observability, and Evaluations.
- Stripe: Minions: Stripe's one-shot, end-to-end coding agents (2026-02-09) and Part 2 (2026-02-19): devbox isolation, custom agent harnesses, blueprint state machines, rule files, MCP tool curation, security controls, and pre-push/CI feedback loops.
- Cognition: What We Learned Building Cloud Agents (2026-04-23): VM isolation, session snapshot/resume, orchestration, governance, audit logging, and integrations for cloud-agent runtimes.
- Cognition: Multi-Agents: What's Actually Working (2026-04-22): generator-verifier loops, clean-context reviewers, smart-friend routing, manager-child coordination, and cross-agent communication boundaries.

- Addy Osmani: Loop Engineering (2026-06-07): the complete loop engineering framework from `/goal` to six primitives — automations, worktrees, skills, connectors, sub-agents, external state — plus the four silent costs.
- Replit: Decision-Time Guidance: Keeping Replit Agent Reliable (2026-01-20, updated 2026-01-23): a lightweight classifier injects short situational guidance at the decision point instead of stuffing all rules into the system prompt.
- Vercel: How we made v0 an effective coding agent (2026-01-07): dynamic system prompts, a streaming rewrite layer, and deterministic/model-driven autofixers.
- Vercel: Introducing deepsec (2026-05-04): a security-focused coding-agent harness with scan, investigate, revalidate, enrich, export, plugin, and refusal-checker steps.
- Sourcegraph: CodeScaleBench (2026-03-03): an eval/tooling harness reference covering MCP tool adoption, tool-use transcripts, benchmark QA, verifier/reproducibility gates, and prompt/preamble iteration.

Strictly 2025-only general references are excluded from the primary list. The original 2025 Anthropic harness article remains because it is a foundation source for the course.

- ETH Zurich: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?: Study of repository context files; task success, inference cost, and the recommendation to keep requirements minimal. See the abstract and conclusion.
- On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents: gpt-5.2-codex; 10 repos / 124 PR tasks; Table 1.

Suggested Reading Order

1. `method-map.md`
2. `initializer-agent-playbook.md`
3. `coding-agent-startup-flow.md`
4. `prompt-calibration.md`
5. OpenAI Harness engineering
6. Anthropic Effective harnesses
7. Anthropic Harness design for long-running application development

8. OpenAI Codex agent loop
9. Anthropic agent evals
0. LangChain Improving Deep Agents
1. Thoughtworks / Martin Fowler Harness engineering for coding agent users
2. Cursor Continually improving our agent harness

Initializer Agent Playbook

Use this playbook for the first serious session in a repository, before incremental feature work begins.

Goal

Create a stable operating surface so later sessions can implement behavior without re-deriving startup commands, current status, or task boundaries.

Required Outputs

The initializer should leave behind at least these artifacts:

- a root instruction file such as `AGENTS.md` or `CLAUDE.md`
- a machine-readable feature surface such as `feature_list.json`
- a durable progress artifact such as `claude-progress.md`
- a standard startup helper such as `init.sh`
- an initial safe commit that captures the baseline scaffold

Checklist

1. Define the standard startup path.
2. Define the standard verification path.
3. Create the progress log and record the starting state.
4. Decompose the work into explicit features with statuses.
5. Create the first clean baseline commit.

Success Test

A fresh session with no prior chat context should be able to answer:

- what this repository does
- how to start it
- how to verify it
- what is unfinished
- what the next best step is

Method Map

This table maps the most common long-running coding-agent failure modes to the artifact or operating rule that usually fixes them first.

Failure mode	What it looks like in practice	Primary fix	Supporting artifact
Cold-start confusion	A new session spends most of its time rediscovering setup and status	Make the repository the system of record	`claude-progress.md`
Scope sprawl	The agent starts several features and finishes none of them cleanly	Restrict active scope	`feature_list.json`
Premature completion	The agent claims done after code edits but before runnable proof	Bind completion to evidence	`clean-state-checklist.md`
Fragile startup	Every session re-learns how to boot the project	Standardize setup and verification	`init.sh`
Weak handoff	The next session cannot tell what is verified, broken, or next	End with an explicit handoff	`session-handoff.md`
Subjective review	Review quality depends on taste or memory	Score output with fixed categories	`evaluator-rubric.md`

Operating Principle

Add the smallest artifact that directly addresses the observed failure mode. Avoid solving every reliability problem by dumping more text into one global instruction file.

Prompt Calibration

Root instructions should define the operating frame, not every possible move.

Keep In The Root File

- repository purpose and scope
- startup path
- verification path
- non-negotiable constraints
- required state artifacts
- end-of-session rules

Move Out Of The Root File

- long historical edge cases
- topic-specific implementation details
- local architecture notes that belong near the code
- examples that only apply to one subsystem

Working Rule

The root file should help a fresh session orient itself quickly. If the file is becoming a dumping ground for every past failure, split the detail into smaller documents and link to them instead.

OpenAI Advanced Pack

This folder packages the more opinionated repository shape described in OpenAI's "Harness engineering: leveraging Codex in an agent-first world" article into copy-ready starter files.

Use this pack when the minimal harness is no longer enough and your repository now needs:

- a short routing-style `AGENTS.md`
- durable system-of-record docs inside the repo
- active and completed execution plans
- explicit product, reliability, security, and frontend policy files
- quality scoring by product domain and architectural layer
- model-friendly reference material folders
- standard operating procedures for architecture, knowledge capture, and runtime validation

---

Included Starter Layout

The starter pack under `repo-template/` mirrors the structure below:

```text
AGENTS.md
ARCHITECTURE.md
docs/
├── design-docs/
│   ├── index.md
│   └── core-beliefs.md
├── exec-plans/
│   ├── active/
│   ├── completed/
│   └── tech-debt-tracker.md
├── generated/
│   └── db-schema.md
├── product-specs/
│   ├── index.md
│   └── new-user-onboarding.md
├── references/
│   ├── design-system-reference-llms.txt
│   ├── nixpacks-llms.txt
│   └── uv-llms.txt
├── DESIGN.md
├── FRONTEND.md
├── PLANS.md
├── PRODUCT_SENSE.md
├── QUALITY_SCORE.md
├── RELIABILITY.md
└── SECURITY.md
```

How To Adopt It

1. Start from the minimal pack if your repo is still small.
2. Copy the files in `repo-template/` into your own repository once you need stronger structure.
3. Keep `AGENTS.md` short. Treat it as a router into the deeper docs, not as an encyclopedia.
4. Update the quality, reliability, and plan docs as part of normal work, not as a separate cleanup day.

5. Keep generated artifacts and external references explicit so agents can find them without relying on chat history.

SOP Library

The `sops/` folder turns the article's diagrams into step-by-step operating procedures:

- layered domain architecture setup
- encode unseen knowledge into the repository
- local observability stack and feedback-loop workflow
- Chrome DevTools validation loop for UI work

Design Principles

- Short entrypoint, deeper linked docs
- Repository as system of record
- Mechanical checks beat remembered rules
- Plans and quality history live beside the code
- Cleanup and simplification are first-class responsibilities

This pack is intentionally opinionated, but it should still be adapted to your project rather than copied blindly.

AGENTS.md

This repository is optimized for long-running coding-agent work. Keep this file short. Use it as the routing layer into the system-of-record docs, not as a giant instruction dump.

Startup Workflow

Before changing code:

1. Confirm the repo root with `pwd`.
2. Read `ARCHITECTURE.md` for the current system map and hard dependency rules.
3. Read `docs/QUALITY_SCORE.md` to see which domains or layers are weakest.
4. Read `docs/PLANS.md`, then open the active plan you are working from.
5. Read the relevant product spec in `docs/product-specs/`.
6. Run the standard bootstrap and verification path for this repo.
7. If baseline verification is failing, repair the baseline before adding scope.

Routing Map

- `ARCHITECTURE.md`: domain map, layer model, dependency rules
- `docs/design-docs/index.md`: design decisions and core beliefs
- `docs/product-specs/index.md`: current product behaviors and acceptance targets
- `docs/PLANS.md`: plan lifecycle and execution-plan policy
- `docs/QUALITY_SCORE.md`: product-domain and layer health
- `docs/RELIABILITY.md`: runtime signals, benchmarks, and restart expectations
- `docs/SECURITY.md`: secrets, sandbox, data, and external-action rules
- `docs/FRONTEND.md`: UI constraints, design system rules, accessibility checks

Working Contract

- Work from one bounded plan or feature slice at a time.
- Do not mark work done from code inspection alone; runnable evidence is required.
- If you change behavior, update the matching product, plan, or reliability docs in the same session.
- If you see repeated review feedback, promote it into a mechanical rule, check, or linter instead of re-explaining it in chat.
- Keep generated material in `docs/generated/` and source references in `docs/references/`.
- Prefer adding small, current docs over growing this file.

Definition Of Done

A change is done only when all of the following are true:

- target behavior is implemented
- required verification actually ran
- evidence is linked from the relevant plan or quality document
- affected docs remain current
- the repository can restart cleanly from the standard startup path

End Of Session

Before ending a session:

1. Update the active execution plan.
2. Update `docs/QUALITY_SCORE.md` if any domain or layer meaningfully changed.
3. Record new debt in `docs/exec-plans/tech-debt-tracker.md` if you deferred it.

4. Move finished plans to `docs/exec-plans/completed/` when appropriate.
5. Leave the repo in a restartable state with a clear next action.

ARCHITECTURE.md

This file is the top-level map of the system. It should stay concise and point to deeper documents when needed.

System Shape

- Product: `[replace with product name]`
- Primary user workflow: `[replace with main workflow]`
- Runtime surfaces: `[desktop / web / cli / services / workers]`
- Source of truth for product behavior: `docs/product-specs/`

Domain Map

Domain	Purpose	Primary Entry Points	Related Spec
`[domain-a]`	`[what it owns]`	`[modules / routes / commands]`	`[spec path]`
`[domain-b]`	`[what it owns]`	`[modules / routes / commands]`	`[spec path]`

Layer Model

Use a fixed directional model so agents do not invent ad hoc architecture:

`Types → Config → Repo → Service → Runtime → UI`

Cross-cutting concerns should enter through explicit provider or adapter boundaries instead of reaching across layers directly.

Hard Dependency Rules

- Lower layers must not depend on higher layers.
- UI must not bypass runtime or service contracts.
- Data access must enter through repositories or equivalent adapters.
- Shared utilities must remain generic and must not accumulate domain logic.
- New dependencies should be justified in the matching plan or design doc.

Cross-Cutting Interfaces

Concern	Approved Boundary	Notes
Logging and tracing	`[provider / utility path]`	`[structured only, no ad hoc console use]`
Auth	`[provider path]`	`[token/session rules]`
External APIs	`[client or provider path]`	`[rate limit / retry guidance]`
Feature flags	`[flag boundary]`	`[ownership]`

Current Hot Spots

- `[area that is hardest for agents to change safely]`
- `[area with weak boundaries or fragile tests]`

Change Checklist

When you touch architecture-relevant code:

1. Update this file if the domain map or allowed boundaries changed.
2. Update the related design doc in `docs/design-docs/` if the reasoning changed.
3. Add or update an executable check if the rule should be enforced mechanically.

Core Beliefs

- The repository is the system of record for agents.
- `AGENTS.md` is a router, not an encyclopedia.
- Verification evidence matters more than confidence.
- One bounded task is better than many half-finished tasks.
- Repeated human feedback should become reusable harness rules.
- Cleanup and simplification are part of shipping, not afterthoughts.
- If an agent cannot discover a fact in-repo, treat that fact as operationally unavailable.

Design Docs Index

Use this index as the discoverable map of design history.

Accepted

- `core-beliefs.md`: agent-first operating beliefs and durable project norms

Proposed

- `[add new design doc paths here]`

Deprecated

- `[move old or superseded design docs here with replacement links]`

Maintenance Rules

- Every design doc should have an owner or update trigger.
- Remove stale docs or mark them deprecated instead of letting them drift.
- Link active execution plans to the design docs they depend on.

DESIGN.md

This file is the design entrypoint. Keep it brief and use it to route into the more detailed files under `docs/design-docs/`.

Purpose

Record durable product and system design decisions that should survive beyond a single chat, sprint, or reviewer memory.

Read This When

- you need the current design philosophy
- you are about to introduce a new pattern
- you need to know which design decisions are settled versus still open

Canonical Design Docs

- `docs/design-docs/index.md`: index of accepted, proposed, and deprecated docs
- `docs/design-docs/core-beliefs.md`: project-wide agent-first beliefs

Design Rules

- Keep design docs small and current.
- Prefer one doc per decision area.
- Link design docs from plans and specs when a change depends on them.
- If a design rule becomes operationally critical, promote it into an automated check or update `ARCHITECTURE.md`.

Active Plans

Keep one markdown file per active execution plan in this folder.

Suggested filename pattern:

- `YYYY-MM-DD-short-topic.md`

Each active plan should be current enough that a fresh agent session can resume work from the repository alone.

Completed Plans

Move finished plans here instead of deleting them. Completed plans are part of the repository memory surface and help later agent runs understand why the code looks the way it does.

Tech Debt Tracker

Use this file for debt that is real, acknowledged, and intentionally deferred.

Date	Area	Debt	Why Deferred	Risk	Next Trigger
YYYY-MM-DD	`[area]`	`[debt]`	`[reason]`	`[risk]`	`[when to revisit]`

FRONTEND.md

This file defines stable frontend expectations so agents do not invent UI patterns unpredictably.

UI Principles

- Optimize for clarity before novelty.
- Keep interaction flows discoverable and restartable.
- Prefer a small number of reusable components over one-off variants.
- Accessibility checks are part of normal verification, not polish work.

Guardrails

- Document the design system or component library in `docs/references/`.
- Record key user-facing states: empty, loading, success, error, retry.
- Keep copy, keyboard behavior, and visual hierarchy consistent across flows.
- When a UI bug is fixed, add or update the matching validation step.

Verification Expectations

- Capture evidence for critical user journeys.
- Record browser or runtime validation steps in the relevant plan.
- If visual regressions are common, standardize screenshot or DOM checks.

Database Schema

Use this folder for generated or derived artifacts that agents should be able to inspect without reverse-engineering them from code.

Source

- Generated from: `[command or source path]`
- Last refreshed: `YYYY-MM-DD`

Notes

- Do not hand-edit generated sections.
- Regenerate this file when the underlying schema changes.

PLANS.md

This file defines how execution plans are created, updated, completed, and archived.

When A Plan Is Required

Create an execution plan when work:

- spans more than one session
- changes more than one subsystem
- has non-trivial verification or rollout risk
- depends on open decisions that should be logged

Plan Locations

- `docs/exec-plans/active/`: plans currently driving work
- `docs/exec-plans/completed/`: finished plans kept for future agent context
- `docs/exec-plans/tech-debt-tracker.md`: deferred work and follow-ups

Minimum Plan Sections

- objective
- scope and out-of-scope
- verification path
- risks and blockers
- progress log
- open decisions

Operating Rules

- One active plan should have one clearly owned current step.
- Update the plan as work progresses; do not treat it as static prose.
- If a decision changes implementation direction, record it in the plan.
- Move finished plans to `completed/` so agents can still discover prior context.

PRODUCT_SENSE.md

This file captures durable product judgment that agents cannot infer reliably from code alone.

Product Core

- Primary user: `[replace]`
- Job to be done: `[replace]`
- Main frustration to remove: `[replace]`
- Quality bar for acceptance: `[replace]`

Product Rules

- Favor user-visible reliability over feature count.
- Treat ambiguous behavior as a spec gap, not as permission to guess.
- If implementation changes what users see or trust, update the matching spec.
- Use product specs for concrete flows, and use this file for cross-cutting product priorities.

No-Go Patterns

- Hidden destructive actions
- Silent failure without user feedback
- Unclear source of truth for visible state
- Features that cannot be explained in one sentence

Product Specs Index

Use this folder for current user-facing behavior specs.

Active Specs

- `new-user-onboarding.md`

Rules

- Specs should describe user-visible behavior and acceptance criteria.
- If implementation diverges from the spec, update one of them in the same session.
- Keep this index current so a fresh agent can discover product scope quickly.

New User Onboarding

Goal

Describe the first-run experience a new user should have.

Entry Conditions

- `[state before the flow starts]`

User Flow

1. `[step one]`
2. `[step two]`
3. `[step three]`

Acceptance Criteria

- `[observable outcome]`
- `[observable outcome]`
- `[observable outcome]`

Failure States

- `[recoverable error and user feedback]`
- `[blocked state and fallback]`

QUALITY_SCORE.md

This document tracks whether the repository is getting stronger or weaker over time.

Grading Scale

- `A`: verified, legible, stable, boundaries enforced
- `B`: working with minor gaps
- `C`: partially working, notable confusion or instability
- `D`: broken, unsafe, or structurally unclear

Product Domains

Domain	Grade	Verification	Agent Legibility	Test Stability	Key Gaps	Last Updated
`[domain-a]`	-	-	-	-	-	-
`[domain-b]`	-	-	-	-	-	-
`[domain-c]`	-	-	-	-	-	-

Architectural Layers

Layer	Grade	Boundary Enforcement	Agent Legibility	Key Gaps	Last Updated
Types	-	-	-	-	-
Services	-	-	-	-	-
Runtime	-	-	-	-	-
UI	-	-	-	-	-

Benchmark Snapshots

Date	Harness Variant	Completion Rate	Retries	Defects Before Review	Notes
YYYY-MM-DD	`[baseline / improved / simplified]`	-	-	-	-

Simplification Log

Date	Component Removed	Outcome	Decision
YYYY-MM-DD	`[component]`	`[degraded / unchanged]`	`[restore / keep removed]`

RELIABILITY.md

This file defines how the system proves it is healthy and restartable.

Standard Paths

- Bootstrap: `[command]`
- Verification: `[command]`
- Start app or service: `[command]`
- Debug or inspect runtime: `[command]`

Required Runtime Signals

- structured logs for startup and critical flows
- health checks for key services
- trace or timing data for slow paths when available
- user-visible error states for recoverable failures

Golden Journeys

- `[journey 1]`
- `[journey 2]`
- `[journey 3]`

Each golden journey should have a repeatable verification path and clear failure signals.

Reliability Rules

- No feature is complete if the system cannot restart cleanly afterward.
- Runtime failures should be diagnosable from repo-local signals.
- If a repeated failure mode appears, add a benchmark or guardrail for it.
- Cleanup is part of reliability, not a separate concern.

SECURITY.md

This file defines the security and safety rules that agents must not guess at.

Secrets And Credentials

- Never hard-code secrets in source or docs.
- Document approved secret-loading paths here.
- Redact tokens, API keys, and personal data from logs and screenshots.

Untrusted Input

- Treat external content as untrusted until validated.
- Record allowed fetch or execution boundaries here.
- If prompt injection or command injection risk exists, document the guardrail.

External Actions

- List which actions require explicit approval.
- Record any production or destructive commands that agents must not run by default.
- Prefer sandbox-safe workflows for debugging and verification.

Dependency And Review Rules

- New dependencies need justification in the active plan.
- Security-sensitive changes require explicit verification steps.
- Repeated security review comments should become checks, not tribal knowledge.

Advanced Repo Template

Copy this starter into a real repository when you want an OpenAI-style agent-first documentation surface instead of only a minimal harness.

Copy Order

1. Copy `AGENTS.md` and `ARCHITECTURE.md` to the repo root.
2. Copy the whole `docs/` tree.
3. Fill in `docs/PRODUCT_SENSE.md`, `docs/QUALITY_SCORE.md`, and `docs/RELIABILITY.md` first.
4. Add your first active plan under `docs/exec-plans/active/`.
5. Keep the entrypoint files short and route detail into the linked docs.

What This Template Optimizes For

- durable repo-local context
- progressive disclosure instead of one giant instruction file
- explicit plan lifecycle
- quality tracking over time
- readable boundaries for agents and humans

Treat every file here as a starter. Replace placeholders, examples, and sample commands with your real project specifics before relying on it.

SOP: Chrome DevTools Validation Loop

Use this SOP when UI work depends on actual runtime interaction and screenshots, DOM state, and console output matter more than code inspection alone.

Goal

Turn UI validation into a repeatable interaction loop the agent can run until the journey is clean.

Core Loop

1. Select the target page or app instance.
2. Clear stale console noise.
3. Capture the BEFORE state.
4. Trigger the UI path.
5. Observe runtime events during interaction.
6. Capture the AFTER state.
7. Apply the fix and restart the app if needed.
8. Re-run validation until the journey is clean.

Required Inputs

- a stable startup command
- a reproducible UI journey
- a way to snapshot DOM, console, or screenshots
- a rule for what counts as "clean"

Execution SOP

1. Write the target journey in the active plan.
2. Define success in observable terms: text present, button enabled, error gone, console clean, request succeeded.
3. Snapshot the initial state before interaction.
4. Trigger exactly one path at a time.
5. Record runtime events, DOM changes, and visible output.
6. If the journey fails, fix the smallest responsible layer and restart.
7. Re-run the same path and compare BEFORE/AFTER evidence.

Clean Criteria

- intended visible state is present
- unexpected errors are absent
- console noise is understood or cleared
- rerunning the same path gives the same result

Repo Artifacts To Update

- active execution plan
- `docs/RELIABILITY.md` if the journey becomes a golden path
- product spec if the visible behavior changed

SOP: Encode Unseen Knowledge Into The Repo

Use this SOP when important context still lives in Google Docs, chat threads, tickets, or people's heads.

Goal

Make agent-invisible knowledge discoverable in the codebase so a fresh session can act on it without relying on prior conversation.

Trigger Signals

- The agent keeps asking how the system works.
- Humans say "we decided this in Slack" or "follow what X said last week."
- Reviews reference product or security rules that are not written in-repo.
- New sessions repeat discovery work that should already be settled.

Execution SOP

1. List the invisible knowledge sources: docs, chats, tacit team rules, verbal decisions.
2. For each source, ask: is this architecture, product behavior, security policy, reliability expectation, plan context, or reference material?
3. Encode it into the matching repo artifact:
   - architecture → `ARCHITECTURE.md`
   - product behavior → `docs/product-specs/`
   - design rationale → `docs/design-docs/`
   - execution state → `docs/exec-plans/`
   - repeated external references → `docs/references/`

- quality or reliability expectations → `docs/QUALITY_SCORE.md` or `docs/RELIABILITY.md`

4. Replace vague statements with operationally useful wording.
5. Remove or deprecate stale copies so the repo keeps one discoverable truth.

Good Encoding Rules

- Write for discoverability, not for literary completeness.
- Prefer short documents with clear filenames.
- Link related artifacts together.
- Store durable rules, not meeting transcripts.
- Update the repo in the same session that the decision is made.

Definition Of Done

- A fresh agent can discover the relevant rule without asking a human.
- The same fact is not scattered across multiple contradictory files.
- The new artifact lives close to the code or workflow it governs.

OpenAI Advanced SOPs

These SOPs translate the article's operating patterns into concrete execution playbooks you can follow or adapt.

Included SOPs

- `layered-domain-architecture.md`: establish explicit layers and cross-cutting boundaries
- `encode-knowledge-into-repo.md`: move invisible knowledge from chat, docs, and memory into repo-local files
- `observability-feedback-loop.md`: give agents logs, metrics, traces, and a repeatable debug loop
- `chrome-devtools-validation-loop.md`: use browser automation and snapshots to validate UI behavior until clean

---

How To Use Them

1. Pick the SOP that matches your current bottleneck.
2. Use the checklist to set up the missing artifacts or tooling.
3. Encode the resulting rules into your copied `repo-template/` docs.
4. Convert repeated review comments into checks, scripts, or guardrails.

These are not meant to be followed blindly. They are meant to make the harness more legible, enforceable, and repeatable.

SOP: Layered Domain Architecture

Use this SOP when the agent keeps violating boundaries, duplicating logic across layers, or producing code that becomes hard to review after a few sessions.

Goal

Make domain boundaries explicit enough that agents can move quickly without silently degrading structure.

Target Model

Within a business domain, prefer this directional flow:

`Types → Config → Repo → Service → Runtime → UI`

Cross-cutting concerns should enter through explicit providers or adapters. Shared utils stay outside the domain and should not accumulate domain logic.

Setup Checklist

- Define the current domains in `ARCHITECTURE.md`.
- Write allowed dependency directions in `ARCHITECTURE.md`.
- Record cross-cutting interfaces such as auth, telemetry, and external APIs.
- Add one short note for the hardest current boundary violation.
- Decide what should be enforced mechanically by lint, tests, or scripts.

Execution SOP

1. Map the codebase into domains before touching implementation style.

2. For each domain, identify the allowed layer sequence.
3. Identify all cross-cutting concerns and route them through providers or adapters.
4. Move ambiguous shared logic either into the owning domain or into truly generic utils.
5. Document the rules in `ARCHITECTURE.md`.
6. Add one executable guardrail for the highest-cost violation.
7. Update quality scoring after the change.

Definition Of Done

- A fresh agent can tell which layer owns a change.
- UI code no longer reaches into repo or external side effects directly.
- Cross-cutting concerns have named entry points.
- At least one important boundary is enforced mechanically.

Repo Artifacts To Update

- `ARCHITECTURE.md`
- `docs/QUALITY_SCORE.md`
- `docs/design-docs/` when the rationale changed
- `docs/PLANS.md` or the active execution plan

SOP: Observability Feedback Loop

Use this SOP when debugging is slow, agents keep claiming success without evidence, or runtime behavior is harder to inspect than the code itself.

Goal

Give the agent a local feedback loop over logs, metrics, traces, and runnable workloads so it can reason from execution, not only from code inspection.

Minimum Stack

- application emits structured logs
- application emits metrics and traces when feasible
- local fan-out or collection layer
- query interfaces for logs, metrics, and traces
- repeatable workload or user journey to rerun after each change

Execution SOP

1. Define the golden runtime journeys that matter most.
2. Add structured logs to startup and the critical path.
3. Add metrics for latency, failure counts, or queue depth where useful.
4. Add traces or timing markers for slow or multi-step flows.
5. Make the signals queryable from the local dev environment.
6. Give the agent one repeatable workload or scenario to rerun.
7. Require the loop: query → correlate → reason → implement → restart → rerun → verify.

Debug Session Checklist

- What failed?
- Which signal proves the failure?
- Which layer owns the failure?
- What changed after the fix?
- Did the app restart cleanly?
- Did the same workload pass after rerun?

Definition Of Done

- The agent can explain a failure mode from runtime evidence.
- The same workload can be rerun after each change.
- Restart and rerun are part of the normal task loop.
- Reliability signals are documented in `docs/RELIABILITY.md`.